

教材目录

第一章：量化基础知识

- 第一节 量化投资的概念和优势
- 第二节 量化投资的历史和未来
- 第三节 量化投资的流程与应用

第二章：量化策略入门

- 第一节：MindGo 量化交易平台
- 第二节：MindGo API 介绍
- 第三节：我的第一个量化策略

第三章：Python 编程

- 第一节：Python 介绍
- 第二节：数据类型
- 第三节：条件与循环
- 第四节：函数
- 第五节：numpy
- 第六节：pandas 基础
- 第七节：pandas 进阶

第四章：经典量化策略集锦

- 第一篇：投资高股息股票
- 第二篇：从“二八轮动”中学择时
- 第三篇：网格交易—动态调仓策略
- 第四篇：进军交易系统，从 Dual Thrust 中学“趋势”
- 第五篇：布林强盗，一个霸道的交易系统
- 第六篇：交易系统终结者—海龟交易法则
- 第七篇：向彼得林奇投资大师学习 PEG 选股
- 第八篇：CAPM 模型的应用
- 第九篇：Fama-French 三因子模型应用
- 第十篇：动量类多因子之择时中选股

第五章：量化研究专题

- 第一篇：用 matplotlib 绘图函数实现数据可视化
- 第二篇：运用 Scipy 模块实现统计技术
- 第三篇：10 分钟学会用 Python 做线性回归
- 第四篇：统计套利：利用相关系数进行配对交易
- 第五篇：数据处理专题：去极值、标准化、中性化

第六篇：数据挖掘专题：分类与预测

第七篇：算法交易入门—VWAP

第八篇：Python 实现马克维兹投资组合理论

第九篇：隐马尔科夫模型【机器学习+数据挖掘】

第十篇：机器学习之神经网络入门

第一章：量化基础知识

第一节 量化投资的概念和优势

量化投资的概念

量化投资是指通过数量化方式及计算机程序化发出买卖指令，以获取稳定收益为目的的交易方式。

量化投资区别于定性投资的鲜明特征就是模型，对于量化投资中模型与人的关系，打个比方来说明，我们先看一看医生治病，中医与西医的诊疗方法不同，中医是望、闻、问、切，最后判断出的结果，很大程度上基于中医的经验，定性程度上大一些；西医就不同了，先要病人去拍片子、化验等，这些都要依托于医学仪器，最后得出结论，对症下药。

医生治疗病人的疾病，投资者治疗市场的疾病，市场的疾病是什么？就是错误定价和估值，没病或病得比较轻，市场是有效或弱有效的；病得越严重，市场越无效。投资者用资金投资于低估的证券，直到把它的价格抬升到合理的价格水平上。

但是，定性投资和定量投资的具体做法有些差异，这些差异如同中医和西医的差异，定性投资更像中医，更多地依靠经验和感觉判断病在哪里；定量投资更像是西医，依靠模型判断，模型对于定量投资基金经理的作用就像CT机对于医生的作用。在每一天的投资运作之前，我会先用模型对整个市场进行一次全面的检查和扫描，然后根据检查和扫描结果做出投资决策。

量化投资的优势

量化投资的优势在于纪律性、系统性、及时性、准确性和分散化。

1.纪律性：严格执行投资策略，不是投资者情绪的变化而随意更改。这样可以克服人性的弱点，如贪婪、恐惧、侥幸心理，也可以克服认知偏差。

2.系统性：量化投资的系统性特征包括多层次的量化模型、多角度的观察及海量数据的观察等。多层次模型包括大类资产配置模型、行业选择模型、精选个股模型等。多角度观察主要包括对宏观周期、市场结构、估值、成长、盈利质量、市场情绪等多个角度分析。此外，海量数据的处理能力能够更好地在广大的资本市场捕捉到更多的投资机会，拓展更大的投资机会。

3.及时性：及时快速地跟踪市场变化，不断发现能够提供超额收益的新的统计模型，寻找新的交易机会。

4.准确性：准确客观评价交易机会，克服主观情绪偏差，从而盈利。

5.分散化：在控制风险的前提下，量化投资可以充当分散化投资的工具。表现为两个方面：一是量化投资不断地从历史中挖掘有望在未来重复的历史规律并且加以利用，这些历史规律都是较大概率取胜的策略；二是依靠筛选出股票组合来取胜，而不是一只或几只股票取胜，从投资组合的理念来看也是捕捉大概率获胜的股票，而不是押宝到单个股票。

第二节 量化投资的历史和未来

量化投资的历史与现状

讲起量化投资，就不得不提华尔街的传奇人物——詹姆斯·西蒙斯(James Simons)。

这位慧眼独具的投资巨擘，有着一份足以支撑其赫赫名声的光鲜履历：20岁时获得学士学位；23岁时在加州大学伯克利分校博士毕业；24岁时成为哈佛大学数学系最年轻的教授；37岁时与中国数学家陈省身联合发表了著名论文《典型群和几何不变式》，并开创了著名的陈—西蒙斯理论；40岁时运用基本面分析法设立了自己的私人投资基金；43岁时与普林斯顿大学数学家勒费尔(Henry Laufer)重新开发了交易策略并由此从基本面分析转向数量分析；45岁时正式成立了文艺复兴科技公司，最终笑傲江湖，成为勇执牛耳的投资霸主。

这段看似青云直上的成名之路，再次为世人印证了一个道理——当代的技术创新，其实大多源自跨越学科的资源整合，而非从无到有的发明创造。具体说来，即使睿智如西蒙斯，在最初之时，他也没有直接想到运用量化方法投资，而是和众多投资者一样着眼于外汇市场，但野心勃勃的西蒙斯并不甘于只是简单因循传统的投资策略。随着经验的不断累积，他开始思考，为何不运用他最为熟悉的数学方法来搭建投资模型，从而能够科学精准地预测货币市场的走势变动？这一大胆的跨学科尝试，最终彻底改变了他的人生走向。

通过将数学理论巧妙融合到投资的实战之中，西蒙斯从一个天资卓越的数学家摇身一变，成为了投资界中首屈一指的“模型先生”。由其运作的大奖章基金(Medallion)在1989-2009的二十年间，平均年收益率为35%，若算上44%的收益提成，则该基金实际的年化收益率可高达60%，比同期标普500指数年均回报率高出20多个百分点，即使相较金融大鳄索罗斯和股神巴菲特的操盘表现，也要遥遥领先十几个百分点。最为难能可贵的是，纵然是在次贷危机全面爆发的2008年，该基金的投资回报率仍可稳稳保持在80%左右的惊人水准。西蒙斯通过将数学模型和投资策略相结合，逐步走上神坛，开创了由他扛旗的量化时代，他的骤富神话更让世人对于量化投资有了最为直观而浅显的认识：这能赚钱，而且能赚很多钱。

但金融行业瞬息万变，老天也没有一味垂青这位叱咤风云的“模型先生”。自2012年以来，由西蒙斯掌印的文艺复兴科技公司可谓祸事不断，厄运缠身。其麾下的“文艺复兴机构期货基金”(RIFF)在2011年仅实现盈利率增长1.84%，到2012年，更是破天荒的亏损了3.17%，这一亏损幅度甚至超过了同年巴克莱CTA指数的平均降幅(1.59%)。RIFF主要通过全球范围的期货和远期交易来实现绝对收益，虽属于文艺复兴公司旗下规模较小的基金产品，但作为公司的明星“印钞机”，其回报率竟会一下暴跌至行业平均水平，难免让众人始料不及。到2012年底，RIFF的资产规模已缩减至7.88亿美元，远远低于2011年的40亿美元。到今年10月底，文艺复兴公司最终宣布正式关闭RIFF，一代“文艺”明星RIFF就此戛然而落。

量化投资在海外的历史已有30多年的历史，其投资业绩稳定，市场规模和份额不断扩大、得到越来越多投资者认可。国外量化投资的兴起和发展主要分为三个阶段：

第一阶段（1971-1977）

1971年，世界上第一只被动量化基金由巴克莱国际投资管理公司发行，1977年世界上第一只主动量化基金也是巴克莱国际投资管理公司发行，发行规模达到70亿美元，是美国

量化投资的开端。

第二阶段（1977-1995）

从 1977 年到 1995 年，量化投资在海外经历一个缓慢的发展，这其中受到诸多因素的影响，随着信息技术和计算机技术方面取得巨大进步，量化投资才迎来了其高速发展的时代。

第三阶段（1995-至今）

从 1995 年到现在，量化投资技术逐渐趋于成熟，同时被大家所接受。在全部的投资中，量化投资大约占比 30%，指数类投资全部采用定量技术，主动投资中，约有 20%-30% 采用定量技术。

事实上，互联网的发展，使得新概念在世界范围的传播速度非常快，作为一个概念，量化投资并不算新，国内投资者早有耳闻。但是，真正的量化基金在国内发展还处于初级阶段。

随着我国改革开放 30 年，证券市场成立二十年，雨后春笋般的在最近短短几年里，国内打着“量化投资”标签的基金已有 9 只，加上最近即将发行的长信量化先锋，量化基金数量将达到 10 只。此前 9 只量化基金策略不同，表现亦是参差不齐，量化投资也因而颇受质疑。

因为我国 A 股的种种特殊性，在我国 A 股市场使用量化策略，好比盲人摸象。市场上最早的几只量化基金，业绩一度饱受病垢。人们对量化基金的争议主要集中于两点，一是 A 股市场是否有量化基金生存的土壤，二是基金的量化策略是否可以接受市场长期的检验。

量化投资的未来

相较于海外成熟市场，A 股市场的发展历史较短，投资者队伍参差不齐，投资理念还不够成熟，留给主动投资发掘市场非有效性，产生阿尔法的潜力和空间也更大。投资理念多元化，也创造出多元分散的 alpha 机会。

量化投资的技术和方法在国内几乎没有竞争者。中医治疗中医擅长的疾病、西医治疗西医擅长的疾病；如果把证券市场看作一个病人的话，每个投资者就是医生，定性投资者挖掘定性投资的机会，治疗定性投资的疾病。证券市场上定性投资者太多了，机会太少，竞争太激烈；量化投资者太少了，机会很多，竞争很少。这给量化投资创造了良好的发展机遇——当其他人都摆西瓜摊的时候，我们摆了一个苹果摊。

总的看来，量化投资和定性投资的差别真的有如中医和西医的差别，互有长短、各有千秋。由此可见，随着 2010 年 4 月股指期货的出台，量化投资国内市场发展潜力逐渐显现，已有北京大学汇丰商学院，上海交通大学安泰管理学院投入数百万开设了专业的量化投资金融实验室，并开办了量化投资高级研修班，为国内量化投资的市场发展提供了良好学术和实战环境。

第三节 量化投资的应用与流程

量化投资应用

量化投资一般应用于科学分析、市场监测、交易执行。

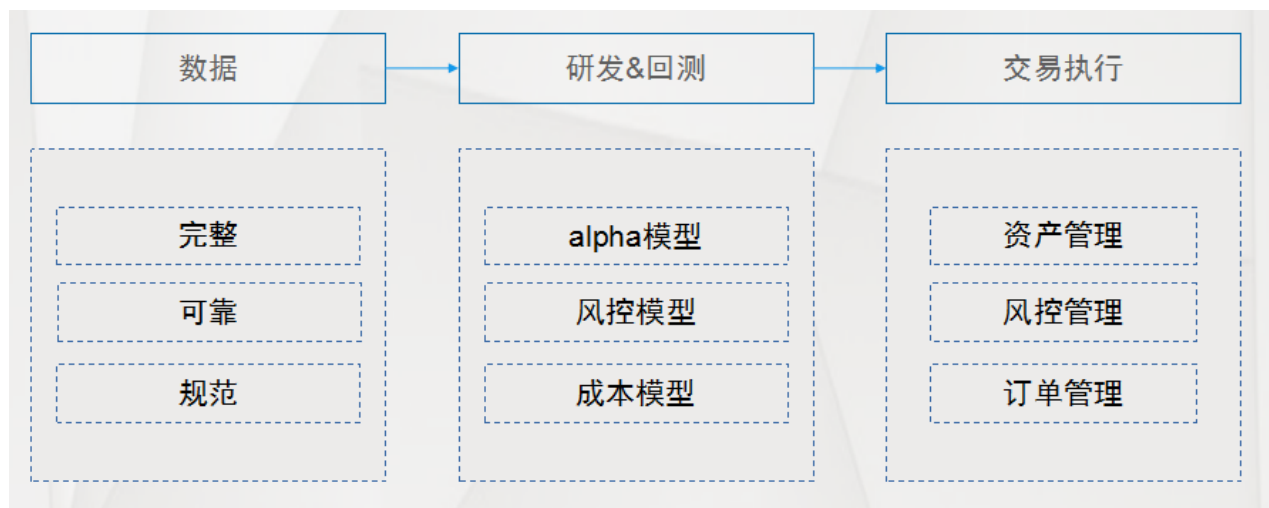
科学分析主要运用计算机技术对历史数据进行处理，最终得出一个科学的结论。比如：投资者如果想要知道每股收益这个财务数据是否可以作为投资参考，其可以通过对历史数据进行分析处理，分别买入较高每股收益的股票并持有一段时间和买入较低每股收益的股票并持有一段时间，如果买入较高的每股收益的股票的获利能力远大于买入较低每股收益的股票的获利能力，那么证明了过去一段时间内，股票的每股收益数据确实会影响股价的涨跌，反之，则不然。

市场监测主要运用计算机程序，实时对整个市场进行监控，包括个股价格波动、市场消息、突发事件等。目前中国股市的上市公司高达 3000 多家，如果人为去监测整个市场的动态，其会消耗大量的人力物力，并且最终效果达不到预期，而量化投资的市场监测功能能较好的解决这个问题，通过计算机程序，实时对所有上市股票进行监测。

交易执行主要运用计算机程序，完成精确的、及时的交易工作。一般而言，多账户多策略的交易执行需要计算机程序实现。人为无法同时操作多个账户，会导致交易不精确，过于迟缓的情况发生。不仅如此，计算机程序还能实现算法交易，能有效降低交易成本。

量化投资流程

一般而言，量化投资的流程为：数据—研发&回测—交易执行。



1.数据：量化投资需要对大量的数据进行分析与研究，这些数据应当是完整、可靠、规范的数据，使得最终的分析 and 研究结论是正确的、可靠的。

2.研发&回测：量化投资在研发策略的过程需要考虑 alpha 模型、风控模型、成本模型。alpha 模型是策略的投资逻辑实现，风控模型作用是控制风险，成本模型能最大化收益。

3.交易执行：交易执行需要考虑资产管理、订单管理、风控管理。资产管理主要有资金

使用、资金分配等。订单管理主要是对订单进行再处理，真实交易中存在未完成的订单。风控管理主要是控制账户的风险以及外部风险：比如网络中断，计算机故障等。

第二章：量化策略入门

第一节：MindGo 量化交易平台

MindGo 量化交易平台是同花顺旗下的人工智能投资平台，拥有高质海量的金融数据，零延迟的回测引擎，最接近真实市场环境的仿真交易平台，干净、完整的 API 文档，同时支持目前广泛使用的脚本语言——Python 语言，致力打造国内一流的专业在线量化交易平台，帮助广大投资者和高校师生实现量化策略，开启 AI 时代，让投资变得更简单！

MindGo 提供以下服务：

1.数据

MindGo 数据基于 2014 年至今完整的 Level-2 数据，包括完整的停牌、复权数据，且会在第二日早晨更新。除此之外，MindGo 还提供上市公司财务数据、场外基金数据、行业指数数据、股指期货数据等等。

2.回测引擎

MindGo 提供了高效快捷的回测环境和简洁的 API 文档，支持对沪深 A 股、ETF 的日级或分钟级回测，回测结果实时显示、快速响应、数据全面，方便用户随时检验和优化策略。

3.模拟交易

MindGo 提供实时的沪深 A 股和 ETF 模拟交易工具，并支持分钟和日级运行，实时呈现策略表现。为量化交易爱好者提供全面、及时、专业、个性的一站式服务。

4.研究平台

MindGo 提供 IPython Notebook 研究平台，初学者可在研究平台上学习 Python 语言，专业研究者可获取数据，进行研究，最终研究结果支持文件导出和策略应用。

5.量化交流社区

MindGo 提供线上交流社区，便于量化爱好者交流量化策略，学习量化知识，一起成长。

第二节：MindGo API 介绍

一、回测引擎介绍

回测环境

MindGo 提供的回测引擎运行在 Python 3.5 之上, 因此策略代码必须兼容 Python3.5, 整个回测环境支持所有的 Python 标准库和部分常用第三方库。

回测过程

1. 您的策略必须在 `initilize` 函数框架下实现:

A. `initilize` 为初始化函数, 用于初始一些全局变量, 在整个回测过程最开始执行一次。

B. `handle_data` 为时间驱动函数, 用于设置买卖条件等, 每个回测时间频率 (每日/分钟/tick) 调用一次。

以下是一个简单的策略, 每日开盘买入 **100** 股贵州茅台, 让我们体验一下整个回测过程!

```
# 初始化账户
def initialize(account):
    # 定义要交易的股票: 贵州茅台
    account.security = '600519.SH'
# 设置买卖条件, 每个交易频率 (日/分钟/tick) 调用一次
def handle_data(account, data):
    # 每天开盘买入 100 股贵州茅台
    order(account.security, 100)
```

2. 完成策略编写后, 选定回测开始日期和结束日期, 选择初始资金、调仓频率 (每日或每分钟) 等参数, 点击 "进行回测", 即开始回测。

2016-10-23	至	2017-09-30	¥ 100000	每日 ▼	进行回测
------------	---	------------	----------	------	------

3. 回测引擎根据您选择的调仓频率调用 `handle_data` 函数, 也就是执行该函数下的代码。回测引擎会实时显示策略当前时间的数据, 如收益、风险指标、持仓等信息。



4.回测引擎会根据您所使用的下单方式进行下单，并根据后续实际成交情况进行订单处理；

日志 错误

```

2017-10-24 17:40:25 - INFO INFO :回测开始运行
2017-10-24 17:40:25 - INFO 回测开始
2016-10-27 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-10-28 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-10-31 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-01 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-02 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-03 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-04 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-07 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-08 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-09 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-10 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-11 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-14 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-15 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-16 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-17 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-18 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);
2016-11-21 09:30:00 - INFO 买入600519.SH没有足够的资金,订单里变动(100->0);

```

5.您可以在任何时候调用 log 函数来打印需要输出的日志;通过 record 函数输出自定义图形。添加 log.info 函数与 record 函数后的代码如下：

```
# 初始化账户
def initialize(account):
    # 定义要交易的股票：贵州茅台
    account.security = '600519.SH'
#设置买卖条件，每个交易频率（日/分钟/tick）调用一次
def handle_data(account,data):
    #每天开盘买入 100 股贵州茅台
    order(account.security,100)
    #获取账户可用资金
    money=account.cash
    #获取账户已用资金
    used_money=account.capital_used*(-1)
    #log.info()函数打印信息
    log.info('已使用资金'+str(used_money))
    #record 函数进行自定义画图，分别为可用资金与已用资金
    record(money=money,used_money=used_money)
```

策略运行结果，回测结果页面上分别显示了可用资金和已用资金曲线：



运行时间

回测引擎通过以下方法对您的策略进行回测

1. 开盘前(9:00)运行：

before_trading_start 函数

2. 盘中运行：

handle_data 函数

> 日回测(9:30:00)运行一次

> 分钟回测(9:30:00-11:30,13:00:00-15:00:00)，每分钟运行一次

3. 盘后(15:30)运行：

after_trading_end 函数

订单处理

对于你的策略在某个单位时间下的单，回测引擎会做如下处理：

1. 按每日回测

A. 交易价格：

> 市价单：开盘价+滑点。

> 限价单：委托价+滑点。

B. 最大成交量：

> 默认为下单个股当日总成交量的 25%。

> 若下单量低于最大成交量，则按下单量成交；若下单量大于最大成交量，则按最大成交量成交。

C. 撮合方式：

> 市价单：开盘下单，一次性撮合，不成交或未成交部分即刻取消委托。若开盘价为涨停价，则买入不成交；若开盘价为跌停价，则卖出不成交。

> 限价单：开盘下单，之后每分钟均按分钟价格撮合一次，未成交部分顺延至下一分钟进行撮合，直到完全成交或者当天收盘为止。

2. 分钟回测

A. 交易价格：

> 市价单：当前分钟起始价+滑点

> 限价单：委托价 + 滑点，如不符合则不成交

B. 最大成交量：

> 默认为下单个股当前分钟总成交量的 50%。

> 若下单量低于最大成交量，则按下单量成交；若下单量大于最大成交量，则按最大成交量成交。

C.撮合方式:

>市价单: 分钟起始点下单, 一次性撮合, 不成交或未成交部分即刻取消委托。若分钟起始价为当日涨停价, 则买入不成交; 若分钟起始价为当日跌停价, 则卖出不成交

>限价单: 分钟起始点下单, 之后每分钟均按分钟价量撮合一次, 未成交部分顺延至下一分钟进行撮合, 直到完全成交或者当天收盘为止。

二、函数 API 介绍

MindGo 量化交易平台有 80 多个函数, 每个函数都有其功能, 本节主要介绍三类函数, 并针对该类函数中的最常用 1 至 2 个函数做简要介绍, 并在下一节中, 能完成第一个量化策略。

第一类量化策略框架函数, 为了驱动策略回测、规范量化策略编写, MindGo 提供以下两个函数:

1.初始化函数: initialize

函数调用方法:

```
def initialize(account):
```

函数功能:

def initialize(account): 是初始化函数, 整个回测只在最开始时执行一次, 用于初始化账户信息、回测参数、全局变量等。

注意事项:

1. 该函数用于初始化账户, 任何一个策略都必须有该函数, 相当于开户。

2. 在该函数下, 你可以设置很多初始条件, 例如: 基准指数, 交易佣金, 滑点, 股票池等等。

3. 该函数不能写成 initialize(account), 而必须写成 python 的函数形式: def

initialize(account): (温馨提示: 不能落下冒号.)

示例:

```
def initialize(account):  
    #初始化策略时设置股票池  
    account.security='000001.SZ'
```

2.定时运行函数:handle_data

函数调用方法:

```
def handle_data(account, data):
```

函数功能

def handle_data(account, data): 函数用来定时执行买卖条件, 每个交易频率 (日/分钟/tick) 自动调用一次。

注意事项:

1. 该函数在回测中的非交易日是不会触发的。(例如 1 月 1 日至 3 日是非交易日, 则 handle_data 在 1 日至 3 日不触发, 直到下一个交易日 4 号触发)

2. 该函数的交易频率根据策略的交易频率, 在盘中运行:

i. 日回测(9:30:00), 每日运行一次。

ii. 分钟回测(9:30:00-11:30, 13:00:00-15:00:00), 每分钟运行一次。

3.在该函数中，可以传入其他函数的运行结果，用来判断买卖条件。
4.该函数不能写成 `handle_data(account, data)`，而必须写成python语言中规范的函数形式：
`def handle_data(account, data):`(温馨提示：不能落下冒号)
示例：

```
# 每个交易频率下单买 100 股的平安银行股票.  
def handle_data(account, data):  
    order('000001.SZ', 100)
```

第二类获取数据函数，以函数形式，通过参数设置以满足您数据获取：

1.行情数据获取函数：history

调用方法：

`history(symbol_list, fields, bar_count, fre_step, skip_paused = False, fq = 'pre', is_panel=0)`

函数功能：获取多只股票多属性的历史行情数据。

函数参数：

参数	含义	参数举例
symbol_list	股票、指数或基金代码列表	填写需要获取数据的股票代码，可多个股票同时获取，例如：['600519.SH']
fields	数据字段	填写需要获取的数据字段，例如：收盘价为close,最高价为high，['close','high']
bar_count	历史长度	填写需要获取的数据历史长度，例如：20
fre_step	时间步长	填写获取数据的时间步长，例如：按天则为'1d'，按x分钟则为'xm'。
skip_paused	是否跳过停牌数据	填写获取数据是否跳过停牌，例如：不跳过为skip_paused=False，跳过为skip_paused=True。
fq	复权选项	填写获取数据的复权选项，fq=None为不复权，fq='post'为后复权，fq='pre'为前复权。
is_panel	返回数据格式	选择返回的数据格式，is_panel=1为pandas.panel格式，is_panel=0为dict格式。

注意事项：

1.该函数没有起始日期，结束日期默认为上一个交易日或上一分钟。

您只需要在策略代码中调用该函数，并确定函数内的参数，即可获取到相应数据，详细参考以下两个示例。

【示例 1】

```
def initialize(account):  
    pass  
def handle_data(account,data):  
    #获取万科 A 与平安银行过去 10 日的收盘价与最高价，并且输出数据  
    price=history(['000001.SZ','000002.SZ'], ['close','high'], 10, '1d', False, 'pre', is_panel=1)  
    log.info('收盘价：'+str(price['close']))  
    log.info('最高价：'+str(price['high']))
```

【示例 2】

```
def initialize(account):  
    pass  
def handle_data(account,data):  
    #获取万科 A 与平安银行过去 10 个 60 分钟级的开盘价，并且输出数据  
    price=history(['600519.SH','300033.SZ'], ['open'], 10, '60m', False, 'pre', is_panel=0)  
    log.info('300033.SZ 开盘价: '+str(price['300033.SZ']))  
    log.info('600519.SH 开盘价: '+str(price['600519.SH']))
```

示例返回结果:

【示例 1】返回结果:

2017-01-03 09:30:00 - INFO

收盘价: 000001.SZ 000002.SZ

2016-12-19	9.20	21.10
2016-12-20	9.11	20.33
2016-12-21	9.16	20.48
2016-12-22	9.14	20.61
2016-12-23	9.08	20.30
2016-12-26	9.12	20.65
2016-12-27	9.08	21.42
2016-12-28	9.06	21.20
2016-12-29	9.08	20.84
2016-12-30	9.10	20.55

2017-01-03 09:30:00 - INFO

最高价: 000001.SZ 000002.SZ

2016-12-19	9.23	22.00
2016-12-20	9.20	21.00
2016-12-21	9.16	20.70
2016-12-22	9.16	20.77
2016-12-23	9.14	20.67
2016-12-26	9.13	20.69
2016-12-27	9.13	21.98
2016-12-28	9.11	21.48
2016-12-29	9.09	21.32
2016-12-30	9.10	20.96

【示例 2】返回结果:

2017-01-03 09:30:00 - INFO

300033.SZ 开盘价: open

2016-12-27 13:59:00	69.00
2016-12-27 14:59:00	68.50
2016-12-28 10:29:00	68.12
2016-12-28 11:29:00	68.69
2016-12-28 13:59:00	68.49
2016-12-28 14:59:00	68.55
2016-12-29 10:29:00	68.90
2016-12-29 11:29:00	68.81
2016-12-29 13:59:00	70.20
2016-12-29 14:59:00	70.11

2017-01-03 09:30:00 - INFO

600519.SH 开盘价: open

2016-12-27 13:59:00 328.30

2016-12-27 14:59:00 326.55
2016-12-28 10:29:00 326.99
2016-12-28 11:29:00 326.00
2016-12-28 13:59:00 325.19
2016-12-28 14:59:00 325.40
2016-12-29 10:29:00 324.01
2016-12-29 11:29:00 323.70
2016-12-29 13:59:00 325.80
2016-12-29 14:59:00 325.80

第三类下单函数，以函数形式，通过参数设置以满足您策略下单交易：

1.按金额下单函数：order_value

函数调用方法：

```
order_value(symbol, amount)
```

函数功能

order_value 是下单函数,根据股数下单，交易股票.

函数参数：

参数	含义	参数举例
symbol	股票、指数或基金代码列表	填写需要下单交易的股票代码，不可多个股票同时获取，例如：'600519.SH'
value	下单金额	填写需要下单的交易的金额，正数表示买入，负数表示卖出

注意事项：

1.由于股票单次买入必须以 1 手（100 股）为单位，实际成交金额小于等于下单金额。

示例：

```
def initialize(account):  
    pass  
def handle_data(account,data):  
    #买入 10000 元平安银行  
    order_value(symbol='000001.SZ',value=10000)
```

2.按股数下单函数：order

函数调用方法：

```
order(symbol, amount)
```

函数功能

order 是下单函数,根据股数下单，交易股票.

函数参数：

参数	含义	参数举例
symbol	股票、指数或基金代码列表	填写需要下单交易的股票代码，不可多个股票同时获取，例如：'600519.SH'
amount	下单股数	填写需要下单交易的股票数量，正数表示买入，负数表示卖出

示例：

```
def initialize(account):
```

```
    pass
```

```
def handle_data(account,data):
```

```
    #买入 100 股平安银行
```

```
    order(symbol='000001.SZ',amount=100)
```

第三节：我的第一个量化策略

对于大部分人来说，量化交易是非常陌生与神秘的。本节内容将带你开启第一个量化策略！

本节内容摘要：1.理解量化策略的基本框架。2.学会编写一个简单的量化交易策略。3.学会将量化交易策略绑定实盘模拟交易，并实时收到交易策略的买卖信号。

1.理解量化策略的基本框架

通常情况下，完整的量化交易策略至少需要确定两件事：

A.交易标的，即**买什么**；

B.确定交易时机，即**怎么买卖**。

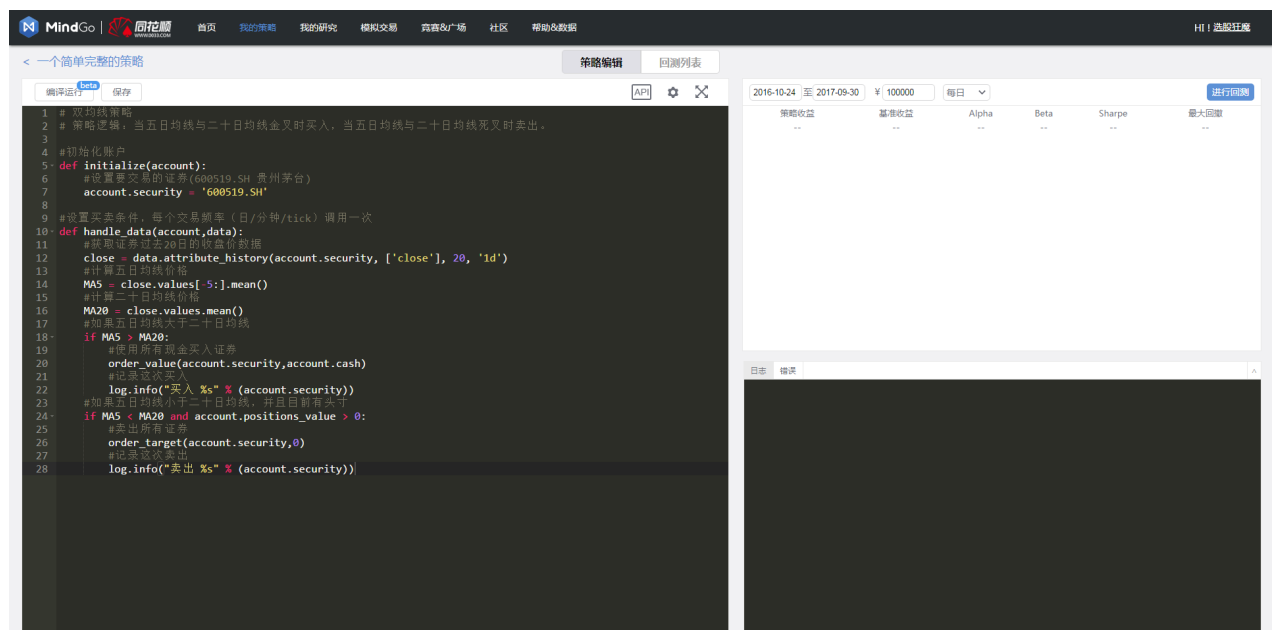
让我们来设计一个简单完整的量化交易策略：

策略交易标的：贵州茅台；

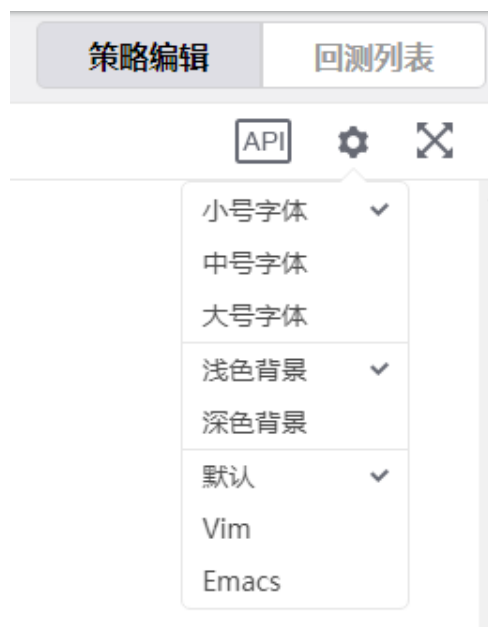
策略交易时机：5日均线与20日均线金叉时，买入；5日均线与20日均线死叉时，卖出。

2.学会编写一个简单的量化交易策略

第一步：打开 MindGo 量化交易平台，先在上方导航栏点击“我的策略”——“策略编译”，再点击蓝色按钮“+新建策略”，接着点击已创建的策略进入策略编译器页面，如下：



温馨提示：“回测列表”下方三个按钮，可以设置编译器字体大小，背景颜色，开启全屏编译，查看 API 文档，如下：



第二步：理解量化交易策略框架对应的代码框架。

```
def initialize(account):
    #初始化函数：确定交易标的
def handle_data(account,data):
    #定时运行函数：确定交易时机
```

框架理解：

1. `def initialize(account)`与 `def handle_data(account,data)`是两个函数，函数格式固定为：`def` 函数名（参数），其中 `def` 后面带空格键，函数末尾必须带冒号。

2. `def initialize(account)`函数是初始化函数，只运行一次，确定初始化条件；`def handle_data(account,data)`函数是定时运行函数，平台默认该函数定时运行。日级策略，每日 9:30；分钟级策略，交易期间内的每分钟。

3. “#”后面为注释内容，用于注释代码，便于编写和阅读。

第三步：确定交易标的：`account.security = '600519.SH'`。

温馨提示：

1. `account` 是账户对象，该对象存放所有账户相关信息，持仓、可用现金、资产盈亏。

2. `account.security` 是在账户对象下，设置 `security` 变量，存放在账户内，这里我们需要确定交易标的，即：`account.security = '600519.SH'`。

`def initialize(account):`

`account.security = '600519.SH' # 已确定交易标的`

`def handle_data(account,data):`

`# 定时运行函数：确定交易时机`

第四步：确定交易时机，即为：5日均线与20日均线金叉时，买入；5日均线与20日均线死叉时，卖出。

从交易时机出发，我们需要计算交易标的5日和20日均线，那么5、20日均线需要用历史行情数据的收盘价来计算。

整个流程即：获取历史行情20日的收盘价数据——计算5、20日均线——判断5、20日均线，进行买卖交易。

A. 获取历史行情20日的收盘价数据：

1. 找到函数历史数据函数：`history`

2. 填写函数参数，获取到数据：

i. 交易标的，即：获取那个股票的数据。

ii. 数据字段：`['close']`收盘价，即：获取哪个数据。

iii. 输入历史长度，即：获取多长时间的数据。

iv. 获取数据的时间步长，即：获取日线级步长数据。

v. 填写是否跳过停牌数据，复权选项，返回数据格式。

最终结果即为：`history(account.security, ['close'], 20, '1d', False, 'pre', is_panel=1)`

3. 将获取到的数据储存，便于计算，即：`close = history(account.security, ['close'], 20, '1d', False, 'pre', is_panel=1)`

`# 获取证券过去20日的收盘价数据`

`close = history(account.security, ['close'], 20, '1d', False, 'pre', is_panel=1)`

B. 计算5、20日均线：

1. 获取数据值，即：`close.values`，`values`可以直接获取储存中的数据值，格式为`close.values`。温馨提示：`close`是我们刚才获取的数据，但是数据有股票、时间、数值，我们直接用`values`获取值用于计算即可。

2.选取数据长度，即：close.values[-5:]。[]用于取值，我们之前获取20个数据，但5日均线只需要过去5日的收盘价，因此[-5:]即为获取倒数第五个到最后一个数据。温馨提示：

- i[:]是获取所有数据。
- ii.[x:]是从第一个获取到第x个，不包括第x个。
- iii.[x:y]是从第x个到第y个，包括x，但不包括y。
- iv.[-x:]获取倒数第x个到最后一个数据。

3.计算均值，即 close.values[-5:].mean()，赋值给 MA5。同理 MA20=close.values.mean()，即对所有值取平均，相当于 MA20=close.values[:].mean()。

```
#计算五日均线价格
MA5 = close.values[-5:].mean()
#计算二十日均线价格
MA20 = close.values.mean()
```

C.判断5、20日均线，进行买卖交易：

1.if 判断条件，即为 if MA5 > MA20:。温馨提示 if 判断函数的格式为 if + 添加判断+;，其中 if 后面必须带一个空格键，其次末尾必须带冒号。

2.当 MA5 小于 MA20 时，我们再对持仓市值判断，如果有持仓，那么持仓市值必然大于 0，需要进行卖出交易，我们直接通过 account 账户对象来获取持仓市值，即为:account.positions_value。

3.下单买入交易：

i.当触发 MA5 大于 MA20 时，我们需要买入股票，这时候我们可以使用 **order_value** 下单函数，该函数以金额下单。

ii.输入下单函数的参数，order_value 函数需要输入两个参数：

1.下单的股票，即为 account.security，我们之前将交易标的传入进去，可以直接用。

2.下单的金额，即 account.cash，这里的 account 是账户对象，我们从账户中获取可用资金，即为 account.cash。

iii.触发条件后程序提醒，当代码执行完下单函数后，我们用 log.info()来打印日志，这样我们也可以看到程序下单了。

温馨提示：log.info()内你可以直接输入中文，例如:log.info('条件满足！买入贵州茅台啦！')

```
if MA5 > MA20:
    #使用所有现金买入证券
    order_value(account.security,account.cash)
    #记录这次买入
    log.info("买入 %s" % (account.security))
```

4.下单卖出交易：

i.当触发 MA5 小于 MA20 时，我们需要卖出股票，这时候我们可以使用 **order_target** 下单函数，该函数以目标股数下单。

ii.输入下单函数的参数，order_target 函数需要输入两个参数：

1.下单的股票，即为 account.security，我们之前将交易标的传入进去，可以直接用。

2.下单的目标股数，即 0，因为我们需要将持仓股票卖出，卖到 0 股为止。

iii.触发条件后程序提醒，当代码执行完下单函数后，我们同 log.info()来打印日志，这样我们也可以看到程序下单了。

```
#如果五日均线小于二十日均线，并且目前有头寸
if MA5 < MA20 and account.positions_value > 0:
    #卖出所有证券
    order_target(account.security,0)
    #记录这次卖出
    log.info("卖出 %s" % (account.security))
```

最终结果：

```
# 双均线策略
# 策略逻辑：当五日均线与二十日均线金叉时买入，当五日均线与二十日均线死叉时卖出。
#初始化账户
def initialize(account):
    #设置要交易的证券(600519.SH 贵州茅台)
    account.security = '600519.SH'
#设置买卖条件，每个交易频率（日/分钟/tick）调用一次
def handle_data(account,data):
    #获取证券过去 20 日的收盘价数据
    close = history(account.security, ['close'], 20, '1d', False, 'pre', is_panel=1)
    #计算五日均线价格
    MA5 = close.values[-5:].mean()
    #计算二十日均线价格
    MA20 = close.values.mean()
    #如果五日均线大于二十日均线
    if MA5 > MA20:
        #使用所有现金买入证券
        order_value(account.security,account.cash)
        #记录这次买入
        log.info("买入 %s" % (account.security))
    #如果五日均线小于二十日均线，并且目前有头寸
    if MA5 < MA20 and account.positions_value > 0:
        #卖出所有证券
        order_target(account.security,0)
        #记录这次卖出
        log.info("卖出 %s" % (account.security))
```

第五步 回测量化交易策略

通过以上4步，我们已经完成了量化交易策略编写，那么接下来我们需要进行量化交易策略回测。

A.首先，我们尝试去跑通整个历史行情，排查代码错误。

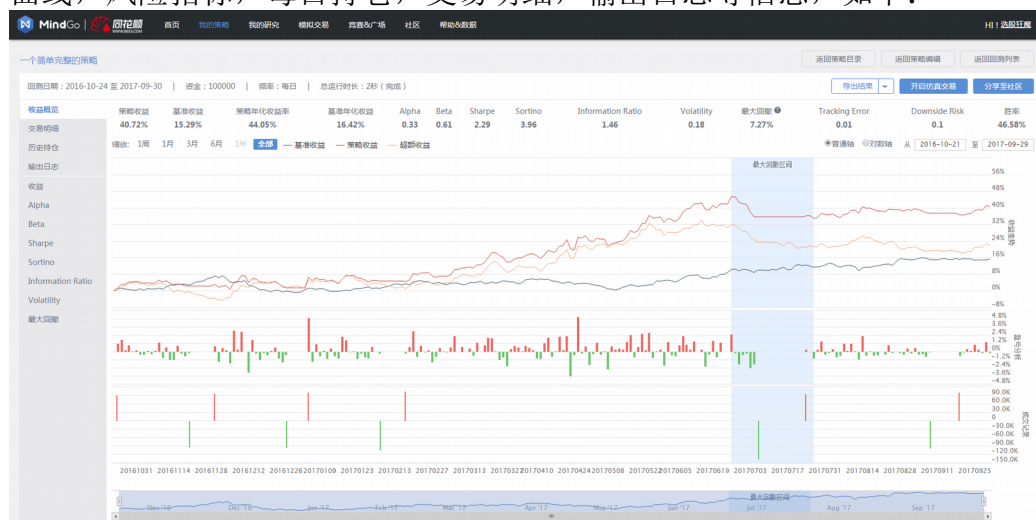
i.右上角设置回测历史长度，设置资金，设置交易频率。



ii.点击左上角“编译运行”按钮，右边出现量化交易策略在历史行情中的表现情况



B.当量化交易策略能跑通整个历史行情后，我们可以确定该代码正确无误，随后点击右上角蓝色按钮“进行回测”。页面跳转至回测页面，在回测详情界面，您可以查看策略收益曲线，风险指标，每日持仓，交易明细，输出日志等信息，如下：



3.学会将量化交易策略绑定实盘模拟交易，并实时收到交易策略的买卖信号

1.在回测显示结果页面，右上角点击蓝色按钮**开启仿真交易**.如下图:



开启仿真交易

选择账户： 新建账户

账户名称： 默认账户

账户类型： 请选择账户类型

取消 确定

2.在账户类型一栏，有股票型、期货型、混合型三种交易账户，选择股票型账户如下图:



开启仿真交易

选择账户： 新建账户

账户名称： 默认账户

账户类型： 请选择账户类型

- 股票型
- 期货型
- 混合型

3.为账户选择相应的资金配置，一旦确定就无法更改，如下图:



开启仿真交易

选择账户： 新建账户

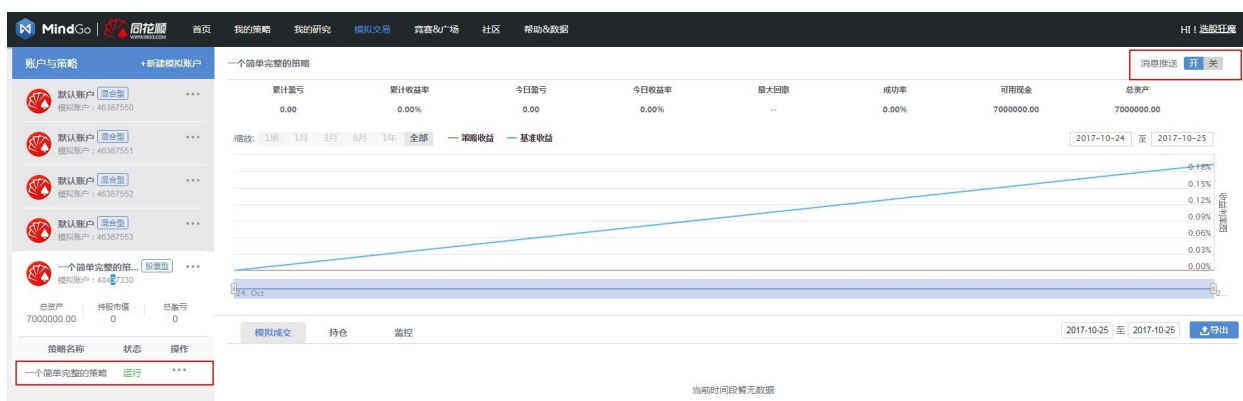
账户名称： 默认账户

账户类型： 股票型

股票资金： 700 万元

取消 确定

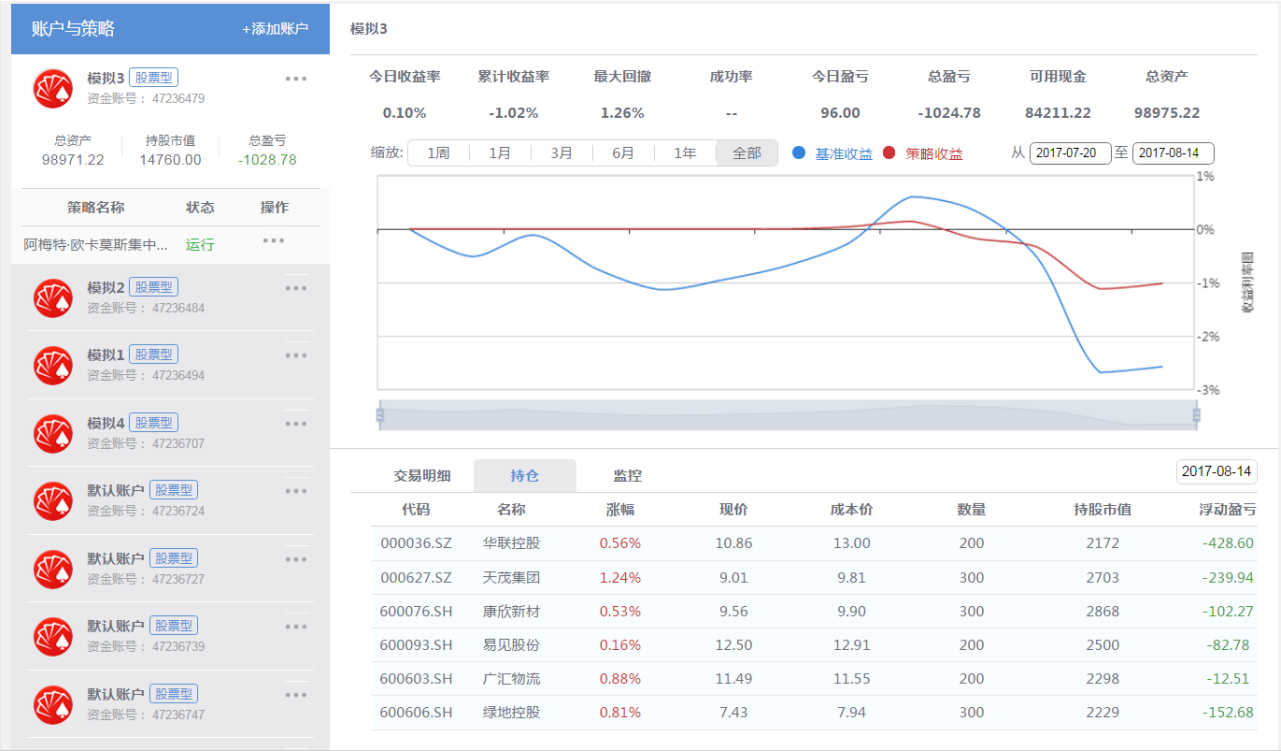
4.至此，我们成功开启了模拟交易。点击模拟交易账户下绑定的策略，**开启交易信号实时推送**，即可将策略的买卖信号发送至同花顺手机 APP 上。如下图:



5.您可以为您模拟交易账户重命名、删除账户、暂停策略运行、发布策略至社区、重启策略、查看策略运行日志、查看策略代码。**注意：重启按钮只会在策略运行错误后显示，如果策略运行正常，显示暂停按钮。**如下图：



6.您可以查看您的模拟交易账户详细情况:交易明细、持仓、盈亏情况、账户风险指标等等。



8.策略运行出错还可能跟系统有关，如果你没发现代码层面的错误，直接点击**重启**按钮，策略就会继续运行啦。



默认账户

混合型

资金账号：45053400

总资产

3091301.52

持股市值

79621.00

总盈亏

-8698.48

策略名称	状态	操作
MACD智能优化策略	错误	...
 默认账户 股票型 资金账号：4505346		...
 默认账户 股票型 资金账号：4684277		...
 默认账户 股票型 资金账号：47493128		...
 测试-双均线账户 股票型 资金账号：47493449		...

重启

发布

删除

查看日志

查看代码

第三章：Python 编程

第一节：Python 介绍

一、Python 简介

Python 是一种计算机程序设计语言。你可能已经听说过很多种流行的编程语言，比如非常难学的 C 语言，非常流行的 Java 语言，适合初学者的 Basic 语言，适合网页编程的 JavaScript 语言等等。

那 Python 是一种什么语言？

Python 的创史人 Guido van Rossum，1989 年在荷兰创造了 python，网络流传是因为他喜欢英国肥皂剧《Monty Python 飞行马戏团》，所以用 python 来命名。

语言分为编译型语言和解释型语言，python 是一门解释型语言，何为解释型语言；

编译型语言：c、c++等

解释型语言有：Python、Php 等

以下表格是两类语言的优缺点介绍

语言	优点	缺点
解释型语言	可跨平台、开发效率高	运行效率低
编译型语言	依赖编译平台，不能跨平台，开发效率低	运行效率高

比如，完成同一个任务，C 语言要写 1000 行代码，Java 只需要写 100 行，而 Python 可能只要 20 行。

所以 Python 是一种相当高级的语言。

你也许会问，代码少还不好？代码少的代价是运行速度慢，C 程序运行 1 秒钟，Java 程序可能需要 2 秒，而 Python 程序可能就需要 10 秒。

那是不是越低级的程序越难学，越高级的程序越简单？表面上来说，是的，但是，在非常高的抽象计算中，高级的 Python 程序设计也是非常难学的，所以，高级程序语言不等于简单。

但是，对于初学者和完成普通任务，Python 语言是非常简单易用的。连 Google 都在大规模使用 Python，你就不用担心学了会没用。

二、第一个 Python 程序

作为初学者，学习一门新语言，尝试写一个程序，可以快速进入状态，第一个程序叫'Hello MindGo'。在 Python 中，它是这个样子：

```
print('Hello MindGo')  
Hello MindGo
```

这是 `print()` 语句的一个示例。`print` 并不会真的往纸上打印文字，而是在屏幕上输出值。程序中引号内的内容表示需要输出的内容，而引号在输出结果中并不显示。

`print()` 语句也可以跟上多个字符串，用逗号 “,” 隔开，就可以连成一串输出：

```
print('This', 'is', 'an', 'apple')  
This is an apple
```

`print()` 也可以打印整数，或者计算结果：

```
print(300)  
300  
print(100 + 200)  
300
```

让我们可以把计算 $100 + 200$ 的过程和结果都打印出来试试：

```
print('100 + 200 =', 100 + 200)  
100 + 200 = 300
```

三、变量

编程语言最强大的功能之一是操作变量的能力。变量是指向一个值的名称。

赋值语句可以建立新的变量，并给他们赋值：

```
quant = 'Hello MindGo'  
x = 500
```

这个例子有两个赋值。第一个将字符串'Hello MindGo'赋值给一个叫做 `quant` 的变量；第二个将 `500` 赋值给 `x`。

他的状态图如下所示：

```
quant  —————  'Hello MindGo'  
  
x      —————  500
```

四、两项基本操作

了解下面两个基本操作对后面的学习是有好处的：

1. 基本的输入输出

可以在 Python 中使用 +、-、*、/ 直接进行四则运算。

```
1+3*3
```

```
10
```

2. 导入模块

使用 `import` 可以导入模块，导入之后，就可以使用这个模块下面的函数了。

比如导入 `math` 模块，然后使用 `math` 模块下面的 `sqrt` 函数：

```
import math  
math.sqrt(9)
```

```
3.0
```

这时你可能会有疑问：

“每次引用函数的时候，`math` 这个模块前缀都要带吗？可不可以不带？”

直接输入 `sqrt(9)` 是会报错的，好烦人，那么有什么办法可以不用每次都带前缀？

办法是有的，用 “`from 模块 import 函数`” 的格式先把函数给 “拿” 出来。

```
from math import sqrt  
sqrt(9)
```

```
3.0
```

这样每次使用 `sqrt` 函数的时候就不用再加 `math` 前缀了。又有了一个问题？

“`math` 模块下面有那么多函数，可不可以写一个语句，然后 `math` 下面所有函数都可以直接使用？”

调用了 `math` 下面的 `sqrt` 函数，写一个 `from...import...`，再调用下面的 `floor`，还要写一个，如此也挺麻烦的，有个办法可以一下把所有函数都给 “拿” 出来：

```
from math import *  
print(sqrt(9))  
print(floor(32.9))
```

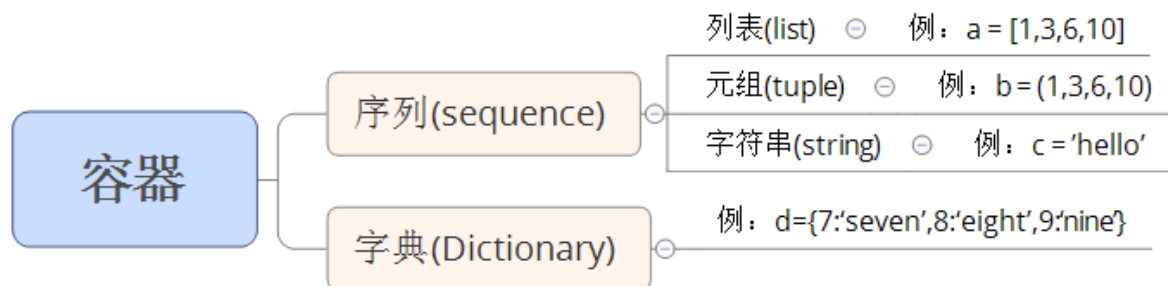
```
3.0
```

```
32.0
```

第二节：数据类型

一、什么是数据结构

开始学 Python 时，经常会被它的数据结构，什么字典、序列、元组等等搞的很混乱，因此有必要梳理清楚数据结构的概念。



首先要从容器说起，Python 中有一种名为容器的数据结构，顾名思义，容器，就是装数据的器具，它主要包括序列和词典，其中序列又主要包括列表、元组、字符串等，如下图：

列表的基本形式比如：[1,3,6,10]或者['yes','no','OK']

元组的基本形式比如：(1,3,6,10)或者('yes','no','OK')

字符串的基本形式比如：'Hello '

以上几种属于序列，序列中的每一个元素都被分配一个序号——即元素的位置，也称为“索引”，第一个索引，即第一个元素的位置是 0，第二个是 1，依次类推。列表和元组的区别主要在于，列表可以修改，而元组不能（注意列表用中括号而元组用括号）。序列的这个特点，使得我们可以利用索引来访问序列中的某个或某几个元素，比如：

```
a=[1,3,6,10]
a[2]
```

```
6
```

```
b=(1,3,6,10)
b[2]
```

```
6
```

```
c='hello'
c[0:3]
```

```
'hel'
```

而与序列对应的“字典”则不一样，它是一个无序的容器：

它的基本形式比如：`d={7:'seven',8:'eight',9:'nine'}`

这是一个“键—值”映射的结构，因此字典不能通过索引来访问其中的元素，而要根据键来访问其中的元素：

```
d={7:'seven',8:'eight',9:'nine'}
d[8]

'eight'
```

二、序列的一些通用操作

除了上面说到的索引，列表、元组、字符串等这些序列还有一些共同的操作。

1.索引

序列的最后一个元素的索引，也可以是-1，倒数第二个也可以用-2，依次类推：

```
a=[1,3,6,10]
print(a[3])
print(a[-1])

10
10
```

2.分片

使用分片操作来访问一定范围内的元素，它的格式为：

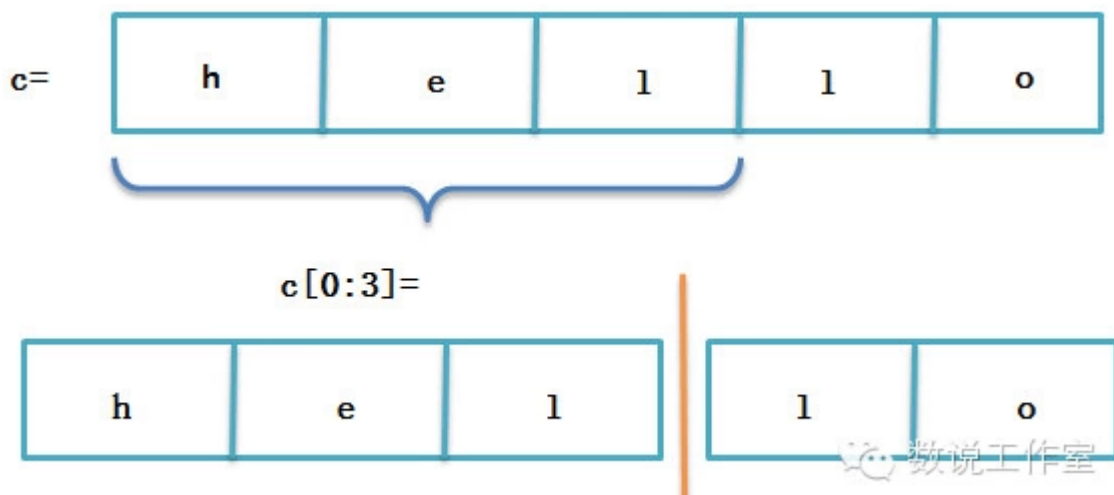
`a[开始索引:结束索引:步长]`

那么访问的是，从开始索引号的那个元素，到结束索引号-1 的那个元素，每间隔步长个元素访问一次，步长可以忽略，默认步长为1。

```
c='hello'
c[0:3]

'hel'
```

这个就好像把一个序列给分成几片几片的，所以叫做“分片”



3.序列相加

即两种序列合并在一起，两种相同类型的序列才能相加。

```
[1,2,3]+[4,5,6]
```

```
[1, 2, 3, 4, 5, 6]
```

```
'hello','+MindGo'
```

```
'hello,MindGo'
```

4.成员资格

为了检查一个值是否在序列中，可以用 in 运算符

```
a='hello'  
print('o' in a)  
print('t' in a)
```

```
True
```

```
False
```

三、列表操作

以上是序列共有的一些操作，列表也有一些自己独有的操作，这是其他序列所没有的

1.List 函数

可以通过 list(序列)函数把一个序列转换成一个列表：

```
list('hello')
```

```
['h', 'e', 'l', 'l', 'o']
```

2.元素赋值、删除

元素删除——del a[索引号]

元素赋值——a[索引号]=值

```
a='hello'  
b=list(a)  
print(b)
```

```
['h', 'e', 'l', 'l', 'o']
```

```
del b[2]  
print(b)
```

```
['h', 'e', 'l', 'o']
```

```
b[2]='t'
print(b)

['h', 'e', 't', 'o']
```

分片赋值——a[开始索引号:结束索引号]=list(值)

为列表的某一范围内的元素赋值，即在开始索引号到结束索引号-1 的区间几个元素赋值，比如，利用上面语句，如何把 hello 变成 heyyo？

```
b=list('hello')
print(b)
['h', 'e', 'l', 'l', 'o']
```

```
b[2:4]=list('yy')
print(b)

['h', 'e', 'y', 'y', 'o']
```

注意虽然 “ll”处于 “hello”这个单词的第 2、3 号索引的位置，但赋值时是用 b[2:4]而不是 b[2:3]，另外注意 list()用小括号。

3.列表方法

上面说过 list 函数，函数这个东西在很多语言中都有，比如 excel 里面的 if 函数、vlookup 函数，SQL 里面的 count 函数，以及各种语言中都有的 sqrt 函数等等，python 中也有很多函数。

Python 中的方法，是一个“与某些对象有紧密联系的”函数，所以列表方法，就是属于列表的函数，它可以对列表实现一些比较深入的操作，方法这样调用：对象.方法(参数)

那么列表方法的调用就理所当然是：列表.方法(参数)
常用的列表方法这么几个，以 a=['h','e','l','l','o']为例：

```
a=['h','e','l','l','o']
print(a)

['h', 'e', 'l', 'l', 'o']
```

给列表 a 的 n 索引位置插入一个元素 m: a.insert(n,m)

```
a.insert(2,'t')
print(a)

['h', 'e', 't', 'l', 'l', 'o']
```

给列表的最后添加元素 m: a.append(m)

```
a.append('q')
print(a)

['h', 'e', 't', 'l', 'l', 'o', 'q']
```

返回 a 列表中，元素 m 第一次出现的索引位置: a.index(m)

```
a.index('e')
```

```
1
```

删除 a 中的 m 元素: a.remove(m)

```
a.remove('e')
```

```
print(a)
```

```
['h', 't', 'l', 'l', 'o', 'q']
```

将列表 a 从大到小排列: a.sort()

```
a.sort()
```

```
print(a)
```

```
['h', 'l', 'l', 'o', 'q', 't']
```

四、字典操作

1.dict 函数

dict 函数可以通过关键字参数来创建字典，格式为：

dict(参数 1=值 1,参数 2=值 2, ...)= {参数 1:值 1, 参数 2=值 2, ...}

比如，如何创建一个名字 name 为 quanter，年龄 age 为 22 的字典？

```
dict(name='quanter',age=22)
```

```
{'age': 22, 'name': 'quanter'}
```

2.基本操作

字典的基本行为与列表在很多地方都相似，下面的例子以序列 a=[1,3,6,10]，字典 f={'age': 27, 'name': 'shushuo'} 为例

功能	列表操作		字典操作	
	格式	例	格式	例
求长度	len(列表)	>>>len(a) 4	len(字典)	>>>len(f) 2
找到某位置上的值	列表[索引号]	>>>a[1] 3	字典[键]	>>>f['age'] 27
元素赋值	列表[索引]=值	>>>a[2]=1 >>>a [1,3,1,10]	字典[键]=值	>>>f['age']=28 >>>f {'age': 28, 'name': 'shushuo'}
元素删除	del 列表[索引]	>>>del a[1] >>>a [1,6,10]	del 字典[键]	>>> del f['name'] >>> f {'age': 28}
成员资格	元素 in 列表	>>> 1 in a True	键 in 字典	>>> 'age' in f True

第三节：条件与循环

一、if 语句

注意两点，一是缩进，Python 是用缩进来标识出哪一段属于本循环。，二是条件后面有冒号：

```
j=2.67
if j<3:
    print('j<3')
j<3
```

对于多条件，要写成 elif，标准格式为：

```
if 条件 1:
    执行语句 1
elif 条件 2:
    执行语句 2
else:
    执行语句 3
```

注意 if...elif...else 三个是并列的，不能有缩进：

```
t=3
if t<3:
    print('t<3')
elif t==3:
    print('t=3')
else:
    print('t>3')
t=3
```

二、while true/break 语句

该语句的格式为

```
while true 即条件为真:
    执行语句
    if 中断语句条件 : break
```

看个例子：

```
a=3
while a<10:
    a=a+1
    print(a)
    if a==8: break
```

```
5
6
7
8
```

虽然 while 后面的条件是 $a < 10$ ，即 a 小于 10 的时候一直执行，但是 if 条件中规定了 a 为 8 时就 break 掉，因此，输出只能输到 8。

三、for 语句

可以遍历一个序列/字典等。

```
a=[1,2,3,4,5]
for i in a:
    print(i)
1
2
3
4
5
```

第四节：函数

函数是组织好的，可重复使用的，用来实现单一，或相关联功能的代码段。函数能提高应用的模块性，和代码的重复利用率。

一、调用函数

Python 内置了很多有用的函数，我们可以直接调用。

要调用一个函数，需要知道函数的名称和参数，比如求绝对值的函数 `abs`，只有一个参数。

调用 `abs` 函数：

```
abs(-100)
```

```
100
```

调用函数的时候，如果传入的参数数量不对，会报 `TypeError` 的错误，并且 Python 会明确地告诉你：`abs()` 有且仅有 1 个参数，但给出了两个：

```
abs(1, 2)
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-22-05b55862d84c> in <module>()
----> 1 abs(1, 2)
```

```
TypeError: abs() takes exactly one argument (2 given)
```

如果传入的参数数量是对的，但参数类型不能被函数所接受，也会报 `TypeError` 的错误，并且给出错误信息：`str` 是错误的参数类型：

```
abs('a')
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-23-3b9a69fe3abb> in <module>()
----> 1 abs('a')
```

```
TypeError: bad operand type for abs(): 'str'
```

二、定义函数

在 Python 中，定义一个函数要使用 `def` 语句，依次写出函数名、括号、括号中的参数和冒号`:`，然后，在缩进块中编写函数体，函数的返回值用 `return` 语句返回。

我们以自定义一个求绝对值的 `abs_my` 函数为例：

```
def abs_my(x):
    if x >= 0:
        return x
    else:
        return -x
```

测试并调用 `abs_my` 看看返回结果是否正确：

```
abs_my(-100)
```

```
100
```

请注意，函数体内部的语句在执行时，一旦执行到 `return` 时，函数就执行完毕，并将结果返回。因此，函数内部通过条件判断和循环可以实现非常复杂的逻辑。

如果没有 `return` 语句，函数执行完毕后也会返回结果，只是结果为 `None`。

`return None` 可以简写为 `return`。

三、函数的参数

定义函数的时候，我们把参数的名字和位置确定下来，函数的接口定义就完成了。对于函数的调用者来说，只需要知道如何传递正确的参数，以及函数将返回什么样的值就够了，函数内部的复杂逻辑被封装起来，调用者无需了解。

Python 的函数定义非常简单，但灵活度却非常大。除了正常定义的必选参数外，还可以使用默认参数、可变参数和关键字参数，使得函数定义出来的接口，不但能处理复杂的参数，还可以简化调用者的代码。

3.1 默认参数

我们仍以具体的例子来说明如何定义函数的默认参数。先写一个计算 y 的平方的函数：

```
def power(y):
```

```
    return y * y
```

当我们调用 `power` 函数时，必须传入有且仅有的一个参数 y ：

```
power(5)
```

```
25
```

现在，如果我们要计算 y^3 怎么办？可以再定义一个 `power3` 函数，但是如果我们要计算 y^4 、 y^5 ……怎么办？我们不可能定义无限多个函数。

函数中再添加一个参数即可帮助我们搞定这个问题！

```
def power(y, t):
```

```
    d = 1
```

```
    while t > 0:
```

```
        t = t - 1
```

```
        d = d * y
```

```
    return d
```

对于这个修改后的 `power` 函数，可以计算任意 n 次方：

```
power(5, 3)
```

```
125
```

但是，旧的调用代码失败了，原因是我们增加了一个参数，导致旧的代码无法正常调用：

```
power(5)
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-28-cacc59fd9139> in <module>()
      5     d = d * y
      6     return d
----> 7 power(5)
```

TypeError: power() missing 1 required positional argument: 't'

这个时候，默认参数就排上用场了。由于我们经常计算 y 的平方，所以，完全可以把第二个参数 n 的默认值设定为 2：

```
def power(y, t=2):
```

```
    d = 1
    while t > 0:
        t = t - 1
        d = d * y
    return d
```

这样，当我们调用 `power(5)` 时，相当于调用 `power(5, 2)`：

```
power(5)
25
```

```
power(5, 2)
25
```

而对于 $n > 2$ 的其他情况，就必须明确地传入 n ，比如 `power(5, 3)`。

从上面的例子可以看出，默认参数可以简化函数的调用。设置默认参数时，有几点要注意：

一、必选参数在前，默认参数在后，否则 Python 的解释器会报错。

二、如何设置默认参数。当函数有多个参数时，把经常改动的参数放前面，不经常改动的参数放后面。不经常改动的参数就可以作为默认参数。

使用默认参数有什么好处？最大的好处是能降低调用函数的难度。

3.2 可变参数

在 Python 函数中，还可以定义可变参数。顾名思义，可变参数就是传入的参数个数是可变的，可以是 1 个、2 个到任意个，还可以是 0 个。

我们以数学题为例子，给定一组数字 $a, b, c, \dots$ ，请计算 $a+b+c+\dots$ 。

要定义出这个函数，我们必须确定输入的参数。由于参数个数不确定，我们首先想到可以把 $a, b, c, \dots$ 作为一个 list 或 tuple 传进来，这样，函数可以定义如下：

```
def sum_my(n):
    d = 0
    for n in n:
        d = d + n
    return d
```

但是调用的时候，需要先组装出一个 list 或 tuple：

```
sum_my([1, 2, 3])
```

6

```
sum_my((1, 3, 5, 7))
```

16

如果利用可变参数，我们把函数的参数改为可变参数：

```
def sum_my(*n):
```

```
    d = 0
```

```
    for n in n:
```

```
        d = d + n
```

```
    return d
```

调用函数的方式可以简化成这样：

```
sum_my(1, 2, 3)
```

6

```
sum_my(1, 3, 5, 7)
```

16

定义可变参数和定义 list 或 tuple 参数相比，仅仅在参数前面加了一个 * 号。在函数内部，参数 numbers 接收到的是一个 tuple，因此，函数代码完全不变。但是，调用该函数时，可以传入任意个参数，包括 0 个参数：

```
sum_my()
```

0

如果已经有一个 list 或者 tuple，要调用一个可变参数怎么办？Python 允许你在 list 或 tuple 前面加一个 * 号，把 list 或 tuple 的元素变成可变参数传进去，可以这样做：

```
number = [1, 2, 3]
```

```
sum_my(*number)
```

6

这种写法相当有用，而且很常见。

3.3 关键字参数

可变参数允许你传入 0 个或任意个参数，这些可变参数在函数调用时自动组装为一个 tuple。而关键字参数允许你传入 0 个或任意个含参数名的参数，这些关键字参数在函数内部自动组装为一个 dict。请看示例：

```
def myclass(name, number, **other):
```

```
    print('name:', name, 'number:', number, 'other:', other)
```

函数 myclass 除了必选参数 name 和 number 外，还接受关键字参数 other。在调用该函数时，可以只传入必选参数：

```
myclass('Lili', 4001)
```

```
name: Lili number: 4001 other: {}
```

也可以传入任意个数的关键字参数：

```
myclass('yunyun', 4012, gender='girl', age='23')
```

```
name: yunyun number: 4012 other: {'gender': 'girl', 'age': '23'}
```

关键字参数有什么用？它可以扩展函数的功能。比如，在 myclass 函数里，我们保证能接收到 name 和 age 这两个参数，但是，如果调用者愿意提供更多的参数，我们也能收到。

试想你正在做一个用户注册的功能，除了用户名和年龄是必填项外，其他都是可选项，利用关键字参数来定义这个函数就能满足注册的需求。

3.4 混合参数

在 Python 中定义函数，可以用必选参数、默认参数、可变参数和关键字参数，这 4 种参数都可以一起使用，或者只用其中某些，但是请注意，参数定义的顺序必须是：必选参数、默认参数、可变参数和关键字参数。

比如定义一个函数，包含上述 4 种参数：

```
def number(a, b=0, *c, **d):  
    print('a =', a, 'b =', b, 'c =', c, 'd =', d)
```

同学们在记忆时，可尝试对应四个选项，采用巧记的方式：a 是 a，b 为 0，c 前一颗星，d 前两颗星，a 必须定义，b 默认为 0，c 可多定义，d 为小字典，可额外补充。

在函数调用的时候，Python 解释器自动按照参数位置和参数名把对应的参数传进去。

```
number(1,2,3,3,d=4)  
a = 1 b = 2 c = (3, 3) d = {'d': 4}
```

第五节：numpy

Numpy 是高性能科学计算和数据分析的基础包。Numpy 本身并没有提供多么高级的数据分析功能，理解 Numpy 数组以及面向数组的计算将有助于你提高处理数据能力。本节内容介绍 ndarray 数组和矩阵。

一、ndarray 数组基础

使用 ndarray 数组，需要导入 Numpy 函数库。

导入方法 1：直接导入该函数库：

```
from numpy import *
```

导入方法 2：指定导入库的别名（在引入多个库的时候，推荐使用这个方法）。

```
import numpy as np
```

下面正式进入 Numpy 的数组世界。如果没有说明，所称数组均为 Numpy 数组对象，与 Python 的列表和 array 模块无关。

1.1 创建数组

创建数组是进行数组计算的第一步，可以通过 numpy 库包内的 array() 函数定义数组实例对象，其参数为 Python 的序列对象，如果想定义多维数组，则传递多层嵌套的序列。

例如下面这条语句定义了一个二维数组，其大小为 (2,3)，即有 2 行，3 列。

```
a = np.array([[1,2,3.0],[2,3,4.0]])  
print(a)
```

```
[[ 1.  2.  3.]  
 [ 2.  3.  4.]]
```

以下三种操作，可以帮助我们查看数组的一些属性：

查看行数使用 ndim

```
a.ndim
```

```
2
```

查看数组的维数使用 shape，返回(n,m)，其中 n 为行数，m 为列数。

```
a.shape
```

```
(2, 3)
```

查看元素的类型使用 dtype，比如 numpy.int32、numpy.float64

```
a.dtype
```

```
dtype('float64')
```

1.2 特殊数组

Numpy 的特殊数组主要有以下三种：

zeros 数组：全零数组，元素全为 0；

ones 数组：全 1 数组，元素全为 1；

empty 数组：空数组，元素全近似为 0；

创建方法分别为：

zeros 数组

```
np.zeros((2,5))  
array([[ 0.,  0.,  0.,  0.,  0.],  
       [ 0.,  0.,  0.,  0.,  0.]])
```

ones 数组

```
np.ones((6,7))  
array([[ 1.,  1.,  1.,  1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.,  1.,  1.,  1.],  
       [ 1.,  1.,  1.,  1.,  1.,  1.,  1.]])
```

empty 数组

```
np.empty((3,3))  
  
array([[ 6.94415038e-310,  6.94415038e-310,  6.94415038e-310],  
       [ 6.94415038e-310,  6.94415038e-310,  6.94415038e-310],  
       [ 6.94415038e-310,  6.94415039e-310,  6.94415038e-310]])
```

1.3 序列数组

arange 函数：属于 Numpy 库，其参数依次为：开始值、结束值、步长。

```
np.arange(1,100,5)  
  
array([ 1,  6, 11, 16, 21, 26, 31, 36, 41, 46, 51, 56, 61, 66, 71, 76, 81,  
       86, 91, 96])
```

linspace 函数创建等差序列数组，其参数依次为：开始值、结束值、元素数量。

```
np.linspace(1,100,5)  
  
array([ 1. , 25.75, 50.5 , 75.25, 100. ])
```

1.4 数组索引

Numpy 数组的每个元素、每行元素、每列元素都可以用索引访问。注意：索引是从 0 开始的。

其操作与列表基本相同，以下是三种操作方式。

```
a = np.array([[1,2,4.0],[3,6,9]])
```

1.取 a 的第一行元素

```
a[0]
```

```
array([ 1.,  2.,  4.])
```

2.取 a 的第二列元素

```
a[:,1]
```

```
array([ 2.,  6.])
```

3.取 a 的第一行的第三个元素

```
a[0,2]
```

4.0

1.5 数组运算，以下介绍 10 种常用运算

```
a = np.array([1,2,3])
```

```
b = np.array([4.,5,6])
```

1.加法运算

```
a + b
```

```
array([ 5., 7., 9.])
```

2.减法运算

```
a - b
```

```
array([-3., -3., -3.])
```

3.乘法运算

```
a * b
```

```
array([ 4., 10., 18.])
```

4.乘方运算：a 的 2 次方

```
a ** 2
```

```
array([1, 4, 9])
```

5.除法运算

```
a/b
```

```
array([ 0.25, 0.4 , 0.5 ])
```

6.数组点乘

```
np.dot(a,b)
```

32.0

7.判断大小，返回 bool 值

```
a >= 2
```

```
array([False,  True,  True], dtype=bool)
```

8.a 中最大的元素

```
a.max()
```

3

9.a 中最小的元素

```
a.min()
```

1

10.a 的和

```
a.sum()
```

6

1.6 数组拷贝

数组的拷贝分为浅拷贝和深拷贝两种，浅拷贝通过数组变量的复制完成，深拷贝使用数组对象的 `copy` 方法完成。

浅拷贝只拷贝数组的引用，如果对拷贝对象修改。原数组也将修改。

以下是浅拷贝的演示：

```
a = np.ones((2,3))
```

```
print(a)
```

```
[[ 1., 1., 1.],  
 [ 1., 1., 1.]]
```

b 为 a 的浅拷贝

```
b = a
```

```
print(b)
```

```
[[ 1., 1., 1.],  
 [ 1., 1., 1.]]
```

对 b 进行修改，a 也会被修改

```
b[1,2] = 9
```

```
print(a)
```

```
[[ 1. 1. 1.]  
 [ 1. 1. 9.]]
```

以下是深拷贝的演示：

深拷贝会复制一份和原数组一样的数组，但他们在内存中是分开存放的，所以改变拷贝数组，原数组不会改变。

```
a = np.ones((2,3))
```

```
print(a)
```

```
[[ 1. 1. 1.]  
 [ 1. 1. 1.]]
```

```
b = a.copy()
```

```
b[1,2] = 9
```

```
print(b)
```

```
[[ 1., 1., 1.],  
 [ 1., 1., 9.]]
```

```
print(a)
```

```
[[ 1., 1., 1.],  
 [ 1., 1., 1.]]
```

二、矩阵

2.1 创建矩阵

Numpy 的矩阵对象与数组对象相似，主要不同之处在于，矩阵对象的计算遵循矩阵数学运算规律。

矩阵使用 `matrix` 函数创建，以 (2,2) 大小的矩阵为例（2 行 2 列），定义方法如下：

```
a = np.matrix([[1.0,2.0],[3.0,4.0]])  
print(a)
```

```
[[ 1.  2.]  
 [ 3.  4.]]
```

`type` 函数可以查看 a 的类型

```
type(a)
```

```
numpy.matrixlib.defmatrix.matrix
```

2.2 矩阵运算

矩阵的常用数学运算有转置、乘法、求逆等。分别一一实例讲述。

创建一个矩阵

```
b=np.matrix([[4.0,6.0],[2.0,8.0]])  
print(b)
```

```
[[ 4.  6.]  
 [ 2.  8.]]
```

1.转置操作

b.T

```
matrix([[ 4.,  2.],  
        [ 6.,  8.]])
```

2.矩阵乘法

```
c=np.matrix([[2.0,2.0],[2.0,2.0]])  
b*c
```

```
matrix([[ 20.,  20.],  
        [ 20.,  20.]])
```

3.逆矩阵

b.I

```
matrix([[ 0.4, -0.3],  
        [-0.1,  0.2]])
```

第六节：pandas 基础

pandas 是基于 Numpy 构建的，让以 Numpy 为中心的应用变得更加简单。pandas 是公认的数据处理利器，本章内容主要介绍 DataFrame 数据结构，在此基础上进行数据处理。除了 DataFrame 格式，pandas 还包括 series、Panel。

格式	数组	释义
Series	一维数组	与 Numpy 中的一维 array 类似。
DataFrame	二维的表格型数据结构	可以将 DataFrame 理解为 Series 的容器
Panel	三维的数组	可以理解为 DataFrame 的容器

开始之前，我们首先掌握导入 pandas 库，方式如下：

```
import pandas as pd
```

注意：以下内容必须在导入 pandas 库之后才能运行。

一、Series 和 DataFrame 介绍

1.Series

由一组数据和与之相关的索引组成。可通过传递一个 list 对象来创建一个 Series，pandas 会默认创建整型索引。

创建一个 Series：

```
s = pd.Series([1,3,5,7,6,8])
```

```
print(s)
```

```
0    1
```

```
1    3
```

```
2    5
```

```
3    7
```

```
4    6
```

```
5    8
```

```
dtype: int64
```

获取 Series 的索引：

```
s.index
```

```
RangeIndex(start=0, stop=6, step=1)
```

2.DataFrame

DataFrame 是一个表格型的数据结构，它含有一组有序的列，每一列的数据结构都是相同的，而不同的列之间则可以是不同的数据结构。DataFrame 中的每一行是一个记录，名称为 Index 的一个元素，而每一列则为一个字段，是这个记录的一个属性，DataFrame 既有行索引也有列索引。

创建 DataFrame

首先来看如何从字典创建 DataFrame。

```
d = {'one': [1, 2, 3], 'two': [1, 2, 3]}
df = pd.DataFrame(d, index=['a', 'b', 'c'])
print(df)
```

```
   one two
a    1    1
b    2    2
c    3    3
```

可以使用 `dataframe.index` 和 `dataframe.columns` 来查看 `DataFrame` 的行和列，`dataframe.values` 则以数组的形式返回 `DataFrame` 的元素：

```
print(df.index) #查看行
print(df.columns) #查看列
print(df.values) #查看元素
```

```
Index(['a', 'b', 'c'], dtype='object')
Index(['one', 'two'], dtype='object')
[[1 1]
 [2 2]
 [3 3]]
```

`DataFrame` 从值是数组的字典创建时，其各个数组的长度需要相同，加强印象，可参考以下报错的例子。

```
d = {'one': [1, 2], 'two': [1, 2, 3]}
df = pd.DataFrame(d, index=['a', 'b', 'c'])
print(df)
```

```
ValueError: Shape of passed values is (2, 2), indices imply (2, 3)
```

如果 `DataFrame` 的值是非数组时，没有这一限制，且自动将缺失值补成 `NaN`。如下示例

```
d = [{'a': 1.6, 'b': 2}, {'a': 3, 'b': 6, 'c': 9}]
df = pd.DataFrame(d)
print(df)
```

```
   a  b  c
0  1.6  2 NaN
1  3.0  6  9.0
```

在实际处理数据时，有时需要创建一个空的 `DataFrame`，可以这么做：

```
df = pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

另一种创建 DataFrame 的方法十分有用，那就是使用 concat 函数创建 DataFrame，其主要是通过两个行或列相同的 DataFrame 链接成一个。

```
a= [{'a': 1.6, 'b': 2}, {'a': 3, 'b': 6}]
df1 = pd.DataFrame(a)
b= [{'a': 4, 'b': 5}]
df2 = pd.DataFrame(b)
df = pd.concat([df1, df2], axis=0)
print(df1)
print(df2)
print(df)
```

```
      a b
0  1.6  2
1  3.0  6
```

```
      a b
0  4  5
```

```
      a b
0  1.6  2
1  3.0  6
0  4.0  5
```

注意：concat 函数内有 axis 参数，其中的 axis=1 表示按列进行合并，axis=0 表示按行合并

二、数据查看

MindGO 量化交易平台上大部分获取数据的函数，最终以 DataFrame 或 Dict(字典)格式呈现。接下来重点介绍 DataFrame 格式的数据查看，数据处理。以 MindGO 平台获取的数据为例进行讲解：

该部分内容需在 MindGo 研究环境中练习。

获取贵州茅台近 10 个工作日的开盘价、最高价、最低价、收盘价，获取格式即为 DataFrame

```
price= get_price('600519.SH', None, '20180125', '1d', ['open', 'high', 'low', 'close'], False, 'pre', 20, is_panel=1)
print(price)
```

```
      close  high  low  open
2017-12-28  718.69  719.90  671.32  687.00
2017-12-29  697.49  726.50  691.60  718.00
2018-01-02  703.85  710.16  689.89  700.00
2018-01-03  715.86  721.40  699.74  701.50
2018-01-04  737.07  743.50  719.33  721.40
2018-01-05  738.36  746.03  728.22  741.00
2018-01-08  752.13  756.50  735.02  735.02
2018-01-09  782.52  783.00  752.21  752.21
2018-01-10  785.71  788.88  773.48  785.00
2018-01-11  774.81  788.00  772.00  787.00
```

```
2018-01-12 788.42 788.80 767.02 773.77
2018-01-15 785.37 799.06 779.02 793.46
2018-01-16 772.94 788.61 768.00 780.48
2018-01-17 747.93 774.00 738.51 770.00
2018-01-18 750.74 765.00 744.09 747.93
2018-01-19 750.18 758.90 739.02 752.90
2018-01-22 773.64 774.00 751.81 751.81
2018-01-23 773.78 780.00 768.60 777.81
2018-01-24 764.46 776.46 758.60 776.44
2018-01-25 769.16 776.00 751.00 761.00
```

以下为数据查看常用的八项操作：

1.查看前几条数据：

```
price.head()
```

```
      close  high  low  open
2017-12-28 718.69 719.90 671.32 687.0
2017-12-29 697.49 726.50 691.60 718.0
2018-01-02 703.85 710.16 689.89 700.0
2018-01-03 715.86 721.40 699.74 701.5
2018-01-04 737.07 743.50 719.33 721.4
```

2.查看后几条数据：

```
price.tail()
```

```
      close  high  low  open
2018-01-19 750.18 758.90 739.02 752.90
2018-01-22 773.64 774.00 751.81 751.81
2018-01-23 773.78 780.00 768.60 777.81
2018-01-24 764.46 776.46 758.60 776.44
2018-01-25 769.16 776.00 751.00 761.00
```

3.查看 DataFrame 的索引

```
price.index
```

```
DatetimeIndex(['2017-12-28', '2017-12-29', '2018-01-02', '2018-01-03',
               '2018-01-04', '2018-01-05', '2018-01-08', '2018-01-09',
               '2018-01-10', '2018-01-11', '2018-01-12', '2018-01-15',
               '2018-01-16', '2018-01-17', '2018-01-18', '2018-01-19',
               '2018-01-22', '2018-01-23', '2018-01-24', '2018-01-25'],
              dtype='datetime64[ns]', freq=None)
```

4.查看 DataFrame 的列名

```
price.columns
```

```
Index(['close', 'high', 'low', 'open'], dtype='object')
```

5.查看 DataFrame 的值

price.values

```
array([[ 718.69,  719.9 ,  671.32,  687.  ],
       [ 697.49,  726.5 ,  691.6 ,  718.  ],
       [ 703.85,  710.16,  689.89,  700.  ],
       [ 715.86,  721.4 ,  699.74,  701.5 ],
       [ 737.07,  743.5 ,  719.33,  721.4 ],
       [ 738.36,  746.03,  728.22,  741.  ],
       [ 752.13,  756.5 ,  735.02,  735.02],
       [ 782.52,  783.  ,  752.21,  752.21],
       [ 785.71,  788.88,  773.48,  785.  ],
       [ 774.81,  788.  ,  772.  ,  787.  ],
       [ 788.42,  788.8 ,  767.02,  773.77],
       [ 785.37,  799.06,  779.02,  793.46],
       [ 772.94,  788.61,  768.  ,  780.48],
       [ 747.93,  774.  ,  738.51,  770.  ],
       [ 750.74,  765.  ,  744.09,  747.93],
       [ 750.18,  758.9 ,  739.02,  752.9 ],
       [ 773.64,  774.  ,  751.81,  751.81],
       [ 773.78,  780.  ,  768.6 ,  777.81],
       [ 764.46,  776.46,  758.6 ,  776.44],
       [ 769.16,  776.  ,  751.  ,  761.  ]])
```

6.使用 describe() 函数对于数据的快速统计汇总：

price.describe()

	close	high	low	open
count	20.000000	20.000000	20.000000	20.000000
mean	754.155500	763.235000	739.924000	750.686500
std	28.005539	26.794003	31.251968	31.581411
min	697.490000	710.160000	671.320000	687.000000
25%	738.037500	745.397500	725.997500	731.615000
50%	758.295000	774.000000	747.545000	752.555000
75%	774.037500	784.250000	767.265000	776.782500
max	788.420000	799.060000	779.020000	793.460000

7.对数据的转置：

price.T

	2017-12-28	2017-12-29	2018-01-02	2018-01-03	2018-01-04	2018-01-05 \
close	718.69	697.49	703.85	715.86	737.07	738.36
high	719.90	726.50	710.16	721.40	743.50	746.03
low	671.32	691.60	689.89	699.74	719.33	728.22
open	687.00	718.00	700.00	701.50	721.40	741.00

	2018-01-08	2018-01-09	2018-01-10	2018-01-11	2018-01-12	2018-01-15 \
close	752.13	782.52	785.71	774.81	788.42	785.37
high	756.50	783.00	788.88	788.00	788.80	799.06
low	735.02	752.21	773.48	772.00	767.02	779.02
open	735.02	752.21	785.00	787.00	773.77	793.46

	2018-01-16	2018-01-17	2018-01-18	2018-01-19	2018-01-22	2018-01-23 \
close	772.94	747.93	750.74	750.18	773.64	773.78
high	788.61	774.00	765.00	758.90	774.00	780.00
low	768.00	738.51	744.09	739.02	751.81	768.60
open	780.48	770.00	747.93	752.90	751.81	777.81

	2018-01-24	2018-01-25
close	764.46	769.16
high	776.46	776.00
low	758.60	751.00
open	776.44	761.00

8.按列对 DataFrame 进行排序

```
print(price.sort_values(by='open', ascending=False))
```

	close	high	low	open
2018-01-15	785.37	799.06	779.02	793.46
2018-01-11	774.81	788.00	772.00	787.00
2018-01-10	785.71	788.88	773.48	785.00
2018-01-16	772.94	788.61	768.00	780.48
2018-01-23	773.78	780.00	768.60	777.81
2018-01-24	764.46	776.46	758.60	776.44
2018-01-12	788.42	788.80	767.02	773.77
2018-01-17	747.93	774.00	738.51	770.00
2018-01-25	769.16	776.00	751.00	761.00
2018-01-19	750.18	758.90	739.02	752.90
2018-01-09	782.52	783.00	752.21	752.21
2018-01-22	773.64	774.00	751.81	751.81
2018-01-18	750.74	765.00	744.09	747.93
2018-01-05	738.36	746.03	728.22	741.00
2018-01-08	752.13	756.50	735.02	735.02
2018-01-04	737.07	743.50	719.33	721.40
2017-12-29	697.49	726.50	691.60	718.00
2018-01-03	715.86	721.40	699.74	701.50
2018-01-02	703.85	710.16	689.89	700.00
2017-12-28	718.69	719.90	671.32	687.00

注意：sort_values 函数内置参数有 by 和 ascending，by 参数是排序指定列，ascending 是排序顺序，False 是从大到小，True 是从小到大。

三、选择数据

依旧采用上个小节案例，继续讲述选择数据的八项基本操作。

1.选择一列数据，选取开盘价这列数据：

```
price['open']
```

2017-12-28	687.00
2017-12-29	718.00
2018-01-02	700.00
2018-01-03	701.50

```
2018-01-04    721.40
2018-01-05    741.00
2018-01-08    735.02
2018-01-09    752.21
2018-01-10    785.00
2018-01-11    787.00
2018-01-12    773.77
2018-01-15    793.46
2018-01-16    780.48
2018-01-17    770.00
2018-01-18    747.93
2018-01-19    752.90
2018-01-22    751.81
2018-01-23    777.81
2018-01-24    776.44
2018-01-25    761.00
```

```
Name: open, dtype: float64
```

同学们动手试试 `price.open~`
它与 `price['open']` 是等效的！

2. 选择多列数据：

```
price[['open','close']]
```

```
          open  close
2017-12-28  687.00  718.69
2017-12-29  718.00  697.49
2018-01-02  700.00  703.85
2018-01-03  701.50  715.86
2018-01-04  721.40  737.07
2018-01-05  741.00  738.36
2018-01-08  735.02  752.13
2018-01-09  752.21  782.52
2018-01-10  785.00  785.71
2018-01-11  787.00  774.81
2018-01-12  773.77  788.42
2018-01-15  793.46  785.37
2018-01-16  780.48  772.94
2018-01-17  770.00  747.93
2018-01-18  747.93  750.74
2018-01-19  752.90  750.18
2018-01-22  751.81  773.64
2018-01-23  777.81  773.78
2018-01-24  776.44  764.46
2018-01-25  761.00  769.16
```

注意：`price[['open','close']]` 中 `['open','close']` 是一个由两个字符串（列名）组成的列表，会自动对应到整个 `DataFrame` 表结构中，获取到相应的数据。

同学们试试 `price['open','close']`，看看能不能获取到数据~

3.选择多行:

```
price[0:3]
```

	close	high	low	open
2017-12-28	718.69	719.90	671.32	687.0
2017-12-29	697.49	726.50	691.60	718.0
2018-01-02	703.85	710.16	689.89	700.0

4.按 index 选取多行:

```
price['2018-01-24':'2018-01-25']
```

	close	high	low	open
2018-01-24	764.46	776.46	758.6	776.44
2018-01-25	769.16	776.00	751.0	761.00

5.使用标签选取数据:

```
price.loc[行标签,列标签]
```

```
price.loc['a':'b'] #选取 ab 两行数据
```

```
price.loc[:, 'open'] #选取 open 列的数据
```

price.loc 的第一个参数是行标签，第二个参数为列标签，两个参数既可以是列表也可以是单个字符，如果两个参数都为列表则返回的是 DataFrame，否则，则为 Series。

```
price.loc['2018-01-24', 'open']
```

```
776.440000000000005
```

```
price.loc['2018-01-24':'2018-01-25']
```

	close	high	low	open
2018-01-24	764.46	776.46	758.6	776.44
2018-01-25	769.16	776.00	751.0	761.00

```
price.loc[:, 'open']
```

2017-12-28	687.00
2017-12-29	718.00
2018-01-02	700.00
2018-01-03	701.50
2018-01-04	721.40
2018-01-05	741.00
2018-01-08	735.02
2018-01-09	752.21
2018-01-10	785.00
2018-01-11	787.00
2018-01-12	773.77
2018-01-15	793.46
2018-01-16	780.48
2018-01-17	770.00
2018-01-18	747.93
2018-01-19	752.90
2018-01-22	751.81
2018-01-23	777.81
2018-01-24	776.44
2018-01-25	761.00

```
Name: open, dtype: float64
```

```
price.loc['2018-01-24':'2018-01-25','open']
```

```
2018-01-24    776.44
```

```
2018-01-25    761.00
```

```
Name: open, dtype: float64
```

6..使用位置选取数据:

```
price.iloc[行位置,列位置]
```

```
price.iloc[1,1] #选取第二行，第二列的值，返回的为单个值
```

```
price.iloc[[0,2],:] #选取第一行及第三行的数据
```

```
price.iloc[0:2,:] #选取第一行到第三行（不包含）的数据
```

```
price.iloc[:,1] #选取所有记录的第三列的值，返回的为一个 Series
```

```
price.iloc[1,:] #选取第二行数据，返回的为一个 Series
```

```
price.iloc[1,1] # 选取第二行，第二列的值，返回的为单个值
```

```
726.5
```

```
price.iloc[[0,2],:] # 选取第一行及第三行的数据
```

```
           close  high  low  open
2017-12-28  718.69  719.90  671.32  687.0
2018-01-02  703.85  710.16  689.89  700.0
```

```
price.iloc[0:2,:] # 选取第一行到第三行（不包含）的数据
```

```
           close  high  low  open
2017-12-28  718.69  719.9  671.32  687.0
2017-12-29  697.49  726.5  691.60  718.0
```

```
price.iloc[:,1] # 选取所有记录的第二列的值，返回的为一个 Series
```

```
2017-12-28    719.90
2017-12-29    726.50
2018-01-02    710.16
2018-01-03    721.40
2018-01-04    743.50
2018-01-05    746.03
2018-01-08    756.50
2018-01-09    783.00
2018-01-10    788.88
2018-01-11    788.00
2018-01-12    788.80
2018-01-15    799.06
2018-01-16    788.61
2018-01-17    774.00
2018-01-18    765.00
2018-01-19    758.90
2018-01-22    774.00
2018-01-23    780.00
2018-01-24    776.46
```

```
2018-01-25    776.00
Name: high, dtype: float64
```

price.iloc[1,:] # 选取第一行数据，返回的为一个 Series

```
close    697.49
high     726.50
low      691.60
open     718.00
Name: 2017-12-29 00:00:00, dtype: float64
```

7.更广义的切片方式是使用.ix，它自动根据给到的索引类型判断是使用位置还是标签进行切片

```
price.ix[1,1]
price.ix['a':'b']
price.ix[1,1]
```

```
726.5
```

```
price.ix['2018-01-24':'2018-01-25']
```

```
           close  high  low  open
2018-01-24  764.46  776.46  758.6  776.44
2018-01-25  769.16  776.00  751.0  761.00
```

```
price.ix['2018-01-24','open']
```

```
776.44
```

```
price.ix[1,'open']
```

```
718.0
```

```
price.ix['2018-01-24',0]
```

```
764.46
```

8.通过逻辑指针进行数据切片：

```
price[逻辑条件]
price[price.one >= 2] #单个逻辑条件
price[(price.one >=1 ) & (df.one < 3) ] #多个逻辑条件组合
```

筛选出 open 大于 750 的数据

```
price[price.open > 750]
```

```
           close  high  low  open
2018-01-09  782.52  783.00  752.21  752.21
2018-01-10  785.71  788.88  773.48  785.00
2018-01-11  774.81  788.00  772.00  787.00
2018-01-12  788.42  788.80  767.02  773.77
2018-01-15  785.37  799.06  779.02  793.46
```

```
2018-01-16 772.94 788.61 768.00 780.48
2018-01-17 747.93 774.00 738.51 770.00
2018-01-19 750.18 758.90 739.02 752.90
2018-01-22 773.64 774.00 751.81 751.81
2018-01-23 773.78 780.00 768.60 777.81
2018-01-24 764.46 776.46 758.60 776.44
2018-01-25 769.16 776.00 751.00 761.00
```

筛选出 open 大于 750 的数据，并且 close 小于 770 的数据
price[(price.open > 750) & (price.close < 770)]

```
      close  high  low  open
2018-01-17 747.93 774.00 738.51 770.00
2018-01-19 750.18 758.90 739.02 752.90
2018-01-24 764.46 776.46 758.60 776.44
2018-01-25 769.16 776.00 751.00 761.00
```

使用 条件过滤来更改数据。

```
price[price>780]
```

```
      close  high  low  open
2017-12-28  NaN   NaN  NaN   NaN
2017-12-29  NaN   NaN  NaN   NaN
2018-01-02  NaN   NaN  NaN   NaN
2018-01-03  NaN   NaN  NaN   NaN
2018-01-04  NaN   NaN  NaN   NaN
2018-01-05  NaN   NaN  NaN   NaN
2018-01-08  NaN   NaN  NaN   NaN
2018-01-09 782.52 783.00  NaN   NaN
2018-01-10 785.71 788.88  NaN 785.00
2018-01-11  NaN 788.00  NaN 787.00
2018-01-12 788.42 788.80  NaN   NaN
2018-01-15 785.37 799.06  NaN 793.46
2018-01-16  NaN 788.61  NaN 780.48
2018-01-17  NaN   NaN  NaN   NaN
2018-01-18  NaN   NaN  NaN   NaN
2018-01-19  NaN   NaN  NaN   NaN
2018-01-22  NaN   NaN  NaN   NaN
2018-01-23  NaN   NaN  NaN   NaN
2018-01-24  NaN   NaN  NaN   NaN
2018-01-25  NaN   NaN  NaN   NaN
```

观察可以发现，price 中小于等于 780 的数都变为 NaN。

我们还可以把大于 780 的数赋值为 1.

```
price[price > 780] = 1
```

	close	high	low	open
2017-12-28	718.69	719.90	671.32	687.00
2017-12-29	697.49	726.50	691.60	718.00
2018-01-02	703.85	710.16	689.89	700.00
2018-01-03	715.86	721.40	699.74	701.50
2018-01-04	737.07	743.50	719.33	721.40
2018-01-05	738.36	746.03	728.22	741.00
2018-01-08	752.13	756.50	735.02	735.02
2018-01-09	1.00	1.00	752.21	752.21
2018-01-10	1.00	1.00	773.48	1.00
2018-01-11	774.81	1.00	772.00	1.00
2018-01-12	1.00	1.00	767.02	773.77
2018-01-15	1.00	1.00	779.02	1.00
2018-01-16	772.94	1.00	768.00	1.00
2018-01-17	747.93	774.00	738.51	770.00
2018-01-18	750.74	765.00	744.09	747.93
2018-01-19	750.18	758.90	739.02	752.90
2018-01-22	773.64	774.00	751.81	751.81
2018-01-23	773.78	780.00	768.60	777.81
2018-01-24	764.46	776.46	758.60	776.44
2018-01-25	769.16	776.00	751.00	761.00

使用 isin() 方法来过滤在指定列中的数据, 案例延续上面赋值后的 price

选取 high 列中数为 1 和 774.00 的数。

```
price[price['high'].isin([1, 774.00])]
```

	close	high	low	open
2018-01-09	1.00	1.0	752.21	752.21
2018-01-10	1.00	1.0	773.48	1.00
2018-01-11	774.81	1.0	772.00	1.00
2018-01-12	1.00	1.0	767.02	773.77
2018-01-15	1.00	1.0	779.02	1.00
2018-01-16	772.94	1.0	768.00	1.00
2018-01-17	747.93	774.0	738.51	770.00
2018-01-22	773.64	774.0	751.81	751.81

四、Panel

MindGo 量化交易平台的 get_price 函数, 如果是获取多支股票数据, 则返回 pandas.Panel 对象。pane 其实就是一张一张 DataFrame 整合。

获取贵州茅台, 招商银行, 中信证券这三只股票近 10 个工作日的开盘价、最高价、最低价、收盘价, 获取格式即为 DataFrame

```
price = get_price(['600519.SH', '600036.SH', '600030.SH'], None, '20180125', '1d', ['open', 'high', 'low', 'close'], False, 'pre', 20, is_panel=1)
print(price)
```

```
<class 'pandas.core.panel.Panel'>
```

```
Dimensions: 4 (items) x 20 (major_axis) x 3 (minor_axis)
```

Items axis: close to open

Major_axis axis: 2017-12-28 00:00:00 to 2018-01-25 00:00:00

Minor_axis axis: 600030.SH to 600519.SH

注意：现在这个 price 不是一张 DataFrame，而是三个股票的 DataFrame 了，那么我们需要通过股票代码、数据字段下标，来分别获取多张 DataFrame，之后的操作就是操作单张 DataFrame 了。

price['close']#获取三个股票的收盘价，注意获取后是个 DataFrame

	600030.SH	600036.SH	600519.SH
2017-12-28	18.12	28.63	718.69
2017-12-29	18.10	29.02	697.49
2018-01-02	18.44	29.62	703.85
2018-01-03	18.61	29.97	715.86
2018-01-04	18.67	29.65	737.07
2018-01-05	18.88	30.10	738.36
2018-01-08	19.54	29.47	752.13
2018-01-09	19.44	29.77	782.52
2018-01-10	19.61	30.53	785.71
2018-01-11	19.28	30.92	774.81
2018-01-12	19.33	31.51	788.42
2018-01-15	19.45	31.94	785.37
2018-01-16	20.25	31.89	772.94
2018-01-17	20.94	31.69	747.93
2018-01-18	21.41	32.32	750.74
2018-01-19	21.29	32.46	750.18
2018-01-22	21.20	33.08	773.64
2018-01-23	21.21	34.05	773.78
2018-01-24	22.92	33.85	764.46
2018-01-25	22.33	33.41	769.16

price['open']#获取开盘价股票数据，注意获取的还是 DataFrame

	600030.SH	600036.SH	600519.SH
2017-12-28	18.06	28.75	687.00
2017-12-29	18.12	28.63	718.00
2018-01-02	18.13	29.02	700.00
2018-01-03	18.36	29.74	701.50
2018-01-04	18.64	30.28	721.40
2018-01-05	18.68	29.87	741.00
2018-01-08	19.00	29.92	735.02
2018-01-09	19.55	29.52	752.21
2018-01-10	19.47	29.66	785.00
2018-01-11	19.46	30.52	787.00
2018-01-12	19.25	31.12	773.77
2018-01-15	19.25	31.48	793.46
2018-01-16	19.26	31.80	780.48
2018-01-17	20.50	32.10	770.00
2018-01-18	21.15	32.10	747.93
2018-01-19	21.36	32.66	752.90
2018-01-22	21.10	32.18	751.81
2018-01-23	21.37	33.20	777.81

2018-01-24	21.40	34.25	776.44
2018-01-25	22.50	34.01	761.00

`price['2018-01-11']` #获取 2018-01-11 日期的三个股票的数据

KeyError: '2018-01-11'

注意这是不可行的，思考下为什么？

第七节：pandas 进阶

本节为 pandas 进阶内容，核心还是 DataFrame 数据处理，注意包括缺失数据处理、函数的应用和映射、数据规整等。

开始之前首先导入库: numpy 和 pandas

```
import pandas as pd
import numpy as np
```

一、缺失数据处理

还是获取 MindGo 平台的数据来演示：

获取招商银行近 10 个工作日的开盘价、最高价、最低价、收盘价。并将部分数据赋值为 NaN，假设为缺失部分。

```
price= get_price('600036.SH', None, '20180125', '1d', ['open', 'high', 'low', 'close'], False, 'pre', 10,
is_panel=1)
price[price > 34] = np.nan
print(price)
```

	close	high	low	open
2018-01-12	31.51	31.58	31.02	31.12
2018-01-15	31.94	32.40	31.30	31.48
2018-01-16	31.89	32.28	31.54	31.80
2018-01-17	31.69	33.11	31.50	32.10
2018-01-18	32.32	32.75	32.10	32.10
2018-01-19	32.46	33.35	32.21	32.66
2018-01-22	33.08	33.64	32.15	32.18
2018-01-23	NaN	NaN	33.20	33.20
2018-01-24	33.85	NaN	33.45	NaN
2018-01-25	33.41	NaN	32.90	NaN

1. 去掉包含缺失值的行

```
price.dropna()
```

	close	high	low	open
2018-01-12	31.51	31.58	31.02	31.12
2018-01-15	31.94	32.40	31.30	31.48
2018-01-16	31.89	32.28	31.54	31.80
2018-01-17	31.69	33.11	31.50	32.10
2018-01-18	32.32	32.75	32.10	32.10
2018-01-19	32.46	33.35	32.21	32.66
2018-01-22	33.08	33.64	32.15	32.18

2. 对缺失值进行填充为 30

```
price.fillna(value=30)
```

	close	high	low	open
2018-01-12	31.51	31.58	31.02	31.12
2018-01-15	31.94	32.40	31.30	31.48
2018-01-16	31.89	32.28	31.54	31.80
2018-01-17	31.69	33.11	31.50	32.10
2018-01-18	32.32	32.75	32.10	32.10
2018-01-19	32.46	33.35	32.21	32.66

```

2018-01-22 33.08 33.64 32.15 32.18
2018-01-23 30.00 30.00 33.20 33.20
2018-01-24 33.85 30.00 33.45 30.00
2018-01-25 33.41 30.00 32.90 30.00

```

3.判断数据是否为 nan

```
pd.isnull(price)
```

```

      close high low open
2018-01-12 False False False False
2018-01-15 False False False False
2018-01-16 False False False False
2018-01-17 False False False False
2018-01-18 False False False False
2018-01-19 False False False False
2018-01-22 False False False False
2018-01-23  True  True False False
2018-01-24 False  True False  True
2018-01-25 False  True False  True

```

二、函数的应用和映射

再次获取 MindGo 平台的数据来演示：

获取招商银行近 10 个工作日的开盘价、最高价、最低价、收盘价。

```

price= get_price('600036.SH', None, '20180125', '1d', ['open', 'high', 'low', 'close'], False, 'pre', 10,
is_panel=1)
print(price)

```

```

      close high low open
2018-01-12 31.51 31.58 31.02 31.12
2018-01-15 31.94 32.40 31.30 31.48
2018-01-16 31.89 32.28 31.54 31.80
2018-01-17 31.69 33.11 31.50 32.10
2018-01-18 32.32 32.75 32.10 32.10
2018-01-19 32.46 33.35 32.21 32.66
2018-01-22 33.08 33.64 32.15 32.18
2018-01-23 34.05 34.17 33.20 33.20
2018-01-24 33.85 35.35 33.45 34.25
2018-01-25 33.41 34.02 32.90 34.01

```

1.列计算平均值

```
price.mean()
```

```

close    32.620
high     33.265
low      32.137
open     32.490
dtype: float64

```

2.行计算平均值

```
price.mean(1)
```

```

2018-01-12    31.3075
2018-01-15    31.7800
2018-01-16    31.8775

```

```

2018-01-17  32.1000
2018-01-18  32.3175
2018-01-19  32.6700
2018-01-22  32.7625
2018-01-23  33.6550
2018-01-24  34.2250
2018-01-25  33.5850

```

dtype: float64

如果你担心你求均值时受到缺失值的影响，你可以：

`price.mean(axis = 1, skipna = True)` # `skipna` 参数默认是 `True` 表示排除缺失值 `axis=1` 是按行 `axis=0` 是按列

```

2018-01-12  31.3075
2018-01-15  31.7800
2018-01-16  31.8775
2018-01-17  32.1000
2018-01-18  32.3175
2018-01-19  32.6700
2018-01-22  32.7625
2018-01-23  33.6550
2018-01-24  34.2250
2018-01-25  33.5850

```

dtype: float64

三、数据规整

Pandas 提供了大量的方法能够轻松的对 Series，DataFrame 和 Panel 对象进行各种符合各种逻辑关系的合并操作，主要介绍三个常用操作。

操作方式	释义
concat	可以沿一条轴将多个对象堆叠到一起。
append	将一行连接到一个 DataFrame 上
drop_duplicates	移除重复数据

1.concat

首先我们分别获取两个 DataFrame。

```

# 获取招商银行 20180125 日的前 5 个工作日的开盘价、最高价、最低价、收盘价。
price1 = get_price('600036.SH', None, '20180125', '1d', ['open', 'high', 'low', 'close'], False, 'pre', 5,
is_panel=1)
print(price1)

```

```

      close high  low  open
2018-01-19  32.46  33.35  32.21  32.66
2018-01-22  33.08  33.64  32.15  32.18
2018-01-23  34.05  34.17  33.20  33.20
2018-01-24  33.85  35.35  33.45  34.25
2018-01-25  33.41  34.02  32.90  34.01

```

```
# 获取招商银行 20170125 日的前 5 个工作日的开盘价、最高价、最低价、收盘价。
price2= get_price('600036.SH', None, '20170125', '1d', ['open', 'high', 'low', 'close'], False, 'pre', 5,
is_panel=1)
print(price2)
```

	close	high	low	open
2017-01-19	18.52	18.74	18.51	18.55
2017-01-20	18.59	18.65	18.47	18.54
2017-01-23	18.50	18.76	18.41	18.68
2017-01-24	18.87	18.88	18.50	18.59
2017-01-25	18.88	18.95	18.68	18.81

纵向拼接(默认):

```
pd.concat([price1,price2],axis=0)
```

	close	high	low	open
2018-01-19	32.46	33.35	32.21	32.66
2018-01-22	33.08	33.64	32.15	32.18
2018-01-23	34.05	34.17	33.20	33.20
2018-01-24	33.85	35.35	33.45	34.25
2018-01-25	33.41	34.02	32.90	34.01
2017-01-19	18.52	18.74	18.51	18.55
2017-01-20	18.59	18.65	18.47	18.54
2017-01-23	18.50	18.76	18.41	18.68
2017-01-24	18.87	18.88	18.50	18.59
2017-01-25	18.88	18.95	18.68	18.81

横向拼接, index 对不上的会用 NaN 填充:

```
pd.concat([price1,price2],axis=1)
```

	close	high	low	open	close	high	low	open
2017-01-19	NaN	NaN	NaN	NaN	18.52	18.74	18.51	18.55
2017-01-20	NaN	NaN	NaN	NaN	18.59	18.65	18.47	18.54
2017-01-23	NaN	NaN	NaN	NaN	18.50	18.76	18.41	18.68
2017-01-24	NaN	NaN	NaN	NaN	18.87	18.88	18.50	18.59
2017-01-25	NaN	NaN	NaN	NaN	18.88	18.95	18.68	18.81
2018-01-19	32.46	33.35	32.21	32.66	NaN	NaN	NaN	NaN
2018-01-22	33.08	33.64	32.15	32.18	NaN	NaN	NaN	NaN
2018-01-23	34.05	34.17	33.20	33.20	NaN	NaN	NaN	NaN
2018-01-24	33.85	35.35	33.45	34.25	NaN	NaN	NaN	NaN
2018-01-25	33.41	34.02	32.90	34.01	NaN	NaN	NaN	NaN

2.append

首先获取数据

```
# 获取招商银行 20180125 日的前 5 个工作日的开盘价、最高价、最低价、收盘价。
price= get_price('600036.SH', None, '20180125', '1d', ['open', 'high', 'low', 'close'], False, 'pre', 5,
is_panel=1)
print(price)
```

	close	high	low	open
2018-01-19	32.46	33.35	32.21	32.66
2018-01-22	33.08	33.64	32.15	32.18

```
2018-01-23 34.05 34.17 33.20 33.20
2018-01-24 33.85 35.35 33.45 34.25
2018-01-25 33.41 34.02 32.90 34.01
```

```
s = price.iloc[0]
print(s)

close    32.46
high     33.35
low      32.21
open     32.66
Name: 2018-01-19 00:00:00, dtype: float64
```

```
price.append(s, ignore_index=False) # ignore_index=False 表示索引不变
```

```
      close high low open
2018-01-19 32.46 33.35 32.21 32.66
2018-01-22 33.08 33.64 32.15 32.18
2018-01-23 34.05 34.17 33.20 33.20
2018-01-24 33.85 35.35 33.45 34.25
2018-01-25 33.41 34.02 32.90 34.01
2018-01-19 32.46 33.35 32.21 32.66
```

```
price.append(s, ignore_index=True) # ignore_index=True 表示索引重置
```

```
      close high low open
0 32.46 33.35 32.21 32.66
1 33.08 33.64 32.15 32.18
2 34.05 34.17 33.20 33.20
3 33.85 35.35 33.45 34.25
4 33.41 34.02 32.90 34.01
5 32.46 33.35 32.21 32.66
```

3.移除重复数据 duplicated

延续 append 示例

```
price2=price.append(s, ignore_index=False) # ignore_index=False 表示索引不变
```

```
      close high low open
2018-01-19 32.46 33.35 32.21 32.66
2018-01-22 33.08 33.64 32.15 32.18
2018-01-23 34.05 34.17 33.20 33.20
2018-01-24 33.85 35.35 33.45 34.25
2018-01-25 33.41 34.02 32.90 34.01
2018-01-19 32.46 33.35 32.21 32.66
```

查看重复数据:

```
price2.duplicated()
```

```
2018-01-19    False
```

```
2018-01-22    False
```

```
2018-01-23    False
```

```
2018-01-24    False
```

```
2018-01-25    False
```

```
2018-01-19     True
```

```
dtype: bool
```

移除重复数据:

```
price2.drop_duplicates()
```

```
          close  high  low  open
```

```
2018-01-19  32.46  33.35  32.21  32.66
```

```
2018-01-22  33.08  33.64  32.15  32.18
```

```
2018-01-23  34.05  34.17  33.20  33.20
```

```
2018-01-24  33.85  35.35  33.45  34.25
```

```
2018-01-25  33.41  34.02  32.90  34.01
```

可以看到'2018-01-19'的重复行被删除了

第四章：经典量化策略集锦

第一篇：投资高股息股票

导语：作为策略锦集的第一篇，我们结合当前 A 股市场的价值投资风向，向大家介绍极具实践意义的股息率选股策略。

一、股息率策略阐述

无论是初入市场的新手，还是经历市场风雨后的老手，每个投资者都会面临“如何选股”的问题，凯利·赖特在 2011 年出版的《选股，一定要看股息》中讲述了股息价值投资策略，并独具匠心地指出，对于那些理解和重视股息收益的投资者，股市会给予他们丰厚的利润作为回报。杰里米 J. 西格尔在《投资者的未来》一书中阐述了一个重要观点：高股息率股票具备长期投资价值。《约翰·聂夫的成功投资》一书介绍了他在执掌温莎基金 31 年间，投资组合增长 55 倍，平均年复利回报率 13.7%，平均超出市场收益率 3.15%，而温莎基金 7 大选股策略之一就是高股息率，年均 3.15% 的超额收益中 2% 来自于股息收入，尤其是熊市中股息收益起到重要的保护作用。

那么股息率到底是什么，为何国内外专业投资者将其作为选股的重要参考标准之一？

股息率全称为股息收益率，是股息与股票价格之间的比率。其公式为：

$$\text{股息率} = \frac{\text{股息}}{\text{股票买入价}} * 100\%$$
，如果连续多年个股股息率超过 1 年期银行存款利率，则这只股票视为收益型股票，股息率越高则越吸引人。其次股息是上市公司给予投资者的回报。一般来说，一家公司的股息率越高，则一定程度上说明企业的盈利状况越好。投资高股息率的股票，可以让投资享有足够的安全边际，在一般行情里也能获取不错收益。

如何构建简单的股息率投资策略呢？

每个月第一个交易日，将当天开盘的沪深 300 指数成分股股票，按股息率数值从大到小排序，筛选出前 10 只个股作为持仓。

以下为策略实现的基本信息：

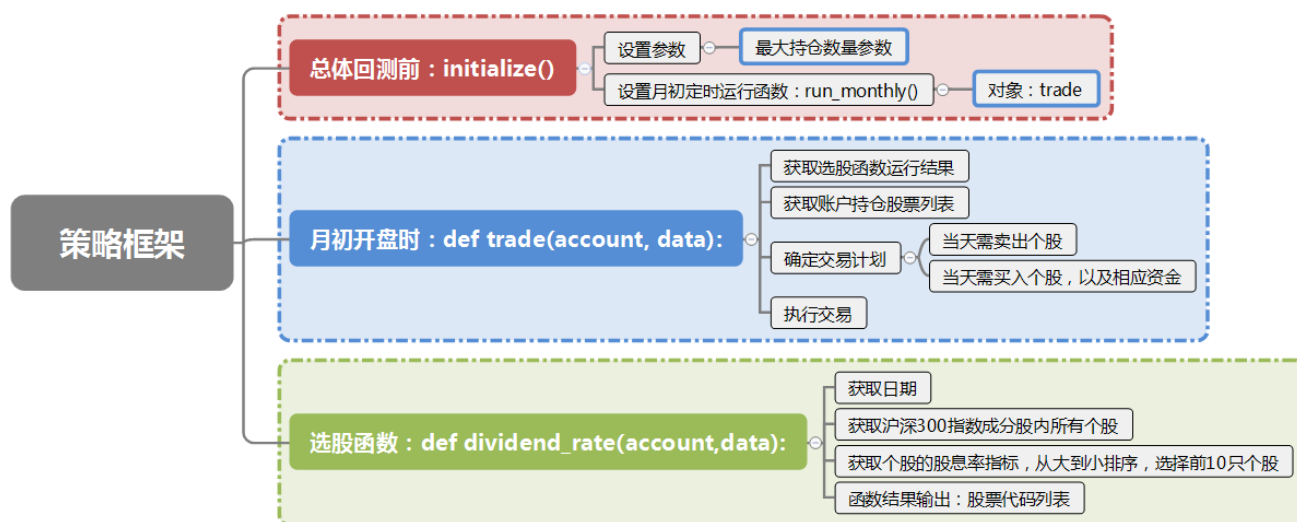
策略实现难度：1

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
initialize()	初始化函数
run_monthly()	按月定时运行函数
get_index_stocks()	获取指数成分股代码
get_last_datetime()	获取前一个交易日或者前一分钟的时间

get_factors()	查询因子数据
account.available_cash	获取账户当前可用资金
order_target()	按目标股数下单
order_target_percent()	按目标市值占比下单

二、代码示意图



三、编写释义

本策略的编写难点主要集中在 MindGo 量化交易平台的函数使用上，以下重点讲述函数使用上的注意点：

1. 定时运行函数一共有 4 个，包括：按月定时、按周定时、按日定时、自定义定时。其函数使用只需确定两点：1. 定时运行哪个函数，2. 具体定时。本策略由于按月选股调仓交易，那么选择 run_monthly 函数，需要将交易函数 trade 定时运行，具体定时为每月第一个交易日，结果即为：run_monthly(trade, date_rule=1)

2. 确定需交易股票的过程中，由于列表 list 内部数据顺序可变，于是代码中主动创建空列表，通过 append 和 remove 来删除添加操作。否则每次回测的结果都会因为买卖顺序的改动而有细微的差异。

3. 股息率指标获取，采用 get_factors 函数，该函数内至少确定 3 点：

- 获取哪些股票的因子。
- 获取哪些因子数据。
- 获取哪个时间点的数据

其次你也可以在获取函数中额外添加筛选条件或者排序条件等等，ps: 排序条件需写在 order_by()。

四、最终结果

策略回测区间：2014.01.01-2018.01.28

回测资金：100000

回测频率：日级

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码：

```
import pandas as pd
import numpy as np

#=====初始化函数=====
def initialize(account):
    account.n = 10 #设置最大持仓数量
    run_monthly(trade,date_rule=1) #将 trade 交易函数设置定时运行：每个月第一个交易日
    account.stock = [] #储存上期的股票池

#=====主函数（选股+交易）=====
def trade(account, data):
    #执行选股函数：dividend_rate(account,data)，并将结果导入，该股票列表是需要买入的个股。
    needstock_list = dividend_rate(account,data)
    #获取上期持仓个股
    holdstock_list = list(account.stock)
    #确定本期需要卖出的个股
    sell_list = list(set(holdstock_list)-set(needstock_list))
    #执行卖出操作，运用 for 循环，逐个操作。
    for s in sell_list:
        order_target(s,0)
    #确定本期需要买入的个股，其余即为继续持仓的个股
    buy_list=[]
```

```

for i in needstock_list:
    if i in holdstock_list:
        pass
    else:
        buy_list.append(i)
#确定可用资金，平分分配至需买入的个股
n=len(buy_list)
cash=account.available_cash/n
#执行买入操作
for s in range(0,n,1):
    stock=list(buy_list)[s]
    order_target_percent(stock,0.1)
#操作完毕，将选股结果放到上期股票池储存变量中，以备下次使用。
account.stock = frozenset(needstock_list)
#=====选股函数，用于筛选沪深 300 指数成分股中的股息率前 10 的个股=====
def dividend_rate(account,data):
    #获取上一个交易日的日期
    ldate = get_last_datetime().strftime('%Y%m%d')
    #获取沪深 300 指数成分股内的所有个股
    stock_list = get_index_stocks('000300.SH',ldate)
    #通过因子函数，获取交易日当天的全市场个股的股息率指标,并以股息率指标从大到小排序。
    dividendrate=get_factors(query(factor.symbol,
                                   factor.dividend_rate_12_months,
                                   ).filter(factor.symbol.in_(stock_list),
                                   factor.date==ldate).order_by(factor.dividend_rate_12_months.desc()))
    #获取前 10 只股票的代码
    needstock_list=[]
    for s in range(0,account.n,1):
        needstock_list.append(dividendrate.iloc[s]['factor_symbol'])
    #完成函数，输出即为 10 只股票代码，作为策略的股票池
    return needstock_list

```

第二篇：从“二八轮动”中学择时

导语：作为策略锦集第二篇，我们以沪深 A 股市场出了名的“二八轮动”现象为基础，向大家介绍经典择时策略：二八轮动策略。

一、策略阐述

在介绍二八轮动策略之前，我们首先来了解一下择时交易，择时交易是指利用某种方法来判断后市走势，如果判断是上涨，则买入持有；如果判断是下跌，则卖出清仓；如果判断是震荡，则进行高抛低吸，这样可以获得远远超越简单买入持有策略的收益率，所以择时交易是收益率最高的一种交易方式。通俗的讲，择时即为选择交易时机，投资者总是希望通过择时，在上涨前买入，在下跌前卖出。

“二八轮动”现象就是大小盘轮动现象，“二”代表数量占比 20%左右的大盘权重股，“八”代表数量占比 80%左右的中小盘股票，二八轮动策略就是指在大盘股与小盘股中间不断切换，轮流持有，在大小盘都看跌的时候，则买入债券或者货币基金。二八轮动策略的本质是择时，从大小盘指数中选择最佳交易时机。

二八轮动是如何轮动的呢？

择时方法：对比当前交易日收盘数据与二十个交易日前的收盘数据，选择沪深 300 指数和中证 500 指数中涨幅较大的一个，于下一个交易日切换为持有该指数 ETF，若两个指数均为下跌，则于下个交易日切换为空仓状态。调仓日为每周第一个交易日。

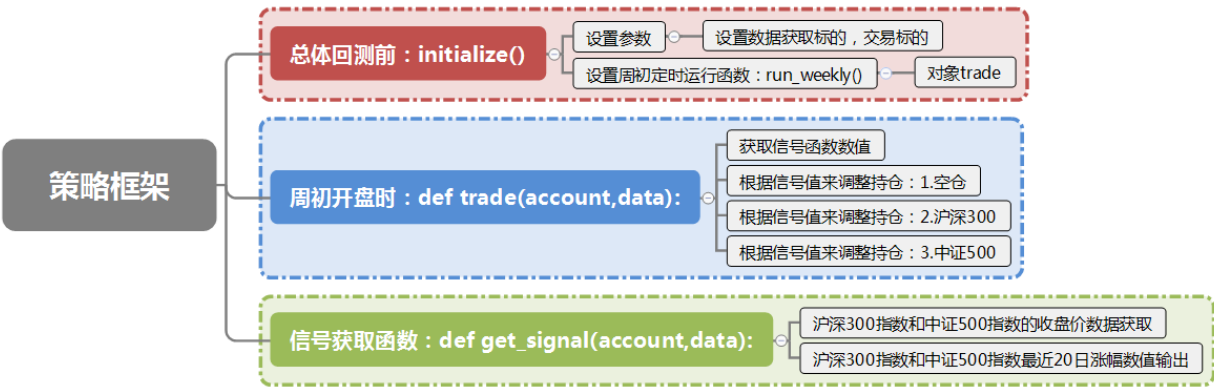
以下为策略实现的基本信息：

策略实现难度：1

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
run_weekly()	按周定时运行函数
account.positions	获取账户持仓信息
account.cash	获取账户当前资金
history()	获取多只股票多属性的历史行情数据

二、代码示意图



三、编写释义

本策略的编写难点在于根据信号值来调整持仓。以下是持仓调整逻辑梳理：

第一步：根据信号值，只可能会出现三种情况：分别是空仓、沪深 300、中证 500，其中空仓是两者信号值都小于 0，否则持仓为信号值大的那个。

第二步：以目标持仓为导向，根据当前持仓情况作出调仓操作：

目标持仓	当前持仓	操作
空仓	无持仓	无操作
	有持仓	卖出持仓
沪深 300	无持仓	买入沪深 300
	沪深 300	不操作
	中证 500	卖出中证 500，随后买入沪深 300
中证 500	无持仓	买入中证 500
	沪深 300	卖出沪深 300，买入中证 500
	中证 500	无操作

四、最终结果

策略回测区间：2014.01.01-2018.01.29

回测资金：100000

回测频率：日级

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码:

```
import pandas as pd
```

```
import numpy as np
```

```
#=====初始化账户=====
```

```
def initialize(account):
```

```
    account.day = 20 #设置数据获取长度为 20
```

```
    account.security = ['000300.SH','000905.SH']#设沪深 300 指数,中证 500 指数
```

```
    account.ETF300 = '510300.OF' #沪深 300ETF 基金
```

```
    account.ETF500 = '510500.OF' #中证 500ETF 基金
```

```
    run_weekly(trade,date_rule=1)
```

```
#=====交易函数=====
```

```
def trade(account,data):
```

```
    #获取信号值
```

```
    signal=get_signal(account,data)
```

```
    hs300=signal[0]
```

```
    zz500=signal[1]
```

```
    #根据信号值，来调整至相应仓位
```

```
    #空仓
```

```
    if hs300 <= 0 and zz500 <= 0:
```

```
        if len(account.positions) > 0:
```

```
            for stock in list(account.positions):
```

```
                order_target(stock, 0)
```

```

#沪深 300
elif hs300 > zz500:
    if account.ETF500 in list(account.positions):
        order_target(account.ETF500, 0)
    if len(account.positions) < 1:
        order_target_value(account.ETF300, account.cash)
#中证 500
elif hs300 < zz500:
    if account.ETF300 in list(account.positions):
        order_target(account.ETF300, 0)
    if len(account.positions) < 1:
        order_target_value(account.ETF500, account.cash)

#=====信号获取函数=====
def get_signal(account,data):
    #获取沪深 300 与中证 500 过去 20 日的收盘价
    close=history(account.security, ['close'], 20, '1d', False, 'pre', is_panel=1)['close']
    #计算涨跌幅
    h=(close.iloc[-1]-close.iloc[0])/close.iloc[0]
    h300=h['000300.SH']
    h500=h['000905.SH']
    return h300,h500

```

第三篇：网格交易—动态调仓策略

导语：市场上股票的价格总是在不断的上下波动，投资者通过低买高卖获利。第三篇将向大家介绍网格交易，一个动态的调仓策略，有效解决仓位和价位控制。

一、策略阐述

窥探网格交易

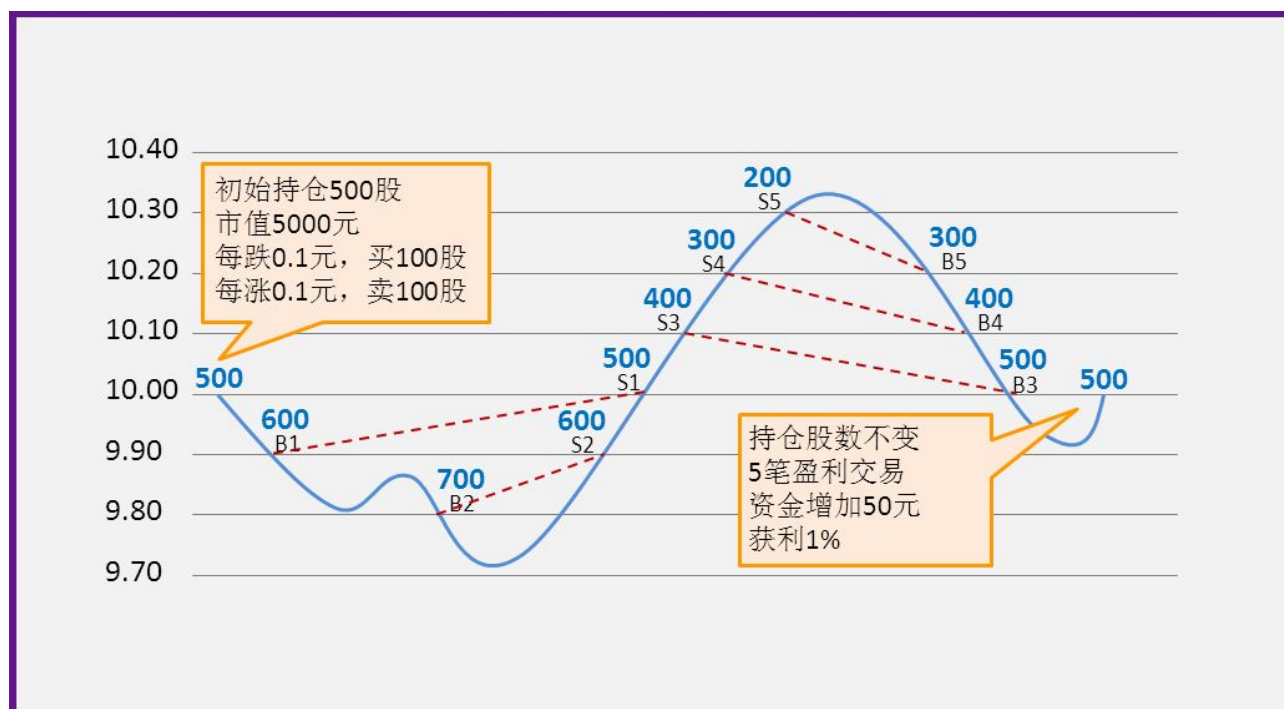
“追涨杀跌”与“高抛低吸”是两种完全不同的交易方式，也是大家听到较多的两个炒股词汇。顾名思义，“追涨杀跌”就是股价上涨买入，股价下跌卖出。“高抛低吸”就是股价下跌买入，股价上涨卖出。

但是纯粹的“高抛低吸”只是概念，根本无法形成一个交易系统。而且A股市场，只有股价上涨，投资者才能从中获利。如果股价跌跌不休，投资者每一次补仓都亏得更多。

网格交易与“高抛低吸”相似，但网格交易更为成熟，主要体现在以下两个方面：

1. 价位管理：网格交易有明确的分批买入价格和卖出价格，不受任何主观意识影响。
2. 仓位管理：网格交易有明确的买入仓位和卖出仓位，不受任何主观意识影响。

具体参考下图：



可以明确的是，上述网格交易中每笔交易都是盈利的。只要股价不断的上下波动，那么策略就可以不断的盈利。

网格交易的注意点

1. 网格交易需要规避长期下跌的个股，由于低买高卖的特征，网格交易盈利需要依赖股价波动，进行长期积累，如果个股长期下跌，那么持股市值亏损可能会吞没低买高卖的盈利。
2. 网格交易需要尽量寻找波动较高的股票，这样就可以进行更多次的低买高卖操作，提高网格交易收益率。
3. 因为网格交易没有止损，只有越跌越买，而且不盈利绝不退出。网格交易需要做好股价继续下跌的措施，保证在极限底部仍然要有现金使用。
4. 网格交易因其上涨卖出的性质，决定了其追求的是绝对收益，以及保证每笔交易是盈利的。

构建网格交易

第一步：我们选取互联网游戏行业龙头：三七互娱为投资标的，该股票业绩优异，波动率大，且股价靠近安全边际。

第二步：建底仓，仓位 40%。

第三步：设置网格，每上涨 2%，卖出 2000 股，每下跌 2%，买入 2000 股。

第四步：对三七互娱该股票进行网格交易回测，并打印日志。

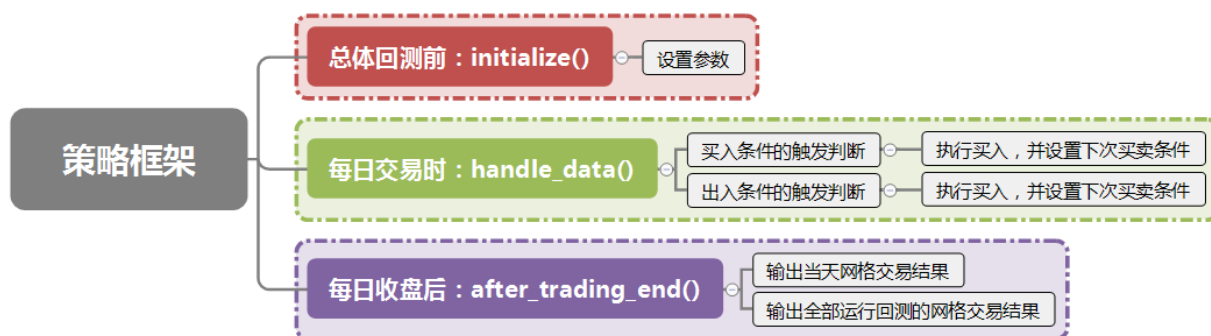
以下为策略实现的基本信息：

策略实现难度：1

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
set_benchmark	设置基准指数
g.	全局变量
account.positions	查询账户信息
get_price	获取历史行情数据
get_datetime()	获取当前时间

二、代码示意图



三、编写释义

本策略使用到了全局变量：**g**，策略编写上需与大家分享全局变量的概念，并讲述全局变量的使用

全局变量、局部变量、对象

一般量化策略同时含有全局变量、局部变量、对象，全局变量即为函数外部的变量，可以在整个程序访问。局部变量即为函数内部的变量，仅仅在某个函数内访问。对象内存储信息，比如资金账户对象 **account** 存储账户信息，**data** 对象用于获取行情数据等等。

【示例】

全局变量：**g**

局部变量：**sh**、**sz**

对象：**account**、**data**

注意：每个量化交易平台的全局变量都是系统定义的，用户无法自定义一个新的全局变量，但是局部变量可以自定义。

全局变量 **g** 和 **account** 是不同的，**account** 是账户对象。在自定义函数中，用户使用该对象的变量时，需在函数中传入 **account**，也就是 **def 函数名(account):**，但是全局变量 **g** 不同，即使函数不输入任何信息，还是可以直接使用全局变量 **g** 中的变量，可以方便策略编写。

四、最终结果

策略回测区间：2017.11.01-2018.02.11

回测资金：1000000

回测频率：分钟级

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



网格交易日志（局部）：

2017-11-03 09:31:00 - INFO 执行卖出操作，价位：23.48，数量：2000

2017-11-03 09:49:00 - INFO 执行买入操作，价位：22.9，数量：2000

2017-11-03 14:15:00 - INFO 执行买入操作，价位：22.35，数量：2000

2017-11-06 09:31:00 - INFO 执行买入操作，价位：21.73，数量：2000

2017-11-06 10:56:00 - INFO 执行卖出操作，价位：22.19，数量：2000

2017-11-07 09:31:00 - INFO 执行卖出操作，价位：22.95，数量：2000

2017-11-07 09:35:00 - INFO 执行卖出操作，价位：23.45，数量：2000

2017-11-07 09:43:00 - INFO 执行买入操作，价位：22.86，数量：2000

2017-11-07 14:51:00 - INFO 执行卖出操作，价位：23.34，数量：2000

2017-11-08 09:39:00 - INFO 执行买入操作，价位：22.82，数量：2000

2017-11-08 09:48:00 - INFO 执行卖出操作，价位：23.3，数量：2000

2017-11-08 10:07:00 - INFO 执行卖出操作，价位：23.9，数量：2000

2017-11-08 10:24:00 - INFO 执行卖出操作，价位：24.58，数量：2000

2017-11-08 11:06:00 - INFO 执行买入操作, 价位: 24.07, 数量: 2000
2017-11-08 13:05:00 - INFO 执行卖出操作, 价位: 24.59, 数量: 2000
2017-11-08 14:16:00 - INFO 执行买入操作, 价位: 24.07, 数量: 2000

2017-11-09 09:31:00 - INFO 执行买入操作, 价位: 23.42, 数量: 2000

2017-11-10 09:54:00 - INFO 执行买入操作, 价位: 22.93, 数量: 2000
2017-11-10 11:00:00 - INFO 执行买入操作, 价位: 22.45, 数量: 2000

策略源代码:

```
import numpy
import pandas as pd
#=====初始化函数=====
def initialize(account):
    g.stock = '002555.SZ'#设置网格交易的股票
    set_benchmark(g.stock)
    g.x1 = 0.02 #上涨多少卖出
    g.x2 = 0.02 #下跌多少买入
    g.y1 = 2000 #卖出多少
    g.y2 = 2000 #买入多少
    g.p = 0 #记录上一次交易的价格
    g.p1 = 0 #下次买入价
    g.p2 = 0 #下次卖出价
    g.day = 1 #记录运行天数
    g.tall = []#记录做 t 资金变动
    pass
def before_trading_start(account,data):
    if g.day == 1:
        account.jc = True
    else:
        account.jc = False
    g.t = []
#=====开盘时=====
def handle_data(account,data):
    date=get_datetime().strftime('%Y%m%d%H%M%S')
    if account.jc == True:
        order_target_percent(g.stock,0.4)
        p=history(g.stock, ['close'], 1, '1m', False, 'pre', is_panel=1)['close'].iloc[0]
        g.p=p
        g.p1 = g.p*(1-g.x1)
        g.p2 = g.p*(1+g.x2)
    else:
        p=history(g.stock, ['close'], 1, '1m', False, 'pre', is_panel=1)['close'].iloc[0]
        if p < g.p1:
            order(g.stock,g.y2)
            g.p=p
```

```

        g.p1 = g.p*(1-g.x1)
        g.p2 = g.p*(1+g.x2)
        log.info('时间: {}, 执行买入操作, 价位: {}, 数量: {}'.format(date,p,g.y2))
        g.t.append(-1*(p*g.y2))
        g.tall.append(-1*(p*g.y2))
    elif p>g.p2:
        order(g.stock,-g.y1)
        g.p=p
        g.p1 = g.p*(1-g.x1)
        g.p2 = g.p*(1+g.x2)
        log.info('时间: {}, 执行卖出操作, 价位: {}, 数量: {}'.format(date,p,g.y1))
        g.t.append(p*g.y1)
        g.tall.append(p*g.y1)
    else:
        pass
def after_trading_end(account,data):
    if g.day==1:
        log.info('首日建仓')
        pass
    else:
        log.info('当日做 T 资金变动: {}'.format(sum(g.t)))
        log.info('全部做 T 资金变动: {}'.format(sum(g.tall)))
        pass
    g.day=g.day+1

```

第四篇：进军交易系统，从 Dual Thrust 中学“趋势”

导语：“趋势”这个字眼，大家肯定不陌生，炒股票的老股民天天口里说着“趋势为王”，那么今天我们向大家介绍一个趋势跟踪交易系统：Dual Thrust，其简单明确，适用度广。

一、趋势跟踪系统阐述

Dual Thrust 的由来

Dual Thrust 是一个趋势跟踪系统，由 Michael Chalek 在 20 世纪 80 年代开发，曾被 Future Thruth 杂志评为最赚钱的策略之一。Dual Thrust 系统具有简单易用、适用度广的特点，其思路简单、参数很少，配合不同的参数、止盈止损和仓位管理，可以为投资者带来长期稳定的收益，被投资者广泛应用于股票、货币、贵金属、债券、能源及股指期货市场等。

Dual Thrust 交易系统介绍

在 Dual Thrust 交易系统中，对于震荡区间的定义非常关键，这也是该交易系统的核心和精髓。Dual Thrust 系统使用 $\text{Range} = \text{Max}(\text{HH}-\text{LC}, \text{HC}-\text{LL})$ 来描述震荡区间的大小。其中 HH 是 N 日 High 的最高价，LC 是 N 日 Close 的最低价，HC 是 N 日 Close 的最高价，LL 是 N 日 Low 的最低价。

第一步：计算 Dual Thrust 的 Range 值

- 1.N 日 High 的最高价 HH, N 日 Close 的最低价 LC
- 2.N 日 Close 的最高价 HC, N 日 Low 的最低价 LL
3. $\text{Range} = \text{Max}(\text{HH}-\text{LC}, \text{HC}-\text{LL})$

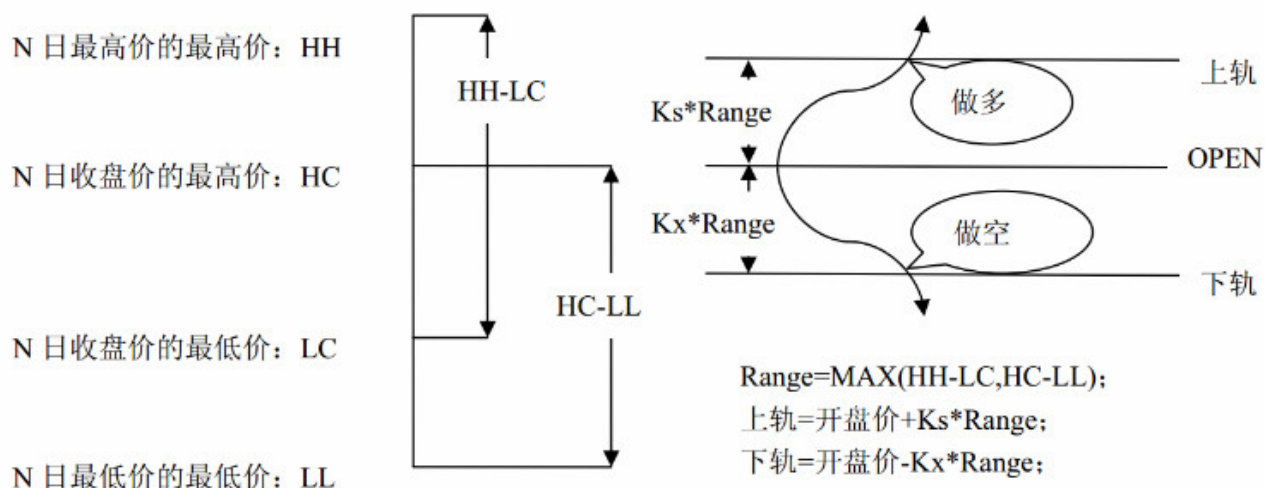
第二步：计算 Dual Thrust 上下轨

1. $\text{BuyLine} = \text{Open} + K1 * \text{Range}$
2. $\text{SellLine} = \text{Open} - K2 * \text{Range}$

第三步：交易系统（考虑到 A 股市场无法做空）

- (1)当盘中价格向上突破上轨时，做多。
- (2)当盘中价格向下突破下轨时，清仓。

参考下图：



本篇内容以沪深 300 指数为例，来向大家展示 Dual Thrust 交易系统。以下是两个注意点：

1. 计算 Dual Thrust 的 Range 值时，N 取 1，即通过昨天的开高低收数据来计算出 Range 值。

2. K_1 与 K_2 保持不对称性，介于 A 股市场大牛大熊轮回切换，且指数长期向上，因此取 $K_1=0.8$ ， $K_2=2$ 。

以下为策略实现的基本信息：

策略实现难度：2

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
before_trading_start()	定时运行函数，每个交易日 9.00 运行
handle_data()	定时运行函数，按交易日或按分钟运行
set_benchmark()	设置基准指数
get_price	获取历史行情数据

二、代码示意图



三、编写释义

注意本策略需分钟运行，考虑到分钟运行时运行速度较为关键，因此一些必要的数据库：例如策略中的 Range 值，应该是盘前计算完毕，盘中直接使用，开盘价也只需要开盘第一分钟获取即可。如果盘中每分钟运行需要消耗较多的时间，则该策略实际运行时可能会错失下单价位。

四、最终结果

策略回测区间：2014.01.01-2018.01.31

回测资金：10000000

回测频率：分钟

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码:

```
import pandas as pd
import numpy as np
#=====初始化函数=====
def initialize(account):
    g.stock = '000300.SH' #设置交易标的为沪深 300 指数
    g.openprice = 0 #记录每天开盘价
    g.Range = 0 #记录每天的 Range 值
    #确定当天的买入价和卖出价
    g.buyprice = 0
    g.sellprice = 0
    #上突破系数
    g.k1 = 0.8
    #下突破系数
    g.k2 = 2
    pass
#=====盘前运行=====
def before_trading_start(account, data):
    date = get_last_datetime().strftime('%Y%m%d')
    #获取 Range 值
    g.Range=dual_thust(g.stock,date)
#=====盘中运行=====
def handle_data(account, data):
    date=get_datetime().strftime('%H%M%S')
    if date == '093100':
        g.openprice=history(g.stock, ['open'], 1, '1m', False, 'pre', is_panel=1)['open'].iloc[0]
        g.buyprice=g.openprice+g.Range*g.k1
        g.sellprice=g.openprice-g.Range*g.k2
    if len(list(account.positions))==0:
        close=history(g.stock, ['close'], 1, '1m', False, 'pre', is_panel=1)['close'].iloc[0]
        if close>g.buyprice:
            order_target_percent(g.stock,1)
    if len(list(account.positions))>0:
        close=history(g.stock, ['close'], 1, '1m', False, 'pre', is_panel=1)['close'].iloc[0]
        if close<g.sellprice:
            order_target(g.stock,0)
    pass
#=====Range 值计算=====
def dual_thust(stock,date):
    price = get_price(stock,None ,date , '1d', ['close','open','low','high'], False,'pre', 1 ,is_panel=0)
    hc = price['high'].iloc[-1]-price['close'].iloc[-1]
    cl = price['close'].iloc[-1]-price['low'].iloc[-1]
    Range = max(hc,cl)
    return Range
```

第五篇：布林强盗，一个霸道的交易系统

导语：作为策略锦集第五篇，再向大家介绍一个霸道的交易系统——布林强盗交易系统。

一、布林强盗交易系统阐述

布林强盗交易系统的由来

布林线（BOLL）是非常著名的一个技术分析指标，其由 John Bollinger 在 20 世纪 60 年代创建的。布林线由上轨、中轨、下轨三条线组成，期初，人们将上轨当做压力位使用，下轨当做支撑位，但经过测试发现，BOLL 技术指标的用法并非如此简单。

布林强盗交易系统其实就是布林线一种使用方法，凭借其在期货交易上的盈利能力，人们称其为“强盗”，可以说是非常高的评价了。

布林强盗交易系统介绍

布林强盗交易系统只有两个组成部分，分别为：进场、离场。

A.进场：

- 1.当日收盘价超过 50 日移动平均线的上轨。注意：上轨值=过去 50 日收盘价平均值+过去 50 日收盘价的标准差
- 2.当日收盘价超过过去 30 个交易日收盘价的最大值。
- 3.同时满足上述两个条件，则下个交易日开盘全仓买入。

B.离场：

- 1.**自适应均值：**进场后，设置自适应移动均值，首日自适应移动均值为过去 50 日收盘价均值，即为中轨。之后持仓时间每多一个交易日，计算移动平值的天数减去 1。但移动均值的计算天数最小递减到 10。如果达到了 10，则不再递减。
- 2.当日收盘价跌破自适应移动均值。
- 3.自适应移动均值小于上轨。此处的上轨依旧是 50 日移动平均线的上轨，注意：上轨值=过去 50 日收盘价平均值+过去 50 日收盘价的标准差
- 4.同时满足上述两个条件，则下个交易日开盘清仓。

本篇内容以沪深 300 指数 ETF 为例，来向大家展示布林强盗交易系统

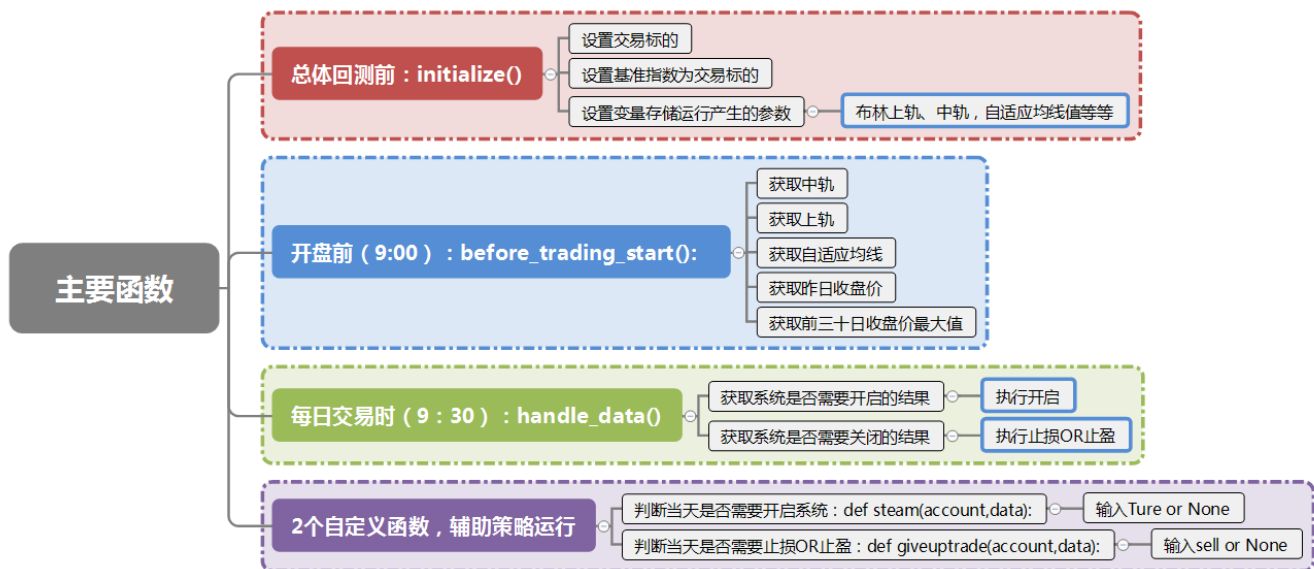
以下为策略实现的基本信息：

策略实现难度：2

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
before_trading_start()	定时运行函数，每个交易日 9.00 运行
history	获取多只股票多属性的历史行情数据

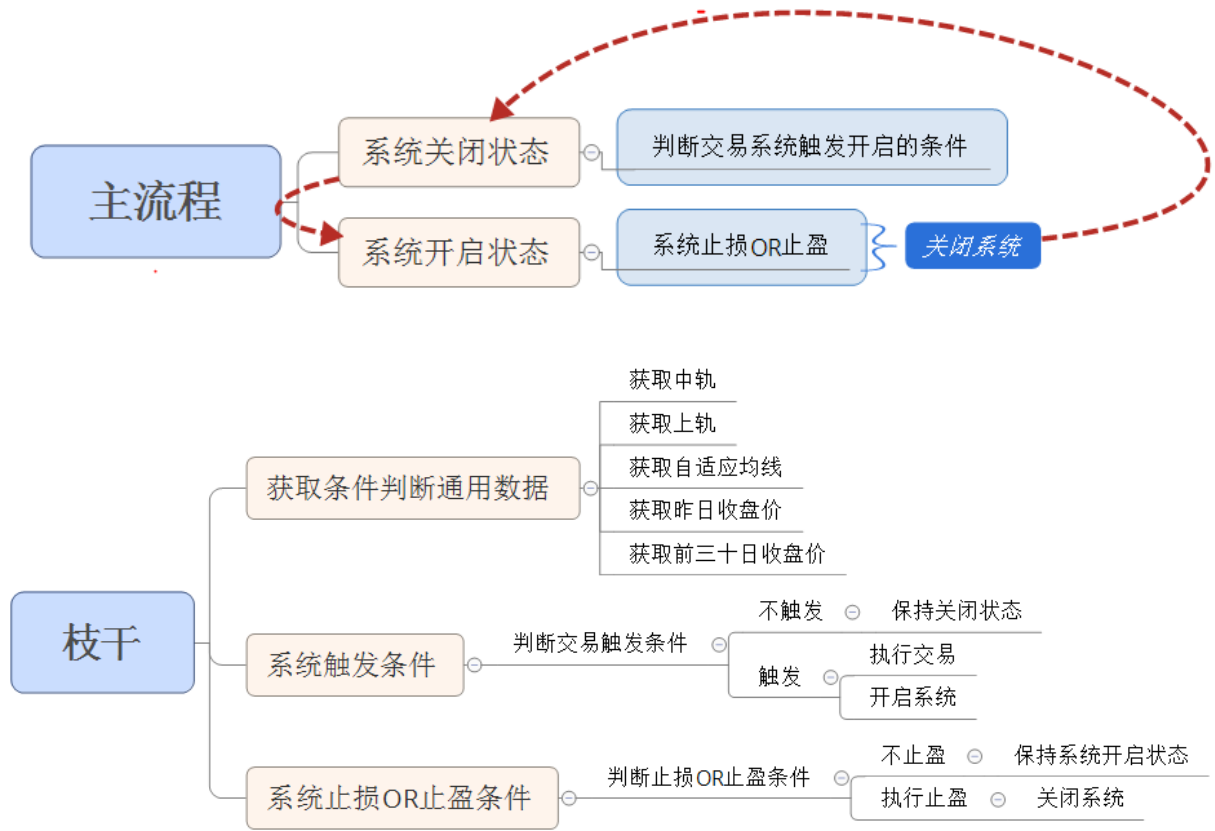
二、代码示意图



三、编写释义

本策略的编写难点在于理解布林强盗交易系统的运行逻辑，以下是布林强盗交易系统运行逻辑的梳理：

编写布林强盗交易系统的时候，依旧建议采用主干+枝干的思路。



四、最终结果

策略回测区间：2014.01.01-2018.01.30

回测资金：100000

回测频率：日级

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码:

```
import pandas as pd
import numpy as np
#=====初始化账户=====
def initialize(account):
    account.security = '159919.OF' #沪深 300 指数 ETF 为例
    set_benchmark(account.security)
    account.steam = False
    account.mid50 = 0 #储存中轨  每日更新以下 5 个数据
    account.up50 = 0 #储存上轨
    account.close = 0 #储存上个交易日的收盘价
    account.closemax = 0 #储存过去 30 个交易日的收盘价最大值
    account.day = 50 #自适应均线计算, 没持仓一天, 减去 1
    account.maday = 0 #储存自适应均线值
#=====盘前运行=====
def before_trading_start(account,data):
    #开盘前获取中轨, 上轨, 自适应均线, 昨日收盘价, 前三十日收盘价, 放置账户信息中,
    以便调用
    close = history(account.security, ['close'], 50, '1d', False, 'pre', is_panel=1)['close']
    account.maday = np.mean(close[-account.day:])
    account.mid50 = np.mean(close) #50 日均线, 中轨
    account.up50 = account.mid50 + np.std(close) #上轨=中轨+1*标准差
    account.close = close[-1]
    account.closemax = max(close[-31:-1])
#设置买卖条件, 每个交易频率(日/分钟/tick)调用一次
def handle_data(account,data):
    #判断系统开启条件
    if account.steam == False:
        account.steam = steam(account,data)
        if account.steam == True:
            #满足系统开启条件买入
            order_target_percent(account.security,1)
        else:
            pass
    #判断系统是否需要止损或者止盈, 以关闭系统
    elif account.steam == True:
        trade=giveuptrade(account,data)
        if trade=='sell':
            #需要止损, 并关闭系统,自适应天数回复
            order_target(account.security,0)
            account.steam = False
            account.day = 50
        else:
            #不需要止损, 但是需要将自适应天数减去 1
            account.day=account.day-1
            if account.day<10:
                account.day = 10
            pass
```

```

#=====离场条件判断，系统是否需要关闭=====
def giveuptrade(account,data):#day 参数主要是计算自适应均线
    if account.close < account.maday:
        if account.maday < account.up50:
            return 'sell'
        else:
            return None
    else:
        return None
#=====入场条件判断，系统是否需要开启=====
def steam(account,data):
    if account.close > account.up50:
        if account.close > account.closemax:
            return True
        else:
            return False
    else:
        return False

```

第六篇：交易系统终结者—海龟交易法则

导语：作为策略锦集第六篇，再向大家介绍非常著名的交易系统—海龟交易法则。

一、策略阐述

1.海龟交易法则的由来

Riachard Dennis 是七八十年代著名的期货投机商，是一位具有传奇色彩的人物，在多年的投机生涯中，Dennis 出尽风头，给人的感觉是常常可以在最低点买进，然后在最高峰反手卖空。

他相信优秀的交易员是后天培养而非天生的。在 1983 年 12 月，他招聘了 23 名新人，昵称为海龟，并对这些交易员进行了一个趋势跟踪交易策略培训。随后给予每个新人 100 万美元的初始资金。经 5 年的运作，大部分“海龟”的业绩非常惊人，其中最好的业绩达到 1.72 亿美元。多年后，海龟交易法则公布于世，我们才有幸看到曾名噪一时的完整的海龟交易法则。

2.海龟交易法则介绍

A.市场与标的

海龟交易法则应用于流动性高的市场中，选择交易量较大的标的进行交易。本文以沪深 300 指数 ETF 为标的，构建海龟交易法则。

B、仓位：仓位是海龟交易系统最核心的部分，通过 ATR 真实波幅指标来管理仓位。

第一步：计算 True Range，简称 TR。

$TR = \text{Max} (H-L, H-P, P-L)$ ，其中 H 为当日日内最高价，L 为当日日内最低价，P 为前一日收盘价。

第二步：计算 ATR

$ATR = \text{mean} (TR, 20)$ ，即计算过去 20 天的 TR 的平均值，ATR 是 TR 的移动平均值

第三步：计算 unit（法则的交易单位）

$\text{Unit} = (\text{value} * 1\%) / ATR$ ，其中 value*1%即为总资产的 1%，考虑到国内最小变化量是 0.01 元，1 手是 100 股，所以 1ATR 即为持 1 股股票的资产最大变动，那么买入 1Unit 单位的股票，使得总资产当天震幅不超过 1%。

C.开仓入市

海龟交易法则分两个交易系统，两者都为分钟回测，即盘中交易：

系统一：

- I、若当前价格高于过去 20 日的最高价，则买入一个 Unit
- II、加仓：若股价在上一次买入的基础上上涨了 $0.5ATR$ ，则加仓一个 Unit。

系统二：

- I、若当前价格高于过去 55 日的最高价，则买入一个 Unit
- II、加仓：若股价在上一次买入的基础上上涨了 $0.5N$ ，则加仓一个 Unit。

举例：若某只股票 A 的 ATR 为 1，20 日最高价为 40。
则当股价突破 40 时买入一个 Unit，当股价突破 $40+0.5\times1=40.5$ 时加仓一个 Unit。
当股价突破 $40.5+0.5\times1=41$ 时加仓一个 Unit。

D.止损：海龟交易法则规定，当价格比最后一次买入价格下跌 $2ATR$ 时，则卖出全部头寸止损。

E.止盈：两个系统分别采用不同参数来止盈。

系统一：当股价跌破 10 日内最低价格时，清仓结束交易

系统二、当股价跌破 20 日内最低价格时，清仓结束交易

考虑到系统一与系统二的差异性集中在参数上，本篇内容实现系统二，来向大家展示海龟交易法则。

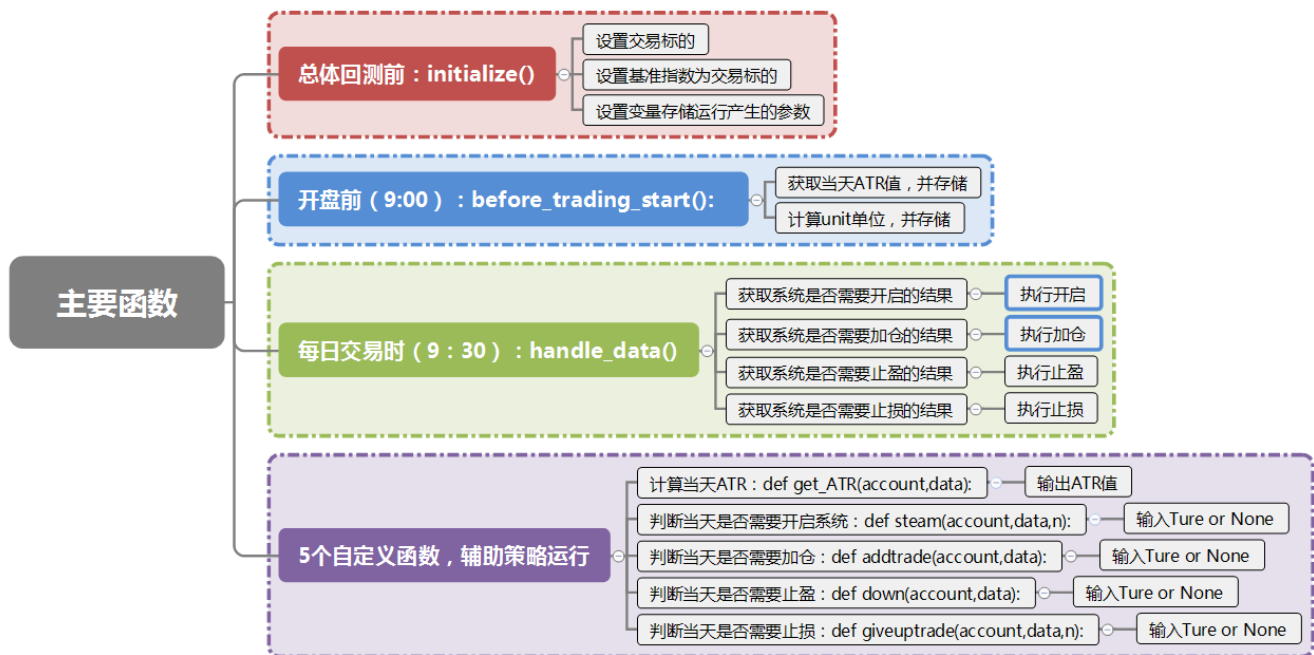
以下为策略实现的基本信息：

策略实现难度：2

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

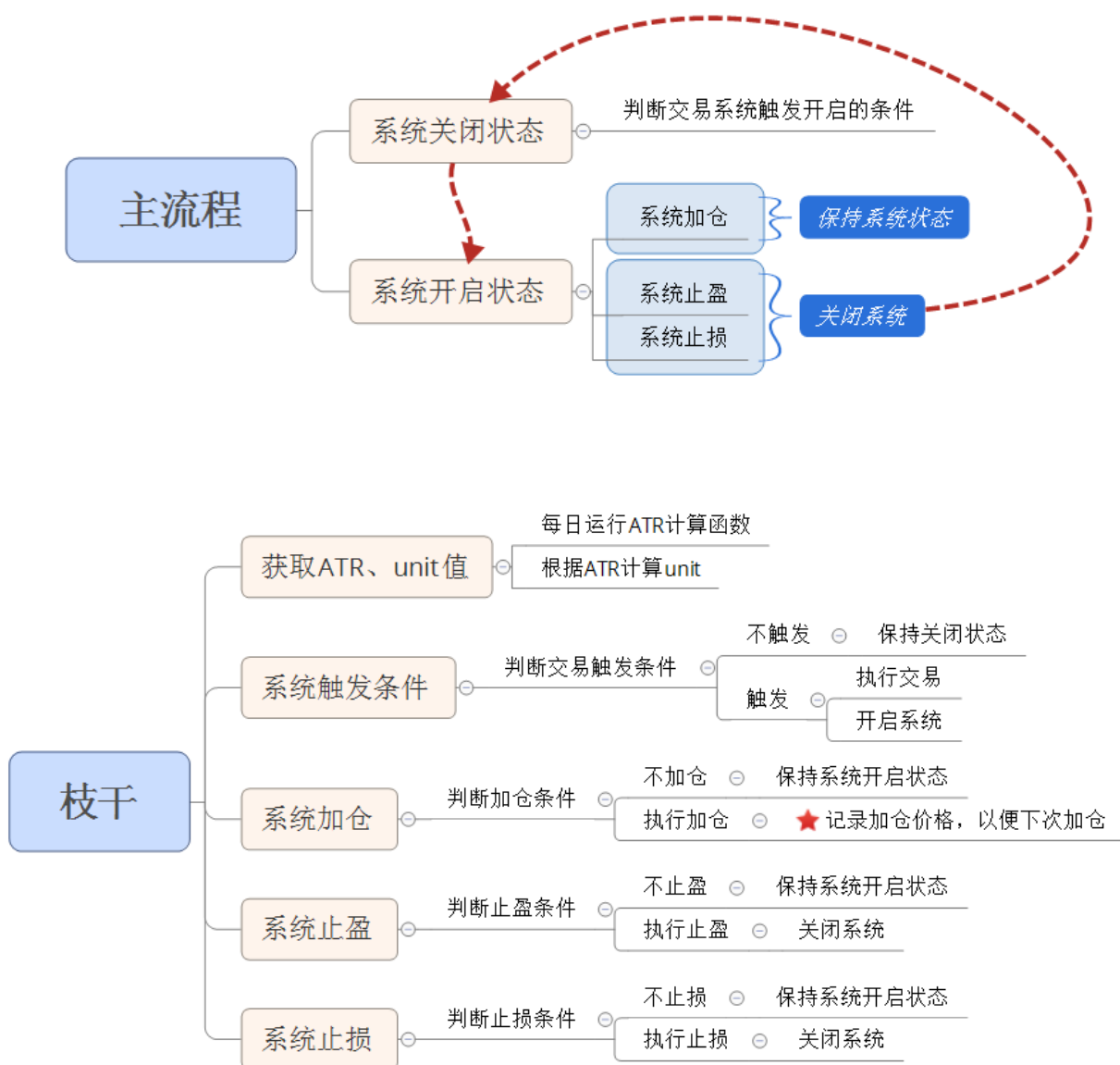
需要用到的 API 函数	功能
account.portfolio_value	获取账户总资产
set_benchmark()	设置基准指数

二、代码示意图



三、编写释义

本策略的编写难点在于理解海龟交易法则的运行逻辑，以下是海龟法则运行逻辑的梳理：
编写海龟交易法则的时候，建议采用主干+枝干的思路。



四、最终结果

策略回测区间：2014.01.01-2018.01.29

回测资金：1000000

回测频率：分钟

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码:

```
import pandas as pd
import numpy as np
#=====初始化账户=====
==
def initialize(account):
    account.security = '159919.OF'#确定交易标的
    set_benchmark(account.security)
    account.ART = 0#储存 ATR 的值，每个交易日更新一次
    account.unit = 0#买卖单位的储存变量
    account.steam = False#交易系统
    account.price = 0 #记录系统的买入价，以便加仓和离市
#=====盘前运行=====
def before_trading_start(account,data):
    #更新 n 值
    ATR=get_ATR(account,data)
    #获取账户总资产
    value=account.portfolio_value
    # 依本策略，计算一个单位的 unit 为多少股,以便后续下单交易,注意一共两个系统，因此
    需要除以 2
    account.unit = (value*0.01)/account.ATR
    if account.unit<100:
        log.info('一个 unit 单位的股数不满 1 手，无法下单！')

#=====设置买卖条件，分钟回测=====
def handle_data(account,data):
    #=====系统 1=====
    #系统是否需要开启运作
    if account.steam == False:
        #获取开启系统的结果
        account.steam = steam(account,data,55)
        if account.steam == False:
            pass
        else:
            order(account.security,account.unit)
            #买入后记录当前价位，以便加仓和离市
            log.info('系统开启')
            nowclose = history(account.security, ['close'], 1, '1m', False, 'pre', is_panel=1)['close']
            account.price = nowclose[0]
    #系统已经开启运作
    if account.steam == True:
        #获取进行加仓结果
        signal=addtrade(account,data)
        #执行结果加仓
        if signal=='buy':
            log.info('系统加仓')
            nowclose = history(account.security, ['close'], 1, '1m', False, 'pre', is_panel=1)['close']
            order(account.security,account.unit)
```

```

        #买入后记录当前价位，以便加仓和离市
        account.price1 = nowclose[0]
    else:
        pass
    #获取止盈结果
    signal=down(account,data)
    #执行止盈，关闭系统
    if signal=='sell':
        log.info('系统 1 止盈')
        order_target(account.security,0)
        #止盈后情况价位记录
        account.price = 0
        #关闭系统
        account.steam = False
    #离场结果判断
    if account.steam==True:
        #获取离场结果
        signal=giveuptrade(account,data,20)
        #执行离场，关闭系统
        if signal=='sell':
            log.info('系统 1 离场')
            order_target(account.security,0)
            #离场后情况价位记录
            account.price=0
            #关闭系统
            account.steam=False
#=====判断是否离场函数=====
def giveuptrade(account,data,n):
    #根据系统来获取相应数据长度
    close = history(account.security, ['low'], n, '1d', False, 'pre', is_panel=1)['low']
    close_min=min(close)
    nowclose = history(account.security, ['close'], 1, '1m', False, 'pre', is_panel=1)['close']
    #最新价格突破过去 N 日最大收盘价，即为突破，开启系统
    if nowclose[0]<close_min:
        return 'sell'
    else:
        return None
#=====判断是否止盈函数=====
def down(account,data):
    nowclose = history(account.security, ['close'], 1, '1m', False, 'pre', is_panel=1)['close']
    n=account.ATR
    TP=account.price-2*n
    if nowclose[0]<TP:
        return 'sell'
    else:
        return None
#=====判断是否加仓函数=====
def addtrade(account,data):
    nowclose = history(account.security, ['close'], 1, '1m', False, 'pre', is_panel=1)['close']

```

```

n=account.ATR
TP=account.price+n/2
if nowclose[0]>TP:
    return 'buy'
else:
    return None

```

=====判断系统开启函数=====

```

def steam(account,data,n):
    close = history(account.security, ['close'], n, '1d', False, 'pre', is_panel=1)['close']
    close_max=max(close)
    nowclose = history(account.security, ['close'], 1, '1m', False, 'pre', is_panel=1)['close']
    #最新价格突破过去 N 日最大收盘价，即为突破，开启系统
    if nowclose[0]>close_max:
        return True
    else:
        return False

```

=====计算 n 值的函数=====

```

def get_ATR(account,data):
    #由于用到前 20 个交易日的 n 值，ATR 计为过去 20 日的 TR 均值
    price = history(account.security, ['close','high','low'], 21, '1d', False, 'pre', is_panel=1)
    h = price['high'].iloc[1:] #最高价，获取 21 个需弃掉第一个
    l = price['low'].iloc[1:] #最低价，获取 21 个需弃掉第一个
    rc = price['close'].shift().iloc[1:] #昨日收盘价，获取 21 个需弃掉第一个
    #shift()操作专门是用于获取前收盘价数据的
    tr_list = []
    for i in range(0,20,1):
        h = price['high'].iloc[i]
        l = price['low'].iloc[i]
        rc = price['close'].iloc[i]
        TR = max(h-l, h-rc, rc-l)
        tr_list.append(TR)
    ATR=np.mean(tr_list)
    account.ATR=ATR
    return account.ATR

```

第七篇：向彼得林奇投资大师学习 PEG 选股

导语：投资大师彼得·林奇有过一个著名的论断：任何一家公司股票如果定价合理的话，市盈率就会与收益增长率相等。本篇内容将为大家讲述彼得·林奇推广的 PEG 选股策略。

一、策略阐述

彼得·林奇

彼得·林奇（Peter Lynch）生于 1944 年 1 月 19 日，是一位卓越的股票投资家和证券投资基金经理，曾被《时代杂志》评为首席基金经理。1977 年至 1990 年，在彼得·林奇管理麦哲伦基金的 13 年间，基金规模大幅扩增，由 2000 万美元成长至 140 亿美元！他对共同基金的贡献，就像是乔丹之于篮球，邓肯之于现代舞蹈。他不是人们日常认识中的那种脑满肥肠的商人，他把整个比赛提升到一个新的境界，他让投资变成了一种艺术。

彼得·林奇一直以他的选股能力而著称，他有一句名言：只要用心对股票做一点点研究，普通投资者也能成为股票投资专家，并且在选股方面的成绩能像华尔街的专家一样出色。

PEG 指标

围绕着彼得·林奇的著名论断，我们来谈谈 PEG 指标。

论断：任何一家公司股票如果定价合理的话，市盈率就会与收益增长率相等。

了解基础指标：每股收益、市盈率和每股收益增长率。

每股收益(EPS)：是税后利润与股本总数的比率。传统的每股收益指标计算公式为：每股收益 = 期末净利润 ÷ 期末股份总数。

每股收益增长率(G)：论断中的收益增长率指的就是每股收益的增长率，同比增长率 = $\{(\text{本期每股收益} - \text{上年同期每股收益}) / \text{上年同期每股收益的绝对值}\} * 100\%$ 。

市盈率(PE)：是当前股价与每股收益的比值。市盈率对个股、类股及大盘都是很重要参考指标。任何股票若市盈率大大超出同类股票或是大盘，都需要有充分的理由支持，而这往往离不开该公司未来盈利率将快速增长这一重点。一家公司享有非常高的市盈率，说明投资人普遍相信该公司未来每股盈余将快速成长，以至数年后市盈率可降至合理水平。一旦盈利增长不如理想，支撑高市盈率的力量无以为继，股价往往会大幅回落。

PEG 指标由上述基础指标演变而来，计算公式为： $PEG = PE / G$ ，其中 PE 为市盈率，G 为每股收益增长率。PE 代表投资者对公司每股收益增长预期，而 G 代表公司真实的每股收益增长预期。从合理定价的角度看，个股的市盈率与每股收益增长率应该保持相等。如果 PEG 小于 1，股价是偏低的，反之则偏高。

彼得·林奇 PEG 选股

PEG 选股策略：计算全市场股票的 PEG 指标，按 PEG 值从小到大排序，选择前 10 只股票作为持仓

就这么简单吗？不是的，PEG 指标选股具有局限性：

- 1.PEG 指标不适用于周期性行业选股，因此需要剔除周期性行业内的股票。由于周期性行业非常多，本文选取 5 个非周期性行业进行 PEG 选股：食品饮料，医药生物，纺织服装，商业贸易，交通运输 。
 - 2.价值型股票与成长性股票不同，价值型股票由于其成长空间有限，导致价值型股票的 PEG 往往会低于 1，以反映低业绩增长的预期。所有直接使用 PEG 指标会出现一边倒现象。更何况 PEG 选股策略主要意图在于挑选出优质成长股。对于个股是否为成长型股票的定义方法并不固定，因此本文采用以下简单粗暴定义(ps:同学们掌握后可以在此处优化哦!)
 - 3.无法衡量公司的收益增率可持续性，考虑到公司可能出现的扭亏为盈、特殊情况导致单季度收益增率高达 1000%+现象，我们需要剔除收益增长率超过 40%的个股，因其增长不可持续。
- 成长型股票池：对所有非周期性行业个股，采用净利润复合增长率、销售净利率两个指标，选取当季两个指标都在前 50%的股票。

第一步：获取成长型股票池。

第一步：每隔 20 个交易日，获取股票池内个股的 PE 和 G 值，剔除 G 大于 40 的个股，然后计算成长型股票池的 PEG 指标。

第二步：按 PEG 值从小到大排序，选择前 5 只股票作为持仓

以下为策略实现的基本信息：

策略实现难度：3

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
get_industry_stocks	获取行业指数成分股
get_fundamentals	获取财务数据

二、代码示意图



三、编写释义

彼得·林奇 PEG 选股核心是获取成长性股票，本篇内容中粗糙定义了成长型股票，建议初学者前往研究环境操作，并能在此基础上加入自己的一些想法。

以下是作者实现过程中，研究环境的代码草稿，分享给同学们。

```
#=====获取非周期性行业个股
def non_cyclical_industry(date):
    #确定申万一级非周期行业
    non_cyclical_industry_list=['S36',#申万一级： 纺织服装
                                'S45',#申万一级： 商业贸易
                                'S37',#申万一级： 医药生物
                                'S34',#申万一级： 食品饮料
                                'S42']#申万一级： 交通运输

    stock_list=[]
    #通过行业成分股获取函数，获取到股票池。
    for i in non_cyclical_industry_list:
        stk=get_industry_stocks(i,date)
        for i in stk:
            stock_list.append(i)
    return stock_list
date='20180125'
stk=non_cyclical_industry(date)
stock_num = int(len(stk)*0.5)
#获取净利润增长率和销售毛利率
q =
query(valuation.symbol,growth.net_profit_compound_growth_ratio,profit.net_profit_margin_on_sales).filter(valuation.symbol.in_(stk))
basic = get_fundamentals(q, date = date)
for i in ['growth_net_profit_compound_growth_ratio','profit_net_profit_margin_on_sales']:
    basic = pd.DataFrame(basic).sort_values(by=i, ascending=False)
    stockmax=list(basic.iloc[:stock_num]['valuation_symbol'])
    stk=list(set(stk)&set(stockmax))
q =
query(valuation.symbol,valuation.pe_ttm,growth.basic_eps_year_growth_ratio).filter(valuation.symbol.in_(stk))
basic = get_fundamentals(q, date = '20180125')
basic['PEG']=basic['valuation_pe_ttm']/basic['growth_basic_eps_year_growth_ratio']
basic = basic[basic['PEG']>0]
basic = basic[basic['growth_basic_eps_year_growth_ratio']<50]
basic = pd.DataFrame(basic).sort_values(by='PEG', ascending=True)
basic.index=basic['valuation_symbol']
del basic['valuation_symbol']
basic
```

运行结果:

	valuation_pe_ttm	growth_basic_eps_year_growth_ratio	PEG
valuation_symbol			
000429.SZ	11.94	41.86	0.285237
000915.SZ	24.77	35.29	0.701899
300146.SZ	31.99	41.03	0.779673
300298.SZ	41.61	49.62	0.838573
002770.SZ	40.76	45.75	0.890929
000828.SZ	12.98	14.10	0.920567
000596.SZ	35.49	36.21	0.980116
300357.SZ	45.14	44.75	1.008715
000858.SZ	37.75	36.53	1.033397
002262.SZ	37.70	35.57	1.059882
300255.SZ	29.85	27.27	1.094609
300233.SZ	28.65	25.64	1.117395
002615.SZ	26.61	23.81	1.117598
002370.SZ	39.41	33.33	1.182418
002572.SZ	42.93	36.14	1.187880
002320.SZ	55.01	44.83	1.227080
002020.SZ	30.08	24.33	1.236334
000089.SZ	28.73	22.85	1.257330

四、最终结果

策略回测区间：2014.01.01-2018.01.31

回测资金：1000000

回测频率：日级

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码：

```
import pandas as pd
import numpy as np
#=====初始化函数=====
def initialize(account):
    set_benchmark('399333.SZ')#设置中小板为基准
    g.day = 0 #记录运行天数
    g.tradeday = 20 #调仓频率
    g.stock = [] #储存上期的股票池
    g.trade = False #是否调仓的开关
    g.stocknum = 10 #持仓数量
    pass
#=====盘前运行=====
def before_trading_start(account,data):
    #判断是否调仓
    if g.day%g.tradeday==0:
        g.trade=True
    else:
        g.trade=False
    g.day=g.day+1
```

#=====盘中交易=====

```
def handle_data(account, data):
    if g.trade==True:
        #获取选股结果
        needstock_list = peg()
        #获取上期持仓个股
        holdstock_list = list(g.stock)
        #确定本期需要卖出的个股
        sell_list = list(set(holdstock_list)-set(needstock_list))
        #执行卖出操作，运用 for 循环，逐个操作。
        for s in sell_list:
            order_target(s,0)
        #确定本期需要买入的个股，其余即为继续持仓的个股
        buy_list=[]
        for i in needstock_list:
            if i in holdstock_list:
                pass
            else:
                buy_list.append(i)
        #确定可用资金，平分分配至需买入的个股
        n=len(buy_list)
        cash=account.available_cash/n
        #执行买入操作
        for s in range(0,n,1):
            stock=list(buy_list)[s]
            order_value(stock,cash)
        #操作完毕，将选股结果放到上期股票池储存变量中，以备下次使用。
        g.stock = frozenset(needstock_list)
    else:
        pass
```

#=====选股函数=====

```
def peg():
    date=get_last_datetime().strftime('%Y%m%d')
    #获取非周期行业股票。
    stock_list1=non_cyclical_industry(date)
    #获取成长型股票池
    stock_list2=growth_list(stock_list1,date)
    #获取 PEG 最小的 10 只股票
    q =
query(valuation.symbol,valuation.pe_ttm,growth.basic_eps_year_growth_ratio).filter(valuation.symbol.in_(stock_list2))
    basic = get_fundamentals(q, date = date)
    basic['PEG']=basic['valuation_pe_ttm']/basic['growth_basic_eps_year_growth_ratio']
    basic = basic[basic['PEG']>0]
    basic = basic[basic['growth_basic_eps_year_growth_ratio']<=40]
    basic = pd.DataFrame(basic).sort_values(by = 'PEG', ascending=True)
    basic.index=basic['valuation_symbol']
```

```

#获取股票的代码
needstock_list=[]
for s in range(0,g.stocknum,1):
    needstock_list.append(list(basic.index)[s])
return needstock_list

#=====获取成长型股票池=====
def growth_list(stk,date):
    stock_num = int(len(stk)*0.5)
    #获取净利润增长率和销售毛利率
    q = query(valuation.symbol,
              growth.net_profit_growth_ratio,
              profit.net_profit_margin_on_sales).filter(valuation.symbol.in_(stk))
    basic = get_fundamentals(q, date = date)
    for i in ['growth_net_profit_growth_ratio','profit_net_profit_margin_on_sales']:
        basic = pd.DataFrame(basic).sort_values(by =i, ascending=False)
        stockmax=list(basic.iloc[:stock_num]['valuation_symbol'])
        stk=list(set(stk)&set(stockmax))
    return stk

#=====获取非周期性行业个股
def non_cyclical_industry(date):
    #确定申万一级非周期行业
    non_cyclical_industry_list =['S36',#申万一级： 纺织服装
                                  'S45',#申万一级： 商业贸易
                                  'S37',#申万一级： 医药生物
                                  'S34',#申万一级： 食品饮料
                                  'S42']#申万一级： 交通运输

    stock_list=[]
    #通过行业成分股获取函数，获取到股票池。
    for i in non_cyclical_industry_list:
        stk=get_industry_stocks(i,date)
        for i in stk:
            stock_list.append(i)
    return stock_list

```

第八篇：CAPM 模型的应用

导语：提起资本资产定价模型，大家可能都有学习过，那么本篇内容主要向大家讲述资本资产定价模型的实际应用，我们将其构建为量化策略，并在历史行情中回测。

一、策略阐述

CAPM 模型公式

夏普发现单个股票或者股票组合的预期回报率的公式如下： $E[r_i] = r_f + \beta(E[r_M] - r_f)$ 。

其中， r_f 是无风险回报率， β 是证券的 Beta 系数， $E[r_M]$ 是市场期望回报率， $E[r_M] - r_f$ 是市场溢价。

超额收益

当一只股票的价格被正确定价的时候，它的风险溢价与市场组合的风险溢价是呈现完全的线性关系，也就是线性回归后应该得到一条斜率为 β ，截距为 r_f 的拟合直线。

实际上，每只股票的收益率与市场组合的收益率进行回归的时候截距都不会等于 r_f ，而这个偏差值 α 的计算公式为： $\alpha = (E[r_i] - r_f) - \beta(E[r_M] - r_f)$

按照 CAPM 模型，当正确定价时，股票的 α 值应该为零，所有 α 值不为零的股票我们都可以认为被错误的定价了。被错误定价的股票可以分为两类

1. $\alpha < 0$ ，此时股票的价格被低估了

2. $\alpha > 0$ ，此时股票的价格被高估了

假设 CAPM 理论是成立的，这些错误定价的股票都会最终回到得到正确的定价，那么选股时应该买入 α 较低的股票。

CAPM 模型实际应用

第一步：我们选取创业板指数成分股为股票池，由于创业板为风险较大板块，因此错误定价的发生概率和偏离度会更大。

第二步：获取股票池内个股与创业板指数的过去 60 个交易日的涨跌幅，进行线性回归，计算出每一个股票的 α 值。

第三步：按 α 从小到大排序，选取前 10 只股票作为持仓，持股一个月。

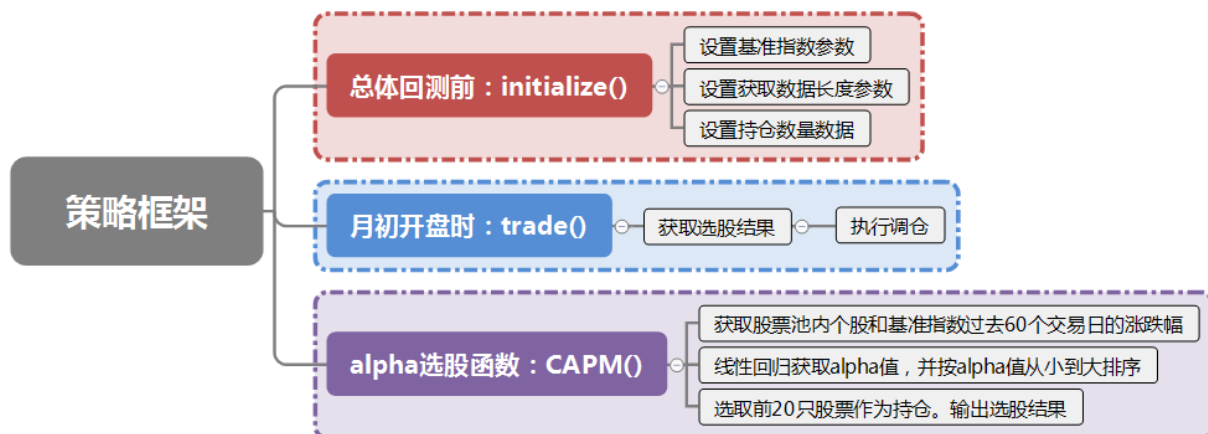
以下为策略实现的基本信息：

策略实现难度：3

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
handle_data()	定时运行函数，按交易日或按分钟运行
set_benchmark()	设置基准指数

二、代码示意图



三、编写释义

本篇内容的关键点是将个股的 α 通过线性回归计算获取，以下是作者实现过程中的笔记，建议初学者到研究环境中使用，尤其是线性回归模板。

```
from scipy import stats
#=====以创业板为股票池，除去 ST 和停牌=====
def stock(date):
    stk=list(get_index_stocks('399006.SZ',date))
    price=get_price(stk, None, date, '1d', ['is_paused', 'is_st'], False, None, 1, is_panel=1)
    stopstk=price['is_paused'].iloc[-1]
    ststk=price['is_st'].iloc[-1]
    startstk=(stopstk[stopstk==0].index)
    okstk=(ststk[ststk==0].index)
    tradestk=list(set(startstk)&set(okstk))
    return tradestk
#=====选股函数=====
date='20180125'
stock_list=stock(date)
ret=get_price(stock_list, None, date, '1d', ['quote_rate'], False, 'pre', 60, is_panel=1)['quote_rate']
ret=(ret/100-0.04/252)*100
ret_jz=get_price('000300.SH', None, date, '1d', ['quote_rate'], False, 'pre', 60, is_panel=1)
['quote_rate']
ret['ret_jz']=(ret_jz/100-0.04/252)*100
#进行线性回归
y=list(ret['ret_jz'])
df=pd.DataFrame(columns=['alpha'])
for i in ret.columns:
    x = list(ret[i])
    beta, stockalpha, r_value, p_value, slope_std_error=stats.linregress(y, x)
    df.loc[i]=stockalpha
```

```
df=pd.DataFrame(df).sort_values(by='alpha',ascending=True)
df['syboml']=df.index
df[df['syboml']!='ret_jz']
df
```

运行结果：

	alpha	syboml
300216.SZ	-1.090145	300216.SZ
300450.SZ	-0.872159	300450.SZ
300115.SZ	-0.821623	300115.SZ
300136.SZ	-0.676162	300136.SZ
300077.SZ	-0.637178	300077.SZ
300156.SZ	-0.576768	300156.SZ
300145.SZ	-0.562735	300145.SZ
300088.SZ	-0.505274	300088.SZ
300222.SZ	-0.491152	300222.SZ
300433.SZ	-0.457817	300433.SZ
300628.SZ	-0.455306	300628.SZ
300323.SZ	-0.449230	300323.SZ
300073.SZ	-0.421026	300073.SZ
300274.SZ	-0.401020	300274.SZ
300496.SZ	-0.400333	300496.SZ
300244.SZ	-0.397090	300244.SZ
300078.SZ	-0.393928	300078.SZ
300083.SZ	-0.371372	300083.SZ
300014.SZ	-0.360717	300014.SZ

四、最终结果

策略回测区间：2014.01.01-2018.01.31

回测资金：1000000

回测频率：日级

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码：

```
import pandas as pd
import numpy as np
from scipy import stats

#=====初始化函数=====
def initialize(account):
    g.n = 10 #设置最大持仓数量为 20 只股票
    g.longday = 60 #设置数据长度
    run_monthly(trade,date_rule=1) #将 trade 交易函数设置为定时运行：每个月第一个交易日
    account.stock = [] #储存上期的股票池
    g.setstock='399006.SZ'
    set_benchmark(g.setstock)

#=====主函数（选股+交易）=====
def trade(account, data):
    #执行选股函数：CAPM(), 并将结果导入，该股票列表是需要买入的个股。
    needstock_list = CAPM()
    #获取上期持仓个股
    holdstock_list = list(account.stock)
    #确定本期需要卖出的个股
    sell_list = list(set(holdstock_list)-set(needstock_list))
    #执行卖出操作，运用 for 循环，逐个操作。
    for s in sell_list:
        order_target(s,0)
```

```

#确定本期需要买入的个股，其余即为继续持仓的个股
buy_list=[]
for i in needstock_list:
    if i in holdstock_list:
        pass
    else:
        buy_list.append(i)
#确定可用资金，平分分配至需买入的个股
n=len(buy_list)
cash=account.available_cash/n
#执行买入操作
for s in range(0,n,1):
    stock=list(buy_list)[s]
    order_target_percent(stock,0.1)
#操作完毕，将选股结果放到上期股票池储存变量中，以备下次使用。
account.stock = frozenset(needstock_list)
#=====选股函数=====
def CAPM():
    date=get_last_datetime().strftime('%Y%m%d')
    stock_list=stock(date)
    ret=get_price(stock_list, None, date, '1d', ['quote_rate'], False, 'pre', g.longday, is_panel=1)
    ['quote_rate']
    ret=(ret/100-0.04/252)*100
    ret_jz=get_price('000300.SH', None, date, '1d', ['quote_rate'], False, 'pre', g.longday, is_panel=1)
    ['quote_rate']
    ret['ret_jz']=(ret_jz/100-0.04/252)*100
    #进行线性回归
    y=list(ret['ret_jz'])
    df=pd.DataFrame(columns=['alpha'])
    for i in ret.columns:
        x = list(ret[i])
        beta, stockalpha, r_value, p_value, slope_std_error=stats.linregress(y, x)
        df.loc[i]=stockalpha
    df=pd.DataFrame(df).sort_values(by='alpha',ascending=True)
    df['syboml']=df.index
    df=df[df['syboml']!='ret_jz']
    #获取股票的代码
    needstock_list=[]
    for s in range(0,g.n,1):
        needstock_list.append(df.iloc[s]['syboml'])
    return needstock_list
#=====以沪深 300 为股票池，除去 ST 和停牌=====
def stock(date):
    stk=list(get_index_stocks(g.setstock,date))
    price=get_price(stk, None, date, '1d', ['is_paused', 'is_st'], False, None, 1, is_panel=1)
    stopstk=price['is_paused'].iloc[-1]
    ststk=price['is_st'].iloc[-1]
    startstk=(stopstk[stopstk==0].index)
    okstk=(ststk[ststk==0].index)
    tradestk=list(set(startstk)&set(okstk))
    return tradestk

```

第九篇：Fama-French 三因子模型应用

导语：在 CAPM 模型的基础上，再向大家讲述 Fama-French 的三因子模型，并构建策略，实际应用于 A 股市场。

一、策略阐述

Fama-French 三因子模型由来

Fama 和 French 在研究股票超额收益率时，提出了一个观点：小公司股票、以及具有较高股权账面-市值比的股票，其历史平均收益率一般会高于 CAPM 模型所预测的收益率。

Fama 和 French 认为：1. 市值较小的公司通常规模比较小，公司整体风险更大，需要获得更高的收益来补偿投资者；2. 账面市值比是账面的所有者权益除以市值，简称 BM。BM 效应是指账面市值比效应，指 BM 值较高的公司平均月收益率高于 BM 值较低的公司。

关于 BM 效应，国内外学者已做了许多研究。Fama 和 French（1992）研究了 1963 到 1990 年所有在 NYSE、AMEX、NASDAQ 上市的股票，发现 BM 值最高的组合月均收益率超过 BM 值最低的组合达 1.53%。肖军，徐信忠（2004）以 1993 年 6 月至 2001 年 6 月沪深股市 A 股股票为样本，计算持有一年、两年、三年的收益率数据，认为 BM 效应存在。

模型介绍

顾名思义，Fama-French 三因子模型中包含三个因子：市值因子(SMB)、账面市值比因子(HML)、市场风险因子(RM)。三因子模型的本质就是把 CAPM 中的未被解释的超额收益分解掉，将其分解成 SMB、BM、RM 和其他未能解释的因素(a)，公式表达即为

$$R_i = a_i + b_i R_M + s_i E(SMB) + h_i E(HML) + \epsilon_i$$

其中：

$R_i = E(r_i - r_f)$ ，指股票 i 比起无风险投资的期望超额收益率。

$R_M = E(r_M - r_f)$ ，为市场相对无风险投资的期望超额收益率。

$E(SMB)$ 是小市值公司相对大市值公司股票的期望超额收益率。

$E(HML)$ 则是高 B/M 公司股票比起低 B/M 的公司股票的期望超额收益率。

a 即观测值，线性回归的常数项。

ϵ_i 是回归残差项。

$E(SMB)$ 的计算方式如下：把市场里面的所有股票按市值排序，随后分成两份：第一份是大市值股票（市值前 20% 的股票），第二份是小市值股票（市值后 20% 的股票）。记大市值股票的平均期望收益率为 $E(r_S)$ ，小市值股票的期望收益率为 $E(r_B)$ 。那么 $E(SMB) = E(r_S) - E(r_B)$ 。 $E(HML)$ 的定义也类似。

假设三因子模型是正确的，市场风险、市值风险、账面市值比这三类风险能很好地解释个股的超额收益，那么 a 的长期均值应该是 0。如果短期内对于某个时期的股票，回归得到 $a < 0$ ，说明这段时间个股收益率偏低。根据有效市场假设，任何非理性的价格最终都会回归

理性，这些短期内收益率偏低的个股，最终都会涨回去。

Fama-French 三因子模型实际应用

考虑到三因子模型主要偏向小市值个股，因此本篇内容创业板指数为基准指数，来向大家展示三因子模型。

策略交易规则：

A. 设置调仓频率，每 20 个交易日进行调仓。

B. 设置样本长度，用于线性回归，考虑到模型观测值 a 长期为 0，因此选取了过去 252 个交易日的样本数据。

C. 持仓数量为 a 值最小的 20 只股票。

策略选股：

第一步：调仓日对于过去 252 个交易日的数据进行回归分析，自变量为：
 RM 、 $E(SMB)$ 、 $E(HML)$ ，应变量为 R_i ，每个股票的回归结果取常数项，即为 a 值。

第二步：选出 a 最小的 20 支股票。

以下为策略实现的基本信息：

策略实现难度：4

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
set_benchmark()	设置基准指数
order_value()	按金额下单
get_index_stocks()	获取指数成分股
get_all_trade_days()	获取所有交易日
get_price()	获取历史行情数据
get_candle_stick()	获取股票、指数、基金的历史行情数据，包括分钟级、日级、周级、月级、年级数据
get_last_datetime()	获取前一个交易日或者前一分钟的时间
get_fundamentals()	查询财务数据
account.available_cash	账户当前可用资金
g.	全局变量 g

二、代码示意图



三、编写释义

三因子模型的核心部分是计算因子值和因子线性回归，建议初学者前往研究环境操作。

因子值的计算完毕后需整理成一个 DataFrame 格式，以便后续进行因子回归，该部分操作需要同学们熟悉 DataFrame 的常用操作，以下是作者实现三因子模型中，研究环境的代码草稿，分享给同学们。

以下代码可在 MINDGO 研究环境中模仿学习：

```
import pandas as pd
import numpy as np
from sklearn import linear_model

def stock(date): #以沪深 300 为股票池，除去 ST 和停牌
    stk=list(get_index_stocks('000300.SH',date))
    price=get_price(stk, None, date, '1d', ['is_paused', 'is_st'], False, None, 1, is_panel=1)
    stopstk=price['is_paused'].iloc[-1]
    ststk=price['is_st'].iloc[-1]
    startstk=(stopstk[stopstk==0].index)
    okstk=(ststk[ststk==0].index)
    tradestk=list(set(startstk)&set(okstk))
    return tradestk

#=====获取当天交易日的前 N 个交易日的日期
def tradeday(today,n):
    daylist=list(get_all_trade_days().strftime('%Y%m%d'))
```

```
calnum = daylist.index(today)#获取今日日期在整个月历中的序
lasttrade=daylist[calnum-n]#去前 N 个序的交易日
return lasttrade
```

```
today='20180131'
ldate=tradeday(today,60)
stock_list = stock(ldate) #获取交易股票列表
stock_num = int(len(stock_list)*0.2)
#查询市值、所有者权益数据
q =
query(valuation.symbol,valuation.market_cap,balance.total_equity).filter(valuation.symbol.in_(stock_list),)
basic = get_fundamentals(q, date = ldate)
basic['B/M']= basic['balance_total_equity']/basic['valuation_market_cap']
ret=get_price(stock_list, None, today, '1d', ['quote_rate'], False, 'pre', 60, is_panel=1)['quote_rate']
ret=ret/100
ret_jz=get_price('000300.SH', None, today, '1d', ['quote_rate'], False, 'pre', 60, is_panel=1)
['quote_rate']
ret_jz=ret_jz/100-0.04/252
df=pd.DataFrame()
df['Rm']=ret_jz
basic.index=basic['valuation_symbol']
del basic['balance_total_equity']
for i in ['valuation_market_cap','B/M']:
    basic = pd.DataFrame(basic).sort_values(by=i, ascending=False)
    stockmax=list(basic.iloc[:stock_num]['valuation_symbol'])
    stockmin=list(basic.iloc[-stock_num:]['valuation_symbol'])
    df[i]=np.mean(ret[stockmin].T)-np.mean(ret[stockmax].T)
ret=ret-0.04/252
df.columns=['RM','SMB','HML']
clf = linear_model.LinearRegression()
#数据无法获取导致的，处理后的 nan 使其为 0
ret.iloc[:] = ret.iloc[:].fillna(0)
df.iloc[:] = df.iloc[:].fillna(0)
#对三因子进行线性回归
x_list=['RM','SMB','HML']
df2=pd.DataFrame(index=['alpha'])
for i in ret.columns:
    y = ret[i].values
    x = df[x_list].values
    clf.fit(x,y)
    df2[i]=clf.coef_[0]
df2=df2.T
df2 = pd.DataFrame(df2).sort_values(by='alpha', ascending=True)
df2.iloc[:10]
```

因子表格：(PS：通过可以对照表格格式，运用 DataFrame 对象基本操作来完成它!)

A、三因子数据

	RM	SMB	HML
2017-11-08	-0.001659	0.009764	-0.006559
2017-11-09	0.006741	-0.005388	0.006308
2017-11-10	0.008641	-0.005918	0.020157
2017-11-13	0.003741	-0.003043	0.003581
2017-11-14	-0.007159	-0.000878	-0.013439
2017-11-15	-0.006459	-0.002609	-0.014312
2017-11-16	0.007541	-0.001883	0.015970
2017-11-17	0.003741	-0.041582	-0.029428
2017-11-20	0.005441	-0.002973	0.008952
2017-11-21	0.017641	-0.013627	0.002454

B、个股 Ri 数据

	000001.SZ	000002.SZ	000008.SZ	000009.SZ	000060.SZ	000063.SZ	000069.SZ	000100.SZ	000156.SZ	000157.SZ	...	601966.SH	601985.SH	601988.SH
2017-11-08	0.017458	-0.007699	-0.016624	0.020146	-0.004692	-0.025122	-0.012064	-0.010576	0.005543	0.002039	...	0.001450	-0.010298	0.010298
2017-11-09	0.016329	0.000926	-0.005739	-0.003890	0.002573	0.064760	0.019118	0.054578	-0.000159	-0.000159	...	-0.006586	-0.001439	-0.001439
2017-11-10	-0.002592	-0.005941	0.011064	0.004835	-0.019233	0.009000	-0.006069	-0.006147	-0.002285	0.008613	...	-0.002315	-0.010415	-0.010415
2017-11-13	0.048621	-0.016517	-0.011258	-0.005128	0.012804	0.037561	0.002219	-0.004175	-0.007972	-0.004507	...	0.019830	-0.002750	0.002750
2017-11-14	0.003717	0.053056	0.002086	-0.026376	-0.002901	-0.000706	-0.008463	-0.028385	-0.007317	-0.015443	...	-0.027172	-0.000159	-0.000159
2017-11-15	-0.004020	-0.002264	0.015518	-0.005287	-0.032240	-0.015199	0.001037	0.001916	0.007051	-0.004594	...	-0.013768	-0.001458	-0.001458
2017-11-16	0.015345	0.017422	-0.011184	-0.001448	-0.004894	0.052034	0.022541	0.006052	-0.006601	-0.000159	...	-0.005678	-0.032669	-0.032669
2017-11-17	0.005948	-0.001541	-0.014652	-0.034998	-0.034412	-0.048972	0.015028	-0.037196	-0.055635	-0.009068	...	-0.031791	0.011938	0.020157
2017-11-20	0.081025	-0.000159	0.006628	0.015884	0.013634	0.088607	-0.018571	0.006251	0.015097	-0.004653	...	-0.003024	-0.012111	0.004653
2017-11-21	0.013876	0.099841	-0.005777	-0.002791	0.000813	0.019968	0.065492	0.027442	0.009608	0.013385	...	0.010761	0.002529	-0.002529

最终结果 a:

	alpha
601899.SH	-0.216540
600256.SH	0.008807
600157.SH	0.024980
002252.SZ	0.033125
000793.SZ	0.104564
002411.SZ	0.123665
600900.SH	0.154926
002424.SZ	0.216914
600816.SH	0.222209
600518.SH	0.250494

四、最终结果

策略回测区间：2014.01.01-2018.01.31

回测资金：1000000

回测频率：日级

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码：

```
import pandas as pd
import numpy as np
from sklearn import linear_model
#=====初始化函数=====
def initialize(account):
    g.bstk = '399006.SZ' #设置指数
    set_benchmark(g.bstk)#设置基准指数
    g.day = 0 #记录运行天数
    g.tradeday = 20 #调仓频率
    g.stock = [] #储存上期的股票池
    g.trade = False #是否调仓的开关
    g.longday = 252 #样本长度
    g.stocknum = 20 #持仓数量
    pass
#=====盘前运行=====
def before_trading_start(account,data):
    #判断是否调仓
    if g.day%g.tradeday==0:
        g.trade=True
    else:
        g.trade=False
```

```

    g.day=g.day+1
#=====盘中交易=====
def handle_data(account, data):
    if g.trade==True:
        #获取选股结果
        needstock_list = alpha_FF()
        #获取上期持仓个股
        holdstock_list = list(g.stock)
        #确定本期需要卖出的个股
        sell_list = list(set(holdstock_list)-set(needstock_list))
        #执行卖出操作，运用 for 循环，逐个操作。
        for s in sell_list:
            order_target(s,0)
        #确定本期需要买入的个股，其余即为继续持仓的个股
        buy_list=[]
        for i in needstock_list:
            if i in holdstock_list:
                pass
            else:
                buy_list.append(i)
        #确定可用资金，平分分配至需买入的个股
        n=len(buy_list)
        cash=account.available_cash/n
        #执行买入操作
        for s in range(0,n,1):
            stock=list(buy_list)[s]
            order_value(stock,cash)
        #操作完毕，将选股结果放到上期股票池储存变量中，以备下次使用。
        g.stock = frozenset(needstock_list)
    else:
        pass
#=====获取股票池=====
def stock(yestoday,today): #要的是过去交易日的股票池，但换仓日不应该停牌
    stk2=list(get_index_stocks(g.bstk,yestoday))#过去的
    stk=list(get_index_stocks(g.bstk,today))#换仓日的
    #获取换仓日的股票再换仓日的 ST 和停牌情况
    price=get_price(stk, None, today, '1d', ['is_paused', 'is_st'], False, None, 1, is_panel=1)
    stopstk=price['is_paused'].iloc[-1]
    ststk=price['is_st'].iloc[-1]
    startstk=(stopstk[stopstk==0].index)
    okstk=(ststk[ststk==0].index)
    tradestk=list(set(startstk)&set(okstk)&set(stk2))#符合不停牌非 ST 和过去交易日的沪深 300
成分股
    return tradestk

#=====获取当天交易日的前 N 个交易日的日期

```

```

def tradeday(today,n):
    daylist=list(get_all_trade_days().strftime('%Y%m%d'))
    calnum = daylist.index(today)#获取今日日期在整个月历中的序
    lasttrade=daylist[calnum-n]#去前 N 个序的交易日
    return lasttrade

=====alpha 计算函数=====
def alpha_FF():
    today=get_last_datetime().strftime('%Y%m%d')#获取当天日期 格式 YMD
    #获取前 N 个交易日的日期
    ldate=tradeday(today,g.longday)
    stock_list = stock(ldate,today) #获取交易股票列表
    stock_num = int(len(stock_list)*0.2)
    #查询市值、所有者权益数据
    q =
query(valuation.symbol,valuation.market_cap,balance.total_equity).filter(valuation.symbol.in_(stock_list),)
    basic = get_fundamentals(q, date = ldate)
    basic['B/M']= basic['balance_total_equity']/basic['valuation_market_cap']
    ret=get_price(stock_list, None, today, '1d', ['quote_rate'], False, 'pre', g.longday, is_panel=1)
['quote_rate']
    ret=ret/100
    ret_jz=get_price(g.bstk, None, today, '1d', ['quote_rate'], False, 'pre', g.longday, is_panel=1)
['quote_rate']
    ret_jz=ret_jz/100-0.04/252
    df=pd.DataFrame()
    df['Rm']=ret_jz
    basic.index=basic['valuation_symbol']
    del basic['balance_total_equity']
    for i in ['valuation_market_cap','B/M']:
        basic = pd.DataFrame(basic).sort_values(by=i, ascending=False)
        stockmax=list(basic.iloc[:stock_num]['valuation_symbol'])
        stockmin=list(basic.iloc[-stock_num:]['valuation_symbol'])
        df[i]=np.mean(ret[stockmin].T)-np.mean(ret[stockmax].T)
    ret=ret-0.04/252
    df.columns=['RM','SMB','HML']
    clf = linear_model.LinearRegression()
    #数据无法获取导致的，处理后的 nan 使其为 0
    ret.iloc[:] = ret.iloc[:].fillna(0)
    df.iloc[:] = df.iloc[:].fillna(0)
    #对三因子进行线性回归
    x_list=['RM','SMB','HML']
    df2=pd.DataFrame(index=['alpha'])
    for i in ret.columns:
        y = ret[i].values
        x = df[x_list].values
        clf.fit(x,y)
        df2[i]=clf.coef_[0]

```

```
df2=df2.T
df2 = pd.DataFrame(df2).sort_values(by ='alpha', ascending=True)
#获取股票的代码
needstock_list=[]
for s in range(0,g.stocknum,1):
    needstock_list.append(list(df2.index)[s])
return needstock_list
```

第十篇：动量类多因子之择时中选股

导语：作为策略集锦压轴篇，向大家介绍完整的多因子选股策略，结合了光大证券多因子选股研报，希望能在多因子研究模块，给大家打好坚实基础。

一、策略阐述

动量因子

动量与反转效应是市场上经常出现的一种情况。所谓动量效应就是在前一段时间强势的股票，在未来一段时间继续保持强势；反转效应就是前一段时间弱的股票，未来一段时间会变强。但问题的关键是这个强势和弱势会保持多长时间和多大程度，这是动量、反转策略需要考虑的关键问题。

衡量市场、个股的动量效应，需要运用动量因子。以下是光大证券-多因子研究系列(五)：动量类因子测试研报中对于动量类因子的定义。

图表 3：动量类因子的定义

定义	因子
PM_12M	过去 12 个月的动量=过去 12 个月收益率
PM_1M	过去 1 个月的动量=过去 1 个月收益率
PM_24M	过去 24 个月的动量=过去 24 个月收益率
PM_2M	过去 2 个月的动量=过去 2 个月收益率
PM_36M	过去 36 个月的动量=过去 36 个月收益率
PM_3M	过去 3 个月的动量=过去 3 个月收益率
PM_60M	过去 60 个月的动量=过去 60 个月收益率
PM_6M	过去 6 个月的动量=过去 6 个月收益率
PM_9M	过去 9 个月的动量=过去 9 个月收益率
PM_6MC1M	PM_6MC1M=过去 6 个月的动量-过去 1 个月的动量
PM_12MC1M	PM_12MC1M=过去 12 个月的动量-过去 1 个月的动量
PM_12MC6M	12M 收益率的 6M 变化率=(1+12M 收益率)/(1+6 个月前的 12M 收益率)-1
PM_1DC1M	过去 1 个月的日收益率的最大值
Success	Success=1-过去一个月的收益率排名/股票总数
PM_1D	过去 1 日收益率
Price_3M	Price_3M=股价/过去 10 周股价平均值-1
Price_12M	Price_12M=股价/过去 10 周股价平均值-1
PM_5D_Indu1	过去 5 日收益率，减去行业均值
PM_1M_Indu1	过去 1 个月的动量=过去 1 个月收益率，减去行业均值

敬请参阅最后一页特别声明

- 4 -

证券研究报告

本篇内容选取 PM_1M、PM_3M、PM_6M、PM_9M、PM_12M、PM_12MC6M、PM_12MC1M、PM_6MC1M 这 8 个动量因子，向大家讲述动量因子选股策略。

动量多因子选股

第一步：以沪深 300 指数成分股为股票池，计算股票池内个股上月的 8 个动量因子。

第二步：计算上个月 8 个动量因子的超额收益数值，作为本期因子权重值。公式：上月因子超额收益=因子值前 10%的个股上月收益率-因子值后 10%的个股上月收益率。

第三步：删除上月无超额收益的动量因子，如果数量超过 4 个，则本月不再选股，定义为市场当前无明显动量，空仓避险。(择时模块)

第四步：计算出本月个股本月的 8 个动量因子，进行因子标准化处理，每个因子乘以上月因子超额收益值，并求和，最终值即为动量 alpha 值。

第五步：按 alpha 值挑选出前 20 只股票作为持仓。

注：标准化公式：因子标准值=(因子值-所有个股的因子平均值)/所有个股的因子标准差

动量多因子选股注意点

1.A 股市场历来以反转市著称，因此简单的通过动量选股在很大程度上是违背市场主流的，因此我们在第三步中，特意加入了动量因子择时模块，事先对市场的动量效应进行简单检测，随后选股操作。

2.考虑到动量类因子中，各因子的值差异严重，因子需要做标准化处理，更好的体现出每个因子的权重。

3.计算出因子超额收益后，需剔除无超额收益的上月动量因子，以更好体现有效因子的价值。

以下为策略实现的基本信息：

策略实现难度：5

实现过程中所需要用到的 API 函数，ps:通过 MindGo 量化交易平台 API 文档快速掌握：

需要用到的 API 函数	功能
initialize()	初始化函数
before_trading_start()	定时运行函数，每个交易日 9.00 运行
handle_data()	定时运行函数，按交易日或按分钟运行
set_benchmark()	设置基准指数
history	获取多只股票多属性的历史行情数据

二、代码示意图



三、编写释义

动量类多因子选股的核心部分是计算因子超额收益，标准化后求出最终 alpha 值，建议初学者前往研究环境操作。

当期因子值和当期因子权重计算完毕后需整理成一个 DataFrame 格式，该部分操作需要同学们熟悉 DataFrame 的常用操作，以下是作者实现动量类因子选股策略过程，研究环境的代码草稿，分享给同学们。

```
#=====以沪深 300 为股票池，除去 ST 和停牌=====
def stock(date):
    stk=list(get_index_stocks('000300.SH',date))
    price=get_price(stk, None, date, '1d', ['is_paused', 'is_st'], False, None, 1, is_panel=1)
    stopstk=price['is_paused'].iloc[-1]
    ststk=price['is_st'].iloc[-1]
    startstk=(stopstk[stopstk==0].index)
    okstk=(ststk[ststk==0].index)
    tradestk=list(set(startstk)&set(okstk))
    return tradestk
date='20180102'
stock_list=stock(date)
quote_rate_all=get_price(stock_list, None, date, '1d', ['quote_rate'], False, 'pre', 273, is_panel=1)
['quote_rate']
nowmonth=quote_rate_all.iloc[-21:]
quote_rate=quote_rate_all.iloc[:-21]
PM_1M=quote_rate.iloc[-21:].sum()
PM_3M=quote_rate.iloc[-63:].sum()
PM_6M=quote_rate.iloc[-126:].sum()
PM_9M=quote_rate.iloc[-189:].sum()
PM_12M=quote_rate.iloc[:].sum()
```

```

PM_12MC6M=quote_rate.iloc[:].sum()-quote_rate.iloc[-126:].sum()
PM_12MC1M=quote_rate.iloc[:].sum()-quote_rate.iloc[-21:].sum()
PM_6MC1M=quote_rate.iloc[-126:].sum()-quote_rate.iloc[-21:].sum()
quote_rate=quote_rate.T
quote_rate['PM_1M']=PM_1M
quote_rate['PM_3M']=PM_3M
quote_rate['PM_6M']=PM_6M
quote_rate['PM_9M']=PM_9M
quote_rate['PM_12M']=PM_12M
quote_rate['PM_12MC6M']=PM_12MC6M
quote_rate['PM_12MC1M']=PM_12MC1M
quote_rate['PM_6MC1M']=PM_6MC1M
yinzi_list=['PM_1M','PM_3M','PM_6M','PM_9M','PM_12M','PM_12MC6M','PM_12MC1M','PM_6MC1M']
for i in quote_rate.columns:
    if i in yinzi_list:
        pass
    else:
        del quote_rate[i]
nowmonth=nowmonth.sum()
quote_rate['nowmonth']=nowmonth
stocknum=int(len(quote_rate.index)*0.1)
yinzi_IClist=[]
zeshi=0
for i in quote_rate.columns:
    if i in yinzi_list:
        quote_rate = pd.DataFrame(quote_rate).sort_values(by =i, ascending=False)
        d=quote_rate['nowmonth'].iloc[:stocknum].mean()-quote_rate['nowmonth'].iloc[-
stocknum:].mean()
        if d <0:
            zeshi=zeshi+1
            yinzi_IClist.append(d)
        else:
            yinzi_IClist.append(d)
quote_rate_all=quote_rate_all.iloc[-252:]
PM_1M=quote_rate_all.iloc[-21:].sum()
PM_3M=quote_rate_all.iloc[-63:].sum()
PM_6M=quote_rate_all.iloc[-126:].sum()
PM_9M=quote_rate_all.iloc[-189:].sum()
PM_12M=quote_rate_all.iloc[:].sum()
PM_12MC6M=quote_rate_all.iloc[:].sum()-quote_rate_all.iloc[-126:].sum()
PM_12MC1M=quote_rate_all.iloc[:].sum()-quote_rate_all.iloc[-21:].sum()
PM_6MC1M=quote_rate_all.iloc[-126:].sum()-quote_rate_all.iloc[-21:].sum()
df=pd.DataFrame(columns=yinzi_list)
df['PM_1M']=PM_1M
df['PM_3M']=PM_3M
df['PM_6M']=PM_6M
df['PM_9M']=PM_9M
df['PM_12M']=PM_12M
df['PM_12MC6M']=PM_12MC6M
df['PM_12MC1M']=PM_12MC1M

```

```

df['PM_6MC1M']=PM_6MC1M
for i in yinzi_list:
    df[i] = (df[i] - df[i].mean())/df[i].std()
df=df.T
df['IC']=yinzi_IClist
df=df[df['IC']>0]
for i in df.columns:
    if i == 'IC':
        pass
    else:
        df[i]=df[i]*df['IC']
del df['IC']
alpha=df.sum()
df=df.T
df['alpha']=alpha
df['syboml']=df.index
df=pd.DataFrame(df).sort_values(by='alpha', ascending=False)
df

```

运行结果展示：

	PM_1M	PM_3M	PM_6M	PM_9M	PM_12M	PM_12MC6M	PM_12MC1M	PM_6MC1M	alpha	syboml
601012.SH	4.123145	14.061065	19.366088	9.526972	9.216536	0.746046	10.245820	19.580743	86.866414	601012.SH
002714.SZ	6.562176	17.113390	17.594895	7.662588	7.252438	0.159777	7.357948	16.378895	80.082108	002714.SZ
601155.SH	8.274691	21.117733	10.907329	7.345679	7.595568	1.531257	7.408383	8.045741	72.226382	601155.SH
000858.SZ	11.908460	16.180331	8.739185	7.133272	7.257843	1.768452	6.229325	3.801573	63.018442	000858.SZ
000063.SZ	3.649492	12.271792	10.394366	8.107323	7.273521	1.475492	8.001821	9.789831	60.963638	000063.SZ
600519.SH	6.879076	15.109679	9.774773	6.729464	6.511970	1.235994	6.397166	7.479089	60.117211	600519.SH
000503.SZ	31.798195	24.955145	13.084500	2.391914	-1.606008	-3.115220	-8.689439	-1.307172	57.511916	000503.SZ
601318.SH	3.484028	13.869412	9.400162	7.580641	6.307707	1.209557	6.871540	8.761455	57.484501	601318.SH
002202.SZ	11.405958	18.505062	12.193275	4.166090	2.939317	-0.853404	1.125008	7.914197	57.395502	002202.SZ
601933.SH	6.749080	12.615905	8.560032	6.819660	6.426785	1.416954	6.321978	6.186434	55.096827	601933.SH
002008.SZ	0.124674	9.543301	9.228921	7.643132	7.326167	1.711196	8.813751	10.253254	54.644397	002008.SZ
000725.SZ	3.365063	15.348574	8.153242	5.650363	6.194571	1.383439	6.760282	7.427304	54.282839	000725.SZ
002230.SZ	-7.196266	7.341559	11.270174	6.510335	7.394072	1.372346	10.450103	16.203049	53.345373	002230.SZ
002081.SZ	5.893926	18.467320	9.413323	3.909611	3.751891	0.026253	3.275850	7.568684	52.306857	002081.SZ
600196.SH	7.620583	13.376270	8.523289	4.827695	5.604555	1.043706	5.144782	5.708697	51.849575	600196.SH
600887.SH	5.774395	10.648087	9.237720	6.076796	5.068868	0.666613	4.890377	7.432293	49.795149	600887.SH
601888.SH	1.832381	11.040806	8.539594	4.538210	6.147788	1.291750	7.029256	8.627104	49.046889	601888.SH
002415.SZ	2.242450	10.758100	4.340365	6.706047	8.408540	3.097956	9.670158	3.727907	48.951523	002415.SZ

四、最终结果

策略回测区间：2014.01.01-2018.01.31

回测资金：1000000

回测频率：日级

回测结果：红色曲线为策略收益率曲线，蓝色曲线为对应的基准指数收益率曲线



策略源代码:

```
import pandas as pd
import numpy as np
#=====初始化函数=====
def initialize(account):
    g.n = 20 #设置最大持仓数量为 20 只股票
    run_monthly(trade,date_rule=1) #将 trade 交易函数设置为定时运行：每个月第一个交易日
    account.stock = [] #储存上期的股票池
    g.stock='000300.SH' #设置指数获取其成分股
    g.kc=False #因子择时结果是够为空仓
#=====主函数（选股+交易）=====
def trade(account, date):
    date=get_last_datetime().strftime('%Y%m%d')
    #执行选股函数：alphastock(date)，并将结果导入，该股票列表是需要买入的个股。
    needstock_list = alphastock(date)
    if g.kc == True:
        account.stock=[]
        for i in list(account.positions):
            order_target(i,0)
        pass
    else:
        #获取上期持仓个股
        holdstock_list = list(account.stock)
        #确定本期需要卖出的个股
        sell_list = list(set(holdstock_list)-set(needstock_list))
        #执行卖出操作，运用 for 循环，逐个操作。
        for s in sell_list:
            order_target(s,0)
        #确定本期需要买入的个股，其余即为继续持仓的个股
        buy_list=[]
        for i in needstock_list:
            if i in holdstock_list:
                pass
            else:
                buy_list.append(i)
        #确定可用资金，平分分配至需买入的个股
        n=len(buy_list)
        cash=account.available_cash/n
        #执行买入操作
        for s in range(0,n,1):
            stock=list(buy_list)[s]
            order_value(stock,cash)
        #操作完毕，将选股结果放到上期股票池储存变量中，以备下次使用。
        account.stock = frozenset(needstock_list)
#=====以沪深 300 为股票池，除去 ST 和停牌=====
def stock(date):
    stk=list(get_index_stocks(g.stock,date))
    #2015 年前上市的
    stk2=list(get_all_securities('stock','20150105').index)
    price=get_price(stk, None, date, '1d', ['is_paused', 'is_st'], False, None, 1, is_panel=1)
```

```

stopstk=price['is_paused'].iloc[-1]
ststk=price['is_st'].iloc[-1]
startstk=(stopstk[stopstk==0].index)
okstk=(ststk[ststk==0].index)
tradestk=list(set(startstk)&set(okstk)&set(stk2))
return tradestk
def alphastock(date):
    stock_list=stock(date)
    quote_rate_all=get_price(stock_list, None, date, '1d', ['quote_rate'], False, 'pre', 273, is_panel=1)
    ['quote_rate']
    nowmonth=quote_rate_all.iloc[-21:]
    quote_rate=quote_rate_all.iloc[:-21]
    PM_1M=quote_rate.iloc[-21:].sum()
    PM_3M=quote_rate.iloc[-63:].sum()
    PM_6M=quote_rate.iloc[-126:].sum()
    PM_9M=quote_rate.iloc[-189:].sum()
    PM_12M=quote_rate.iloc[:].sum()
    PM_12MC6M=quote_rate.iloc[:].sum()-quote_rate.iloc[-126:].sum()
    PM_12MC1M=quote_rate.iloc[:].sum()-quote_rate.iloc[-21:].sum()
    PM_6MC1M=quote_rate.iloc[-126:].sum()-quote_rate.iloc[-21:].sum()
    quote_rate=quote_rate.T
    quote_rate['PM_1M']=PM_1M
    quote_rate['PM_3M']=PM_3M
    quote_rate['PM_6M']=PM_6M
    quote_rate['PM_9M']=PM_9M
    quote_rate['PM_12M']=PM_12M
    quote_rate['PM_12MC6M']=PM_12MC6M
    quote_rate['PM_12MC1M']=PM_12MC1M
    quote_rate['PM_6MC1M']=PM_6MC1M

    yinzi_list=['PM_1M','PM_3M','PM_6M','PM_9M','PM_12M','PM_12MC6M','PM_12MC1M','PM_6MC1M']
    for i in quote_rate.columns:
        if i in yinzi_list:
            pass
        else:
            del quote_rate[i]
    nowmonth=nowmonth.sum()
    quote_rate['nowmonth']=nowmonth
    stocknum=int(len(quote_rate.index)*0.1)
    yinzi_IClist=[]
    zeshi=0
    for i in quote_rate.columns:
        if i in yinzi_list:
            quote_rate = pd.DataFrame(quote_rate).sort_values(by =i, ascending=False)
            d=quote_rate['nowmonth'].iloc[:stocknum].mean()-quote_rate['nowmonth'].iloc[-stocknum:].mean()
            if d <0:
                zeshi=zeshi+1
                yinzi_IClist.append(d)
            else:
                yinzi_IClist.append(d)

```

```

if zeshi > 4:
    g.kc = True
else:
    g.kc = False
quote_rate_all=quote_rate_all.iloc[-252:]
PM_1M=quote_rate_all.iloc[-21:].sum()
PM_3M=quote_rate_all.iloc[-63:].sum()
PM_6M=quote_rate_all.iloc[-126:].sum()
PM_9M=quote_rate_all.iloc[-189:].sum()
PM_12M=quote_rate_all.iloc[:].sum()
PM_12MC6M=quote_rate_all.iloc[:].sum()-quote_rate_all.iloc[-126:].sum()
PM_12MC1M=quote_rate_all.iloc[:].sum()-quote_rate_all.iloc[-21:].sum()
PM_6MC1M=quote_rate_all.iloc[-126:].sum()-quote_rate_all.iloc[-21:].sum()
df=pd.DataFrame(columns=yinzi_list)
df['PM_1M']=PM_1M
df['PM_3M']=PM_3M
df['PM_6M']=PM_6M
df['PM_9M']=PM_9M
df['PM_12M']=PM_12M
df['PM_12MC6M']=PM_12MC6M
df['PM_12MC1M']=PM_12MC1M
df['PM_6MC1M']=PM_6MC1M
for i in yinzi_list:
    df[i] = (df[i] - df[i].mean())/df[i].std()
df=df.T
df['IC']=yinzi_IClist
df=df[df['IC']>0]
for i in df.columns:
    if i == 'IC':
        pass
    else:
        df[i]=df[i]*df['IC']
del df['IC']
alpha=df.sum()
df=df.T
df['alpha']=alpha
df['syboml']=df.index
df=pd.DataFrame(df).sort_values(by='alpha', ascending=False)
#获取前 20 只股票的代码
needstock_list=[]
for s in range(0,g.n,1):
    needstock_list.append(df.iloc[s]['syboml'])
#完成函数，输入即为 10 只股票代码，作为策略的股票池
return needstock_list

```

第五章：量化研究专题

第一篇：用 matplotlib 绘图函数实现数据可视化

导语：数据分析能力是一项非常重要的能力，尤其是在分析股票数据时，挖掘其中的有用信息是成功的必要因素。而数据可视化可谓是秀数据分析能力的最好方式，本章内容主要介绍 python 的 matplotlib 模块，让你的数据分析结果，show 出来！

matplotlib 绘图

开始之前，还是学习一个模块导入操作

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

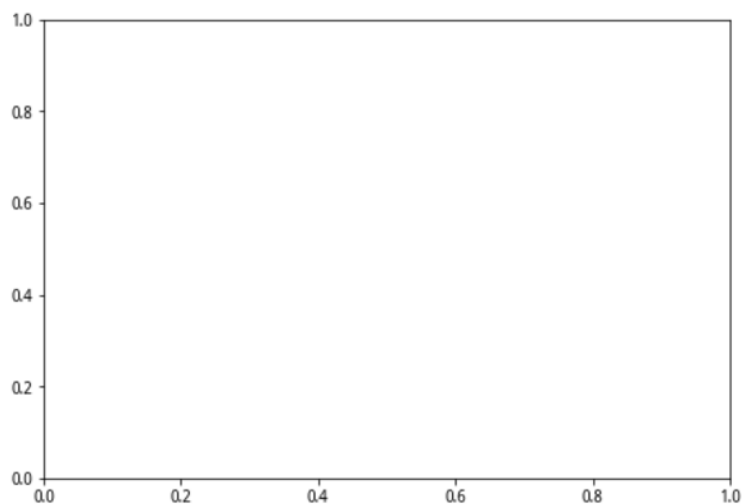
```
import numpy as np
```

让我们先搬上小白板

```
fig = plt.figure()
```

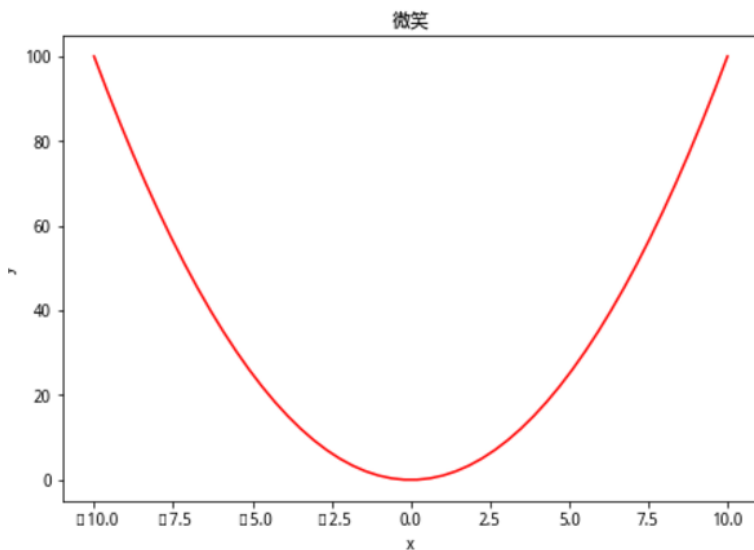
```
axes = fig.add_axes([0.2, 0.2, 1, 1]) # 左侧间距, 底部间距, 宽度, 高度 (从 0 到 1)
```

```
axes
```



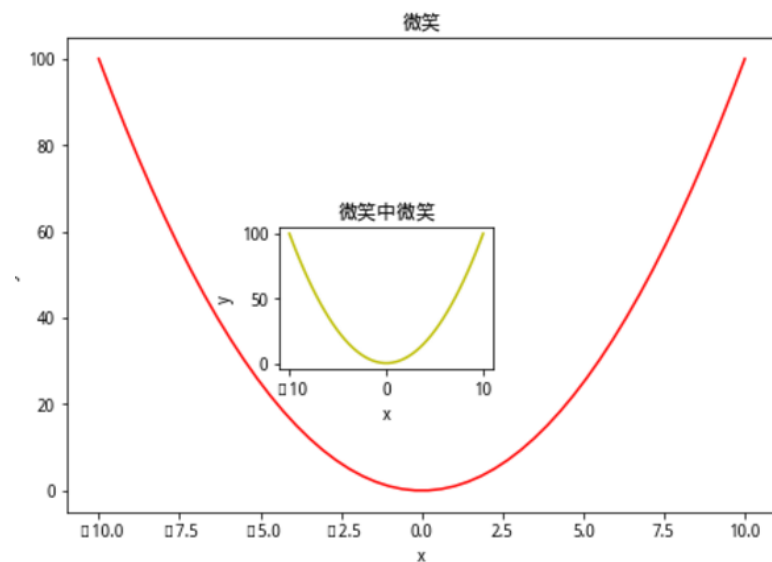
先画个 $y=x^2$ 曲线：

```
x = np.linspace(-10, 10)
y = x ** 2
axes.plot(x, y, 'r')
axes.set_xlabel('x') #设置 X 轴
axes.set_ylabel('y') #设置 Y 轴
axes.set_title('微笑') #设置标题
```



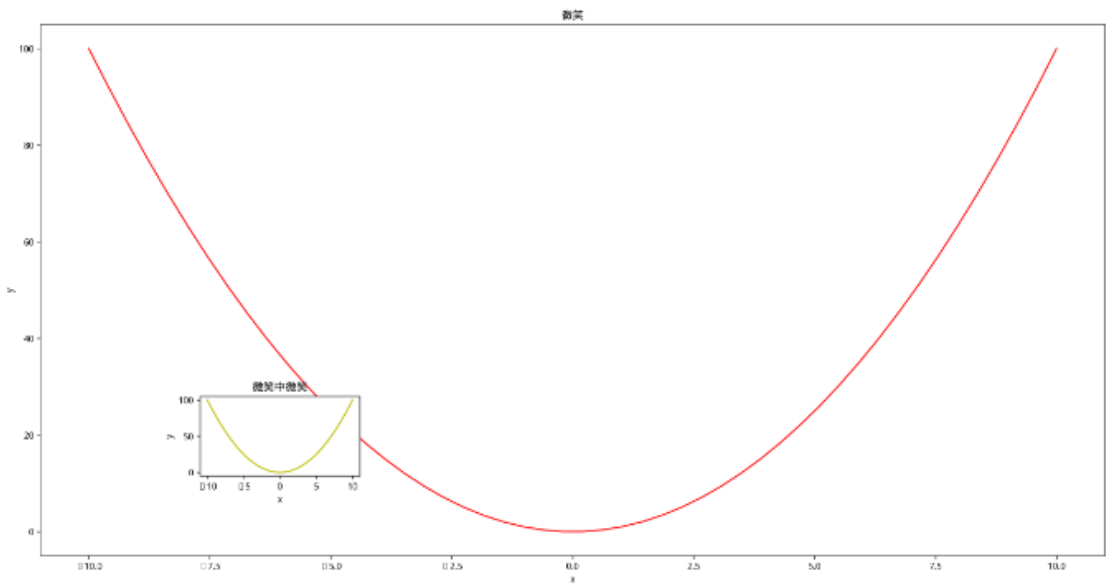
小白板上再插入一个小白板

```
axes2 = fig.add_axes([0.5, 0.5, 0.3, 0.3]) #插入面板 2
axes2.plot(x, y, 'y')
axes2.set_xlabel('x')
axes2.set_ylabel('y')
axes2.set_title('微笑中微笑')
```



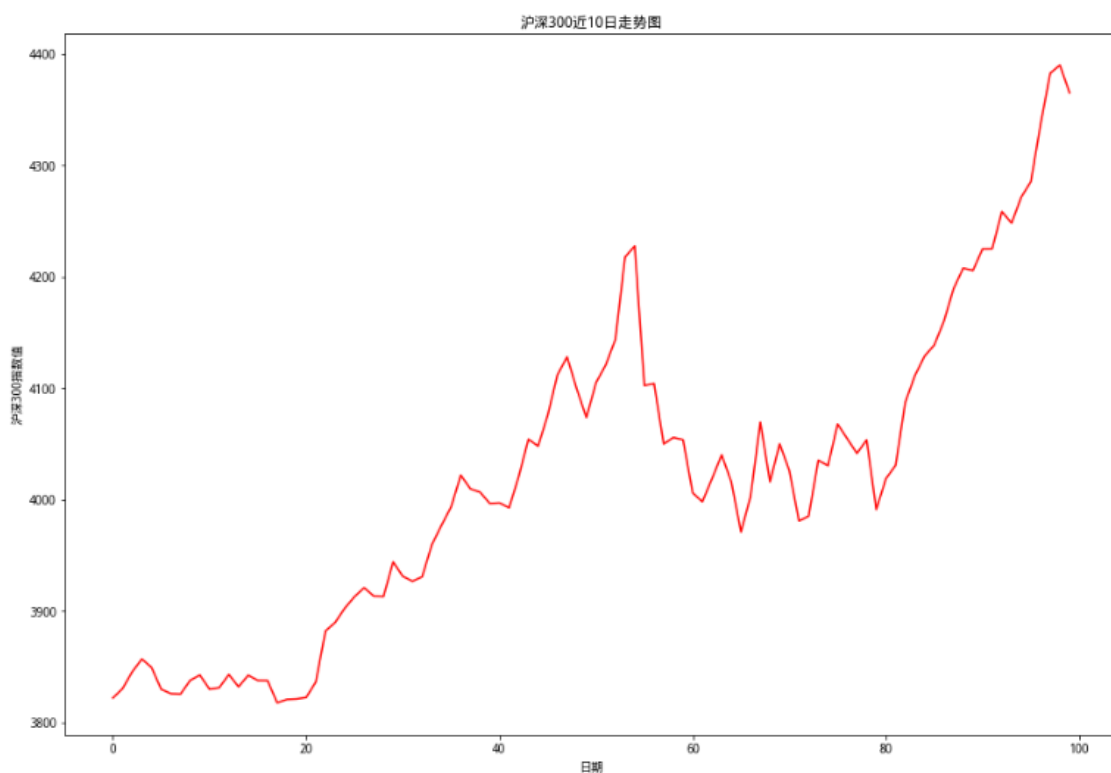
plt.figure()函数内部有 figsize 和 dpi 参数，用于设置图像的大小和精度。

我们将上述代码中 fig = plt.figure()改成 fig = plt.figure(figsize=(8,4), dpi=100)，结果如下：



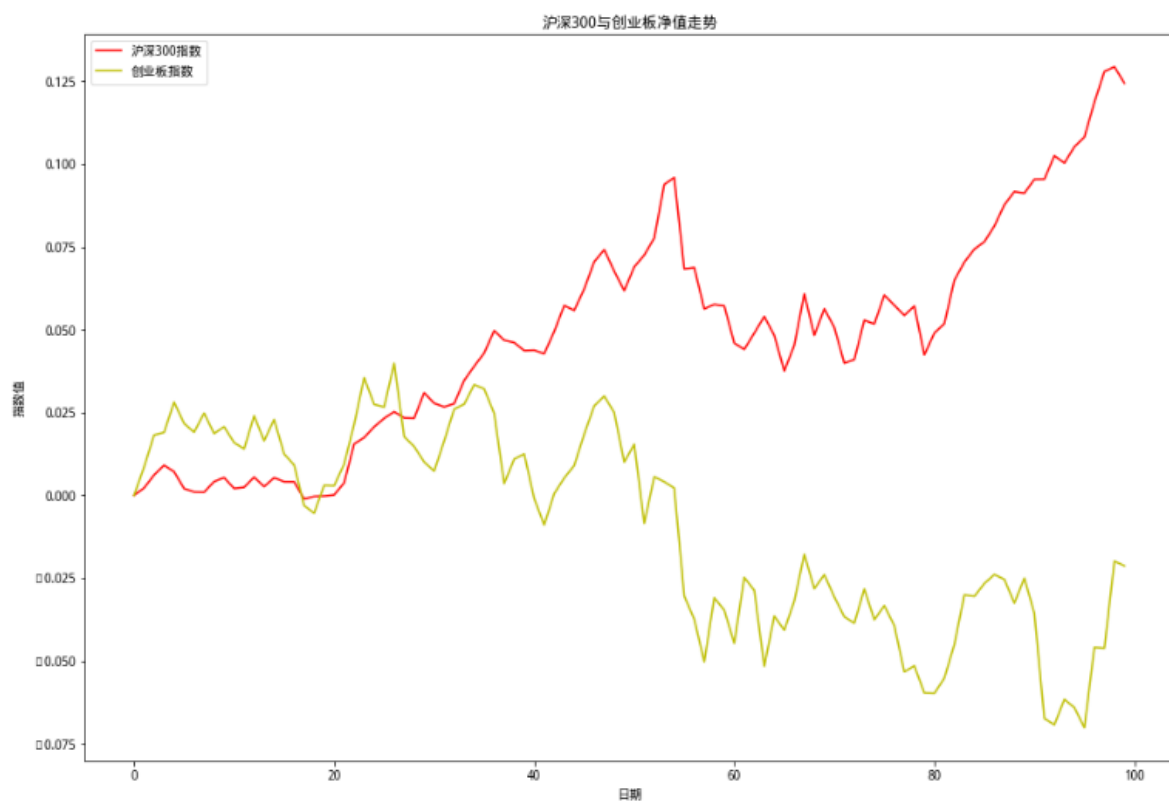
让我们尝试画一个沪深 300 指数收盘价走势图：

```
x1_list=get_price('000300.SH', None, '20180125', '1d', ['close'], False, 'pre', 100, is_panel=1)
['close']
fig = plt.figure()
axes = fig.add_axes([0.1, 0.1, 2, 2]) #插入面板 2
y=np.array(x1_list)
x=np.array(range(0,len(x1_list)))
axes.plot(x, y, 'r')
axes.set_xlabel('日期')
axes.set_ylabel('沪深 300 指数值')
axes.set_title('沪深 300 近 10 日走势图')
```



同时小白板上绘制沪深 300 指数和创业板指数（净值数据）

```
fig = plt.figure()
axes = fig.add_axes([0.1, 0.1, 2, 2]) #插入面板
x1_list=get_price('000300.SH', None, '20180125', '1d', ['close'], False, 'pre', 100, is_panel=1)
['close']
x1_list=list((x1_list-x1_list.iloc[0])/x1_list)
y=np.array(x1_list)
x=np.array(range(0,len(x1_list)))
axes.plot(x, y, 'r')
x2_list=get_price('399006.SZ', None, '20180125', '1d', ['close'], False, 'pre', 100, is_panel=1)
['close']
x2_list=list((x2_list-x2_list.iloc[0])/x2_list)
y2=np.array(x2_list)
x2=np.array(range(0,len(x2_list)))
axes.plot(x2, y2, 'y')
axes.set_xlabel('日期')
axes.set_ylabel('指数值')
axes.set_title('沪深 300 与创业板净值走势')
axes.legend(['沪深 300 指数','创业板指数'])
```



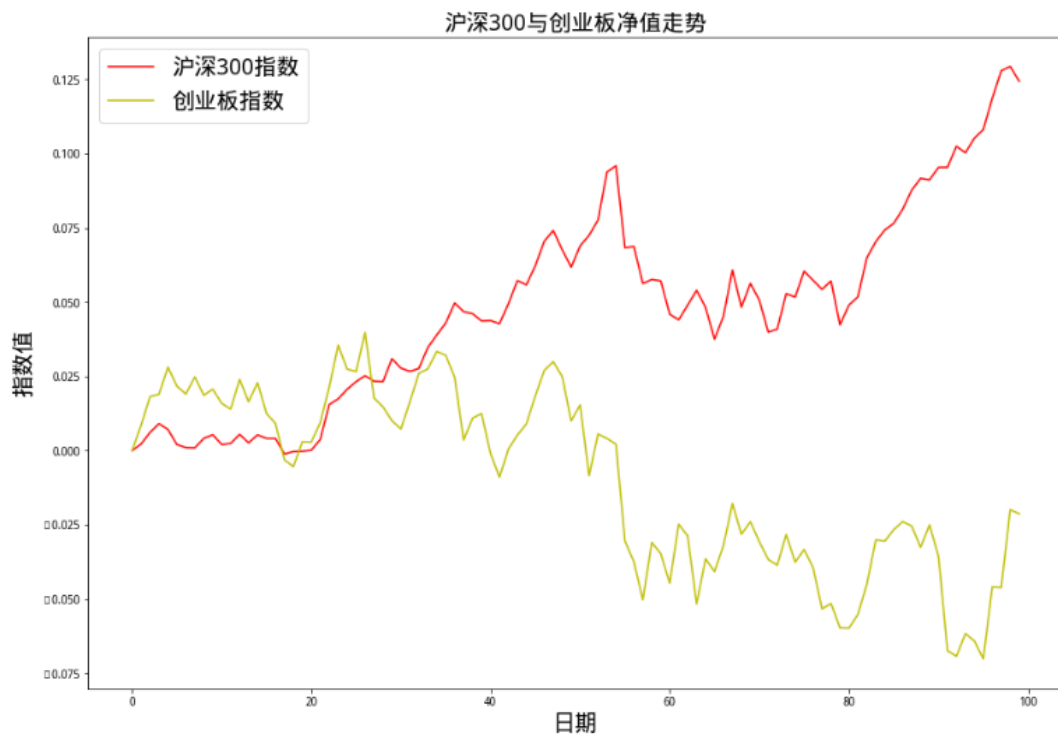
是不是觉得小白板上的标题和 X,Y 轴字体太小了，我们需要设置字体大小。

```
axes.set_xlabel('日期',fontsize=20)
```

```
axes.set_ylabel('指数值',fontsize=20)
```

```
axes.set_title('沪深 300 与创业板净值走势',fontsize=20)
```

```
axes.legend(['沪深 300 指数','创业板指数'],fontsize=20)
```



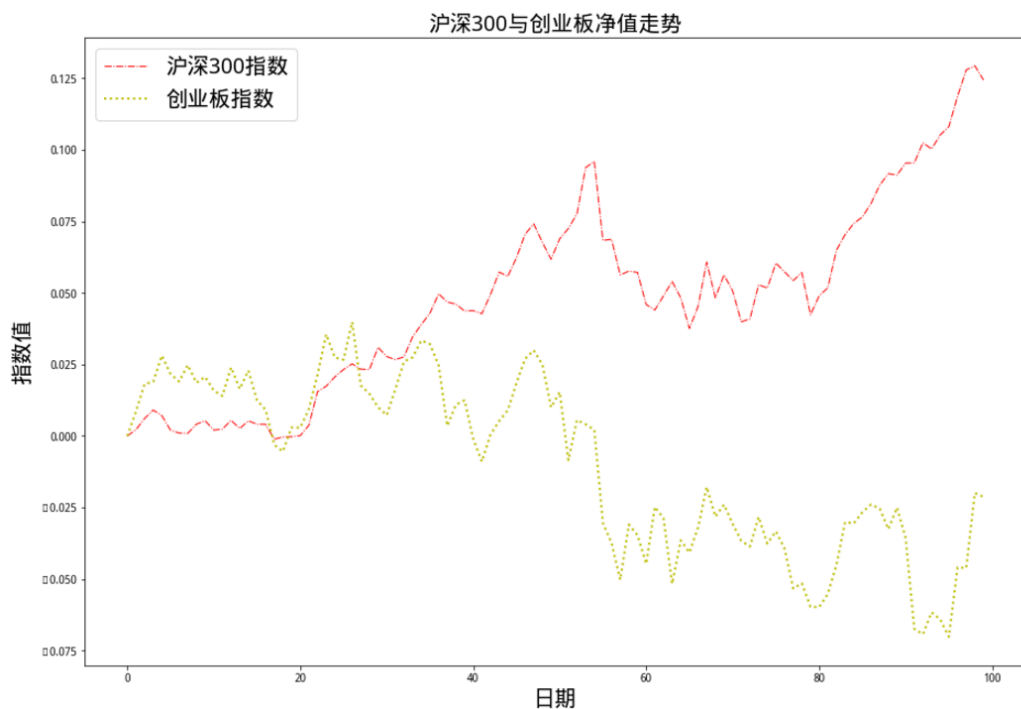
当然，我们可以通过全局操作，一条语句控制字体和大小

```
matplotlib.rcParams.update({'font.size': 10, 'font.family': 'serif'})
```

如何改变线条宽度、类型、颜色呢？`axes.plot()`函数内，`color`代表颜色参数，`linewidth`是宽度参数，`linestyle`是样式参数，将上述代码的`axes.plot()`内置参数进行如下修改。

```
axes.plot(x, y, color='r',linewidth=1,linestyle='-.')
```

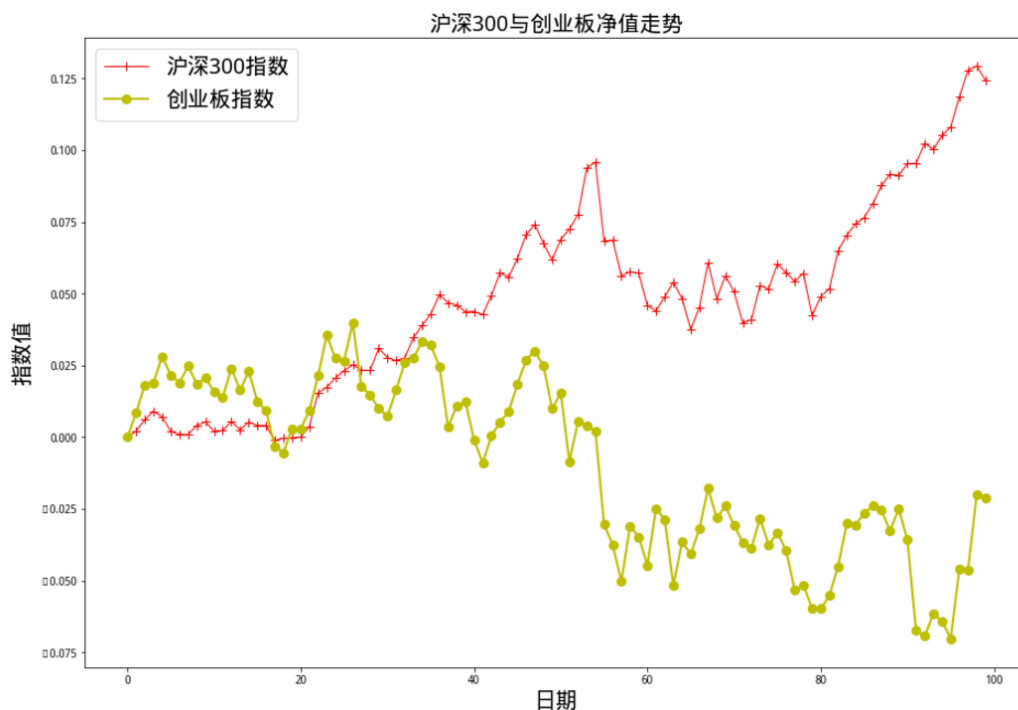
```
axes.plot(x2, y2, color='y',linewidth=2,linestyle=':')
```



除此之外，我们还能在曲线上做标记，并设置标的形状，大小，颜色。axes.plot()函数内 marker 代表标记形状，markersize 代表标记大小，markerfacecolor 代表标记颜色。将上述代码的 axes.plot() 内置参数进行如下修改。

```
axes.plot(x, y, color='r',linewidth=1,linestyle='-',marker='+', markersize=8, markerfacecolor='r')
```

```
axes.plot(x2, y2, color='y',linewidth=2,linestyle='-',marker='o', markersize=8, markerfacecolor='y')
```

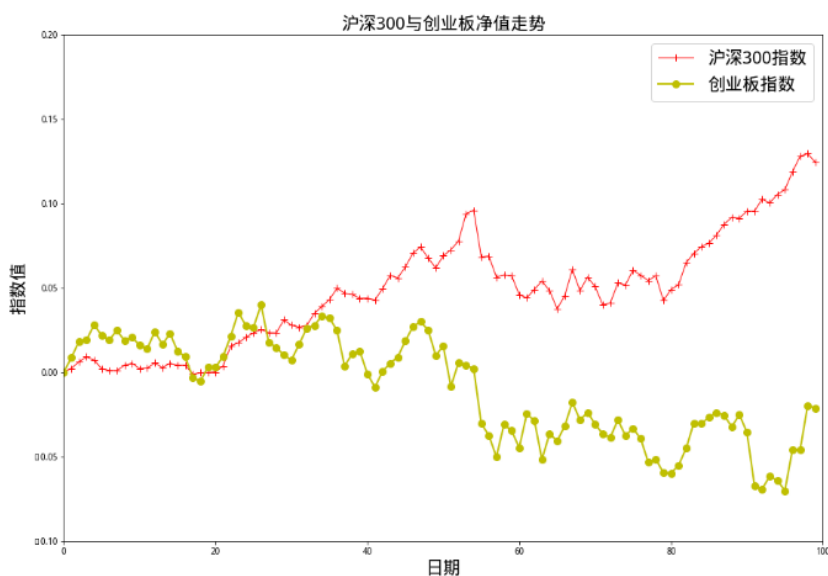


设置坐标轴范围

```
#设置范围
```

```
axes.set_ylim([-0.1,0.2 ])
```

```
axes.set_xlim([0, 100])
```



你也可以自定义坐标轴。

#设置 X 轴

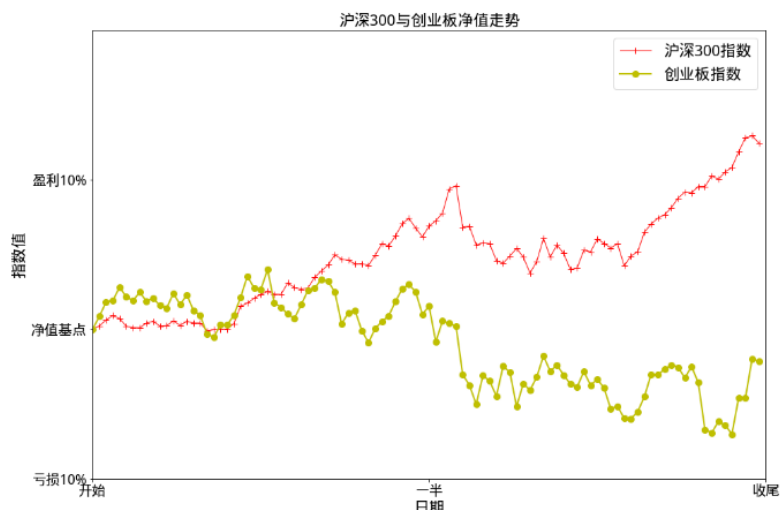
```
axes.set_xticks([0,50,100])
```

```
axes.set_xticklabels(['开始','一半','收尾'], fontsize=18)
```

#设置 Y 轴

```
axes.set_yticks([-0.1,0,0.1])
```

```
axes.set_yticklabels(['亏损 10%','净值基点','盈利 10%'], fontsize=18)
```



加入文字注释和自定义网格

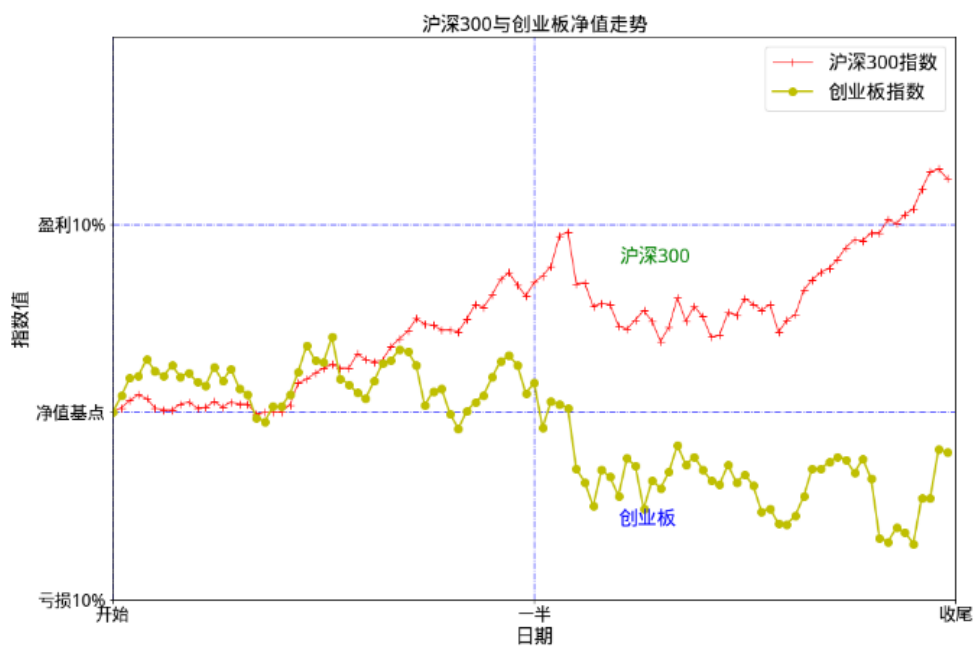
#网格

```
axes.grid(color='b', alpha=1, linestyle='-.', linewidth=1)
```

#文字注释

```
axes.text(60, -0.06, r'创业板', fontsize=20, color='blue')
```

```
axes.text(60, 0.08, r'沪深 300', fontsize=20, color='green')
```



接下来介绍其他二维绘图样式

```
1.axes.scatter()
```

```
fig = plt.figure()
```

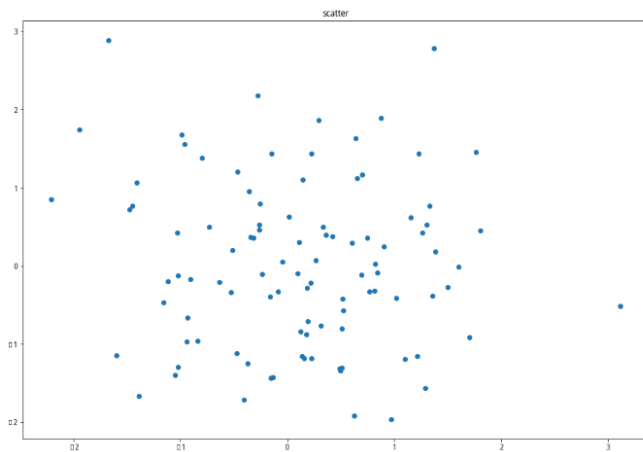
```
axes = fig.add_axes([0.1, 0.1, 2, 2])
```

```
x=np.random.randn(100)
```

```
y=np.random.randn(100)
```

```
axes.scatter(x,y)
```

```
axes.set_title('scatter')
```



```
2.axes.step()
```

```
fig = plt.figure()
```

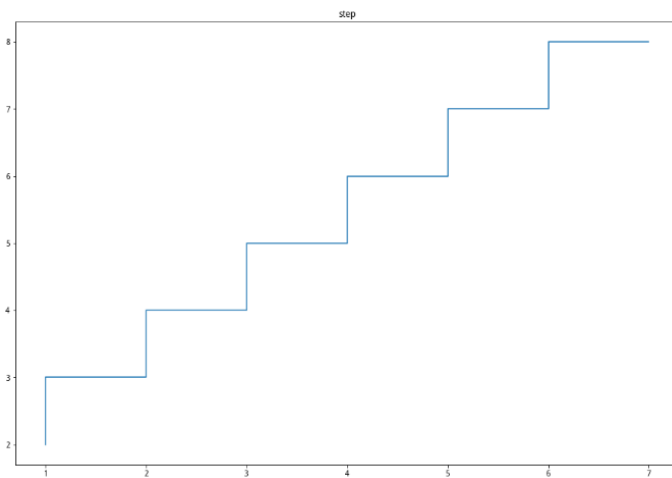
```
axes = fig.add_axes([0.1, 0.1, 2, 2])
```

```
x=[1,2,3,4,5,6,7,]
```

```
y=[2,3,4,5,6,7,8,]
```

```
axes.step(x, y)
```

```
axes.set_title('step')
```



3.axes.bar()

```
fig = plt.figure()
```

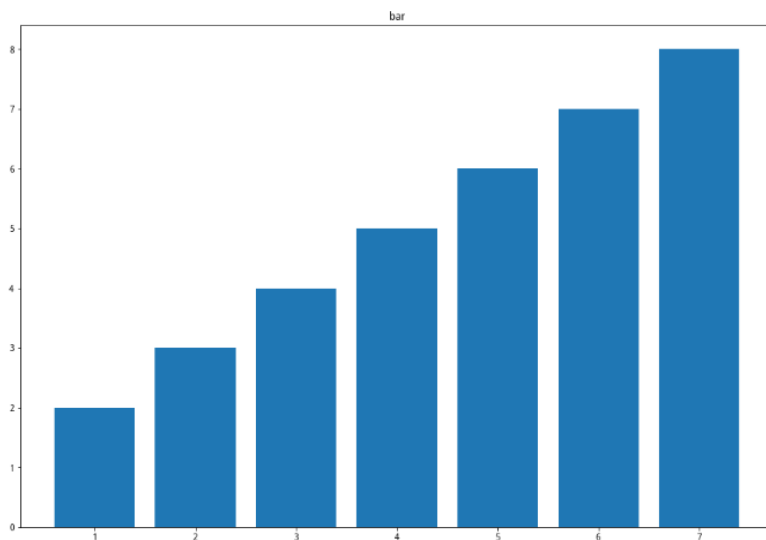
```
axes = fig.add_axes([0.1, 0.1, 2, 2])
```

```
x=[1,2,3,4,5,6,7,]
```

```
y=[2,3,4,5,6,7,8,]
```

```
axes.bar(x,y)
```

```
axes.set_title('bar')
```



4.axes.fill_between()

```
fig = plt.figure()
```

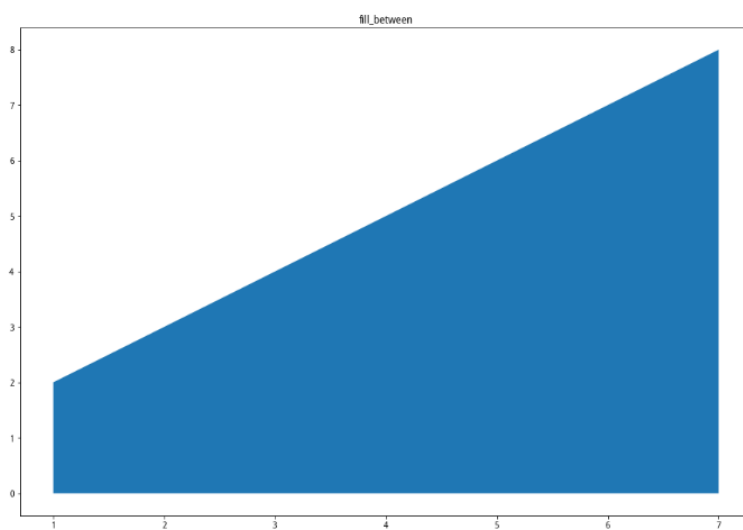
```
axes = fig.add_axes([0.1, 0.1, 2, 2])
```

```
x=[1,2,3,4,5,6,7,]
```

```
y=[2,3,4,5,6,7,8,]
```

```
axes.fill_between(x, y);
```

```
axes.set_title('fill_between')
```



学习绘制沪深 300 指数 K 线图走势，希望同学们结合上面的知识点，学习更多绘图技巧。

绘制结果：



绘制代码：

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.finance import candlestick2_ohlc
import datetime

data=get_price(['000300.SH'], None, '20171110', '1d', ['open','high','low','close'], True, None, 200,
is_panel=0)
data=data['000300.SH']
#时间转化格式
time=data.index
t=[]
for x in time:
    x=str(x).split()[0]
    x=x.split('-')
    x=x[0]+x[1]+x[2]
    x=int(x)
    t.append(x)
#画图数据
time=t
open1=list(data['open'])
high1=list(data['high'])
low1=list(data['low'])
close1=list(data['close'])
```

```

#画图
fig,ax = plt.subplots(figsize = (20,8),facecolor='pink')
fig.subplots_adjust()
plt.xticks()
plt.yticks()
plt.title("沪深 300K 线走势图")
plt.ylabel("股指")
ticks = ax.set_xticks(range(1,200,40))
labels = ax.set_xticklabels([time[0],time[40],time[80],time[120],time[160]])
candlestick2_ohlc(ax,open1,high1,low1,close1,width=0.6,colorup='red',colordown='green')

#支撑线
plt.plot([75,200],[3316,3954],'g',linewidth=10)

# 红星：回踩 1
plt.plot(75, 3316, 'r*', markersize = 40.0,label='趋势线')
plt.annotate(r'二次低位', xy=(75, 3316),
            xycoords='data', xytext=(-90, -50),
            textcoords='offset points', fontsize=26,
            arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=.2"))

# 红星：回踩 2
plt.plot(140, 3650, 'r*', markersize = 40.0)
plt.annotate(r'止跌，形成趋势线', xy=(140, 3650),
            xycoords='data', xytext=(-90, -50),
            textcoords='offset points', fontsize=26,
            arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=.2"))

# 红星：回踩 3
plt.plot(172, 3800, 'r*', markersize = 40.0)
plt.annotate(r'回踩趋势线', xy=(172, 3800),
            xycoords='data', xytext=(-90, -50),
            textcoords='offset points', fontsize=26,
            arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=.2"))

#MA5
data['ma5']=pd.rolling_mean(data['close'],5)
plt.plot(list(data['ma5']),label='五日均线')

#MA10
data['ma10']=pd.rolling_mean(data['close'],10)

```

```
plt.plot(list(data['ma10']),label='十日均线')
#MA20
data['ma20']=pd.rolling_mean(data['close'],20)
plt.plot(list(data['ma20']),label='二十日均线')
#MA30
data['ma30']=pd.rolling_mean(data['close'],30)
plt.plot(list(data['ma30']),label='三十日均线')
#MA60
data['ma60']=pd.rolling_mean(data['close'],60)
plt.plot(list(data['ma60']),label='六十日均线')
plt.legend()
print('沪深 300 走势图分析')
```

第二篇：运用 Scipy 模块实现统计技术

导语：python 语言中 numpy 和 pandas 模块是处理数据的利器，除此之外，继续向大家介绍 Scipy 模块，这个模块专门运用于统计和优化技术，本文主要讲述 Scipy 模块在统计中的运用。

统计学基础知识

开始之前，我们先导入 Scipy 模块

```
import numpy as np
import pandas as pd
import scipy.stats as stats
```

1.理解平均数和变异性

平均数也叫做集中趋势量数，一般有三种形式：均值、中位数和众数。

平均数	公式
均值(mean)	数据组中所有数值的总和除以该组数值的个数
中位数(median)	数据组中数据的中点。中位数对极值不敏感，均值对极值敏感。
众数(mode)	数据组中出现数据最多的数值

python 实现：

```
#获取平安银行过去 20 个交易日的收盘价数据，形成一个列表。
x_list=list(get_price('000001.SZ', None, '20180125', '1d', ['close'], True, None, 20, is_panel=1)
['close'].values)
```

```
print('平安银行过去 20 日收盘价均值： {}'.format(np.mean(x_list)))
print('平安银行过去 20 日收盘价中位数： {}'.format(np.median(x_list)))
print('平安银行过去 20 日收盘价众数： {}'.format(np.argmax(np.bincount(x_list))))
```

```
平安银行过去 20 日收盘价均值： 13.8315
平安银行过去 20 日收盘价中位数： 13.625
平安银行过去 20 日收盘价众数： 13
```

变异性反映数值之间的不同。平均数和变异性可共同描述数据分布的特征，并说明数据分布之间的差异。变异性的三种量数：极差、标准差、方差。用于反映一组数据的变异性、散布或者离散度。

python 实现

```
print('平安银行过去 20 日收盘价极差: {}'.format(np.max(x_list)-np.min(x_list)))
print('平安银行过去 20 日收盘价标准差: {}'.format(np.std(x_list)))
print('平安银行过去 20 日收盘价方差: {}'.format(np.var((x_list))))
```

```
平安银行过去 20 日收盘价极差: 1.8399999999999999
平安银行过去 20 日收盘价标准差: 0.602746007867327
平安银行过去 20 日收盘价方差: 0.36330274999999984
```

2.检验数据是否服从假设的分布。最为常见的解决方案是采用 K-S 检验。

单样本 K-S 检验的原假设是给定的数据来自和原假设分布相同的分布，在 SciPy 中提供了 kstest 函数，参数分别是数据、拟检验的分布名称和对应的参数。我们以上述平安银行数据为例。

```
stats.kstest(x_list, 'norm', (np.mean(x_list), np.std(x_list)))
```

```
KstestResult(statistic=0.17976051493694484, pvalue=0.48934819583934308)
```

从 K-S 检验结果看，数据符合正态分布。

K-S 检验结果查看技巧：一般 pvalue > 0.05 即为符合正态分布

3.在正态性的前提下，我们可进一步检验这组数据的均值是不是 0。典型的方法是 t 检验，其中单样本的 t 检验函数为 ttest_1samp。

```
stats.ttest_1samp(x_list, 0)
```

```
Ttest_1sampResult(statistic=100.02573215027174, pvalue=2.4856090267508661e-27)
```

从 t 检验看，数据的并均值不是 0。

4.计算简单相关系数

相关系数是反映两个变量之间关系的量化指标。这个统计值得值域范围是-1 到 1。含义是某一个变量发生变化时，另一个变量如何变化。如果变量变化方向相同，相关是正相关，反之，相关是负相关。ps：了解 python 实现过程即可。

$$r_{xy} = \frac{n \sum XY - \sum X \sum Y}{\sqrt{[n \sum X^2 - (\sum X)^2] [n \sum Y^2 - (\sum Y)^2]}}$$

其中 r_{xy} 是 X 与 Y 之间的相关系数

n 是样本规模

X 是变量 X 的具体数值 og.csdn.net/

Y 是变量 Y 的具体数值

XY 是每一个 X 值与相应的 Y 值的乘积

X^2 是 X 值的平方

Y^2 是 Y 值的平方

python 实现：我们获取万科 A 的数据与平安银行的数据进行相关系数计算

```
#我们再获取万科的过去 20 日的收盘价数据
y_list=list(get_price('000002.SZ', None, '20180125', '1d', ['close'], True, None, 20, is_panel=1)
['close'].values)
```

```
#计算相关系数
stats.pearsonr(x_list,y_list)[0]
```

0.8276759436667197

5.方差分析（F 检验）

投资者总是希望通过自己明智的选股逻辑，来获取超额收益，那么如何证明选股对获取更高收益率是显著影响的呢？我们需要通过方差分析来检验。

假设我们有 4 个策略，其中一个就是不选股，直接买指数，另外三个分别选择一只股票长期持有。

第一步：计算出 4 个策略的每天净值变化。

第二步：对 4 个策略的净值变化做方差分析。

第三步：得到分析结果的 p 值,如果 $p < 0.05$ ，则选股对获取更高收益率有显著影响，反之，无显著影响。

```
from scipy.stats import f_oneway
x1_list=get_price('000300.SH', None, '20180125', '1d', ['close'], False, 'pre', 252, is_panel=1)
['close']
x1_list=(x1_list-x1_list.iloc[0])/x1_list
x2_list=get_price('002601.SZ', None, '20180125', '1d', ['close'], False, 'pre', 252, is_panel=1)['close']
x2_list=(x2_list-x2_list.iloc[0])/x2_list
x3_list=get_price('601006.SH', None, '20180125', '1d', ['close'], False, 'pre', 252, is_panel=1)
['close']
x3_list=(x3_list-x3_list.iloc[0])/x3_list
x4_list=get_price('600216.SH', None, '20180125', '1d', ['close'], False, 'pre', 252, is_panel=1)
['close']
x4_list=(x4_list-x4_list.iloc[0])/x4_list
args=[x1_list,x2_list,x3_list,x4_list]
f_oneway(*args)
```

F_onewayResult(statistic=390.98877592011445, pvalue=3.4512917737493422e-168)

通过最后的 p 值结果，我们可以看到，选股对获取更高收益率有显著影响。

6.pearsonr 相关系数检验

大部分投资者都非常认可一个观点：“A 股市场有明显的大小盘轮动现象”，那么我们是否可以通过 pearsonr 相关系数来检验呢？以沪深 300 指数和创业板指数为例。

第一步：获取两者的收盘价，并计算成净值变化。

第二步：进行 pearsonr 相关系数检验，首先判断 P 值，p 值小于 0.05，则说明两者呈线性关系；反之，不存在线性关系。其次如果 $R > 0$ ，则代表正相关；反之，负相关，且相关强度参照下表。

r 值	相关强度
0.8-1.0	极强相关
0.6-0.8	强相关
0.4-0.6	中等相关
0.2-0.4	弱相关
0.0-0.2	极弱相关或者无相关

python 实现：

```
x1_list=get_price('000300.SH', None, '20180125', '1d', ['close'], False, 'pre', 252, is_panel=1)
['close']
x1_list=list((x1_list-x1_list.iloc[0])/x1_list)
x2_list=get_price('399006.SZ', None, '20180125', '1d', ['close'], False, 'pre', 252, is_panel=1)['close']
x2_list=list((x2_list-x2_list.iloc[0])/x2_list)
stats.pearsonr(x1_list,x2_list)

(-0.29113778573749477, 2.5899115476937018e-06)
```

从结果看，沪深 300 指数与创业板指数存在线性关系，呈负弱相关。

第三篇：10 分钟学会用 Python 做线性回归

导语：线性回归作为数据分析的一项基本操作，是非常关键的，也是初学者必须掌握的内容，针对线性回归，本篇主要阐述如果用 Python 实现线性回归。

一、理解什么是线性回归

线性回归也被称为最小二乘法回归。它的数学模型是这样的： $y = a + b \cdot x + e$

其中， a 被称为常数项或截距； b 被称为模型的回归系数或斜率； e 为误差项。 a 和 b 是模型的参数。当然，模型的参数只能从样本数据中估计出来： $y' = a' + b' \cdot x$

我们的目标是选择合适的参数，让这一线性模型最好地拟合观测值。拟合程度越高，模型越好。

那么，接下来的问题就是，我们如何判断拟合的质量呢？

高斯和勒让德找到的方法是：被选择的参数，应该使算出来的回归线与观测值之差的平方和最小。用函数表示为：

$$\min_{\vec{b}} \sum_{i=1}^n (y_m - y_i)^2.$$

这就是通常我们说的最小二乘法,其原理为：当预测值和实际值距离的平方和最小时，就选定模型中的两个参数 a 、 b 。

python 的 statsmodels 包已经有进行最小二乘法的函数,我们只要会用就行,想详细了解其数学推导过程的同学可以自己翻阅计量经济学相关书籍。

二、利用 statsmodels 进行线性回归

首先我们创造一条线

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

# 创找一系列从 0 到 10 等步长的 array，共 100 个值
x = np.linspace(0, 10, 100)
x[:20]

array([ 0.    ,  0.1010101,  0.2020202,  0.3030303,  0.4040404,
        0.5050505,  0.6060606,  0.7070707,  0.8080808,  0.9090909,
        1.0101010,  1.1111111,  1.2121212,  1.3131313,  1.4141414,
        1.5151515,  1.6161616,  1.7171717,  1.8181818,  1.9191919])
```

为 x 增加一列 1 的常数项

```
X = sm.add_constant(x)
```

```
X[:10]
```

```
array([[ 1.      ,  0.      ],
       [ 1.      ,  0.1010101 ],
       [ 1.      ,  0.2020202 ],
       [ 1.      ,  0.3030303 ],
       [ 1.      ,  0.4040404 ],
       [ 1.      ,  0.5050505 ],
       [ 1.      ,  0.6060606 ],
       [ 1.      ,  0.7070707 ],
       [ 1.      ,  0.8080808 ],
       [ 1.      ,  0.9090909 ]])
```

然后设置模型里的 a 和 b,这里我们设置成 2 和 8

```
beta = np.array([2, 8])
```

用 np.random.normal 生成一个长度为 100 的随机数作为误差项,数据服从正态分布。

```
e = np.random.normal(size=100)
```

```
e[:10]
```

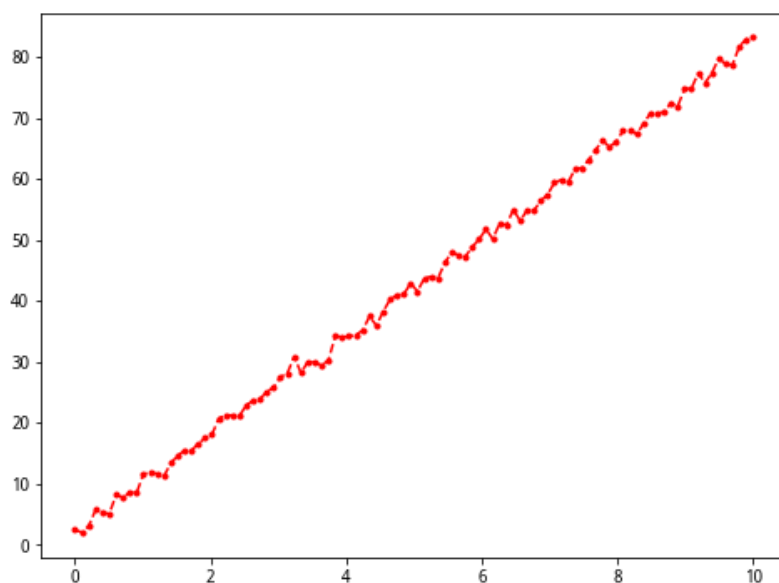
```
array([ 0.5006727, -0.74937251, -0.48312831,  1.33287776,  0.14562751,
       -0.93530541,  1.4053712,  0.139276,  0.19147624, -0.6911677 ])
```

生成对应的 y 值,并画出曲线

```
y = np.dot(X,beta) + e
```

```
plt.plot(x,y,'r--.',label='data')
```

```
plt.show()
```



仅知道 x,y 情况下利用 statsmodels 进行 OLS 回归,计算系数。

```
model = sm.OLS(y,X)
```

获得拟合结果。

```
results = model.fit()
```

打印计算到的的系数

```
print(results.params)
```

```
[ 2.21932989  8.00854553]
```

打印更详细的回归数据,包括对各参数的 t 检验,R-squared,AIC,BIC 等

```
results.summary()
```

OLS Regression Results

Dep. Variable:	y	R-squared:	0.998
Model:	OLS	Adj. R-squared:	0.998
Method:	Least Squares	F-statistic:	6.326e+04
Date:	Wed, 07 Feb 2018	Prob (F-statistic):	1.54e-139
Time:	09:24:00	Log-Likelihood:	-133.45
No. Observations:	100	AIC:	270.9
Df Residuals:	98	BIC:	276.1
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	2.2193	0.184	12.042	0.000	1.854	2.585
x1	8.0085	0.032	251.521	0.000	7.945	8.072

Omnibus:	0.247	Durbin-Watson:	1.903
Prob(Omnibus):	0.884	Jarque-Bera (JB):	0.361
Skew:	0.109	Prob(JB):	0.835
Kurtosis:	2.802	Cond. No.	11.7

取出其中的具体数据方法为：

输入 results. 再按 tab

```
print(results.aic)
print(results.bic)
print(results.fvalue)
```

```
270.907413167
276.117753539
63262.8116348
```

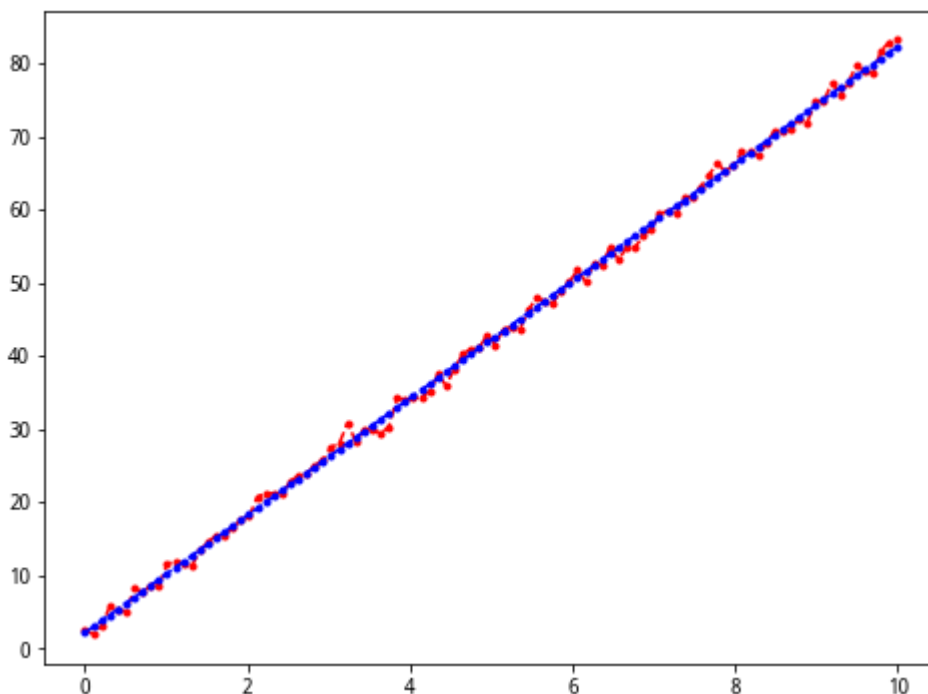
获得拟合数据

```
y_predict = results.predict()
y_predict[:10]
```

```
array([ 2.21932989,  3.02827388,  3.83721787,  4.64616187,  5.45510586,
        6.26404985,  7.07299384,  7.88193784,  8.69088183,  9.49982582])
```

画出原曲线和拟合曲线

```
plt.subplots(figsize=(8,6))
plt.plot(x,y,'r-.',label='data')
plt.plot(x,y_predict,'b--.',label='OLS')
plt.show()
```



应用:利用 OLS 计算股票历史 Beta

首先获得上证指数一年的数据,并计算每日涨幅。

```
value1 = get_price('000001.SH',None,'20160105','1d',['close'],True,'pre',240)
value1['roc1'] = value1.close/value1.close.shift(1) - 1.0
value1[:3]
```

	close	roc1
2015-01-13	3235.30	NaN
2015-01-14	3222.44	-0.003975
2015-01-15	3336.46	0.035383

获得天津港 600717.SH 一年的数据,并计算每日涨幅。

```
value2 = get_price('600717.SH',None,'20160105','1d',['close'],True,'pre',240)
value2['roc2'] = value2.close/value2.close.shift(1) - 1.0
value2[:3]
```

	close	roc2
2015-01-13	16.65	NaN
2015-01-14	16.31	-0.020420
2015-01-15	16.83	0.031882

将两数据横向合并,删除空数据。

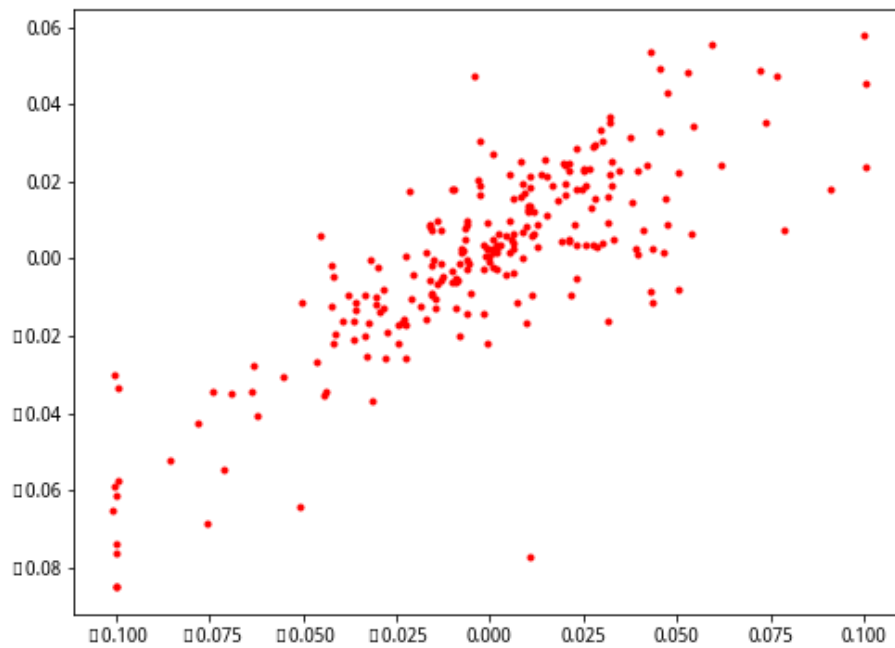
```
value3 = pd.concat([value1,value2],axis=1)
value3 = value3.dropna() #删掉 nan 数据
value3[:3]
```

	close	roc1	close	roc2
2015-01-14	3222.44	-0.003975	16.31	-0.020420
2015-01-15	3336.46	0.035383	16.83	0.031882
2015-01-16	3376.50	0.012001	17.00	0.010101

以天津港 600717.SH 每日涨幅为横轴,上证指数每日涨幅为纵轴,画图。

```
y = value3.roc1.values
x = value3.roc2.values
```

```
plt.subplots(figsize=(8,6))
plt.plot(x,y,'r.',label='data')
plt.show()
```

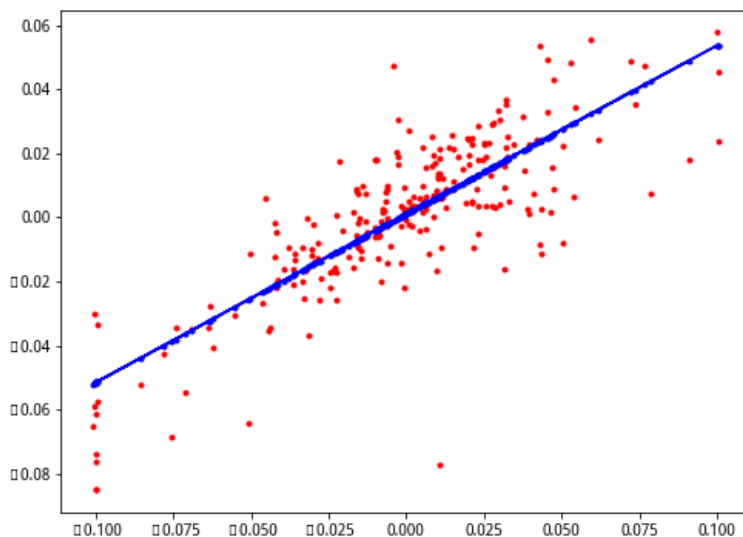


用 statsmodels 包进行拟合,并画图。

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
```

```
X = sm.add_constant(x)
model = sm.OLS(y,X)
results = model.fit()
y_predict = results.predict()
```

```
plt.subplots(figsize=(8,6))
plt.plot(x,y,'r.',label='data')
plt.plot(x,y_predict,'b.-',label='data')
plt.show()
```



计算的到的 k 值即股票相对大盘的 Beta 值

```
print('天津港 600717.SH 相对上证指数的 Beta 值为:'+str(results.params[1]))
```

```
天津港 600717.SH 相对上证指数的 Beta 值为:0.526072480204
```

第四篇：统计套利：利用相关系数进行配对交易

导语：本篇内容你将会学会如何运用相关系数找出适合套利的股票标的，并编写套利策略，进行套利交易。

什么是配对交易？

配对交易是指八十年代中期华尔街著名投行 Morgan Stanley 的数量交易员 Nunzio Tartaglia 成立的一个数量分析团队提出的一种市场中性投资策略。

Ganapathy Vidyamurthy 在《Pairs Trading: Quantitative Methods and Analysis》一书中定义配对交易为两种类型：一类是基于统计套利的配对交易，一类是基于风险套利的配对交易。

基于统计套利的配对交易策略是一种市场中性策略，具体的说，是指从市场上找出历史股价走势相近的股票进行配对，当配对的股票价格差（Spreads）偏离历史均值时，则做空股价较高的股票同时买进股价较低的股票，等待他们回归到长期均衡关系，由此赚取两股票价格收敛的报酬。

配对交易原理

配对交易策略的基本原理是基于两个相关性较高的股票或者其他证券，如果在未来时期保持着良好的相关性，一旦两者之间出现了背离的走势，且这种背离在未来是会得到纠正的，那么就可能产生套利的机会。对于配对交易的实践而言，如果两个相关性较高的股票或者其他证券之间出现背离，就应该买进表现相对较差的，卖出表现相对较好的。当未来两者之间的背离得到纠正，那么可以进行相反的平仓操作来获取利润。

由于配对交易利用配对间的短期错误定价，通过持有相对低估，卖空相对高估，因此其本质上是一个反转投资策略，其核心是学术文献中的股票价格均值回复。尽管配对交易策略非常简单，但却被广泛应用，其之所以能被广泛应用的主要原因是：首先，配对交易的收益与市场相独立，即市场中性，也就是说它与市场的上涨或者下跌无关；其次，其收益的波动性相对较小；第三，其收益相对稳定。

统计套利：利用相关系数进行配对交易

1. 首先想到利用统计套利，可能会想到两只股票的相关系数是否会让两只股票的走势有一种特定关系。

导入相应的 python 库包

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

首先相关系数的公式定义为下，将协方差标准化即得到我们需要的相关系数

$$r = \frac{\text{Cov}(X, Y)}{\text{std}(X)\text{std}(Y)} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{var}(X)}\sqrt{\text{var}(Y)}} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Cov}(X, X)}\sqrt{\text{Cov}(Y, Y)}}$$

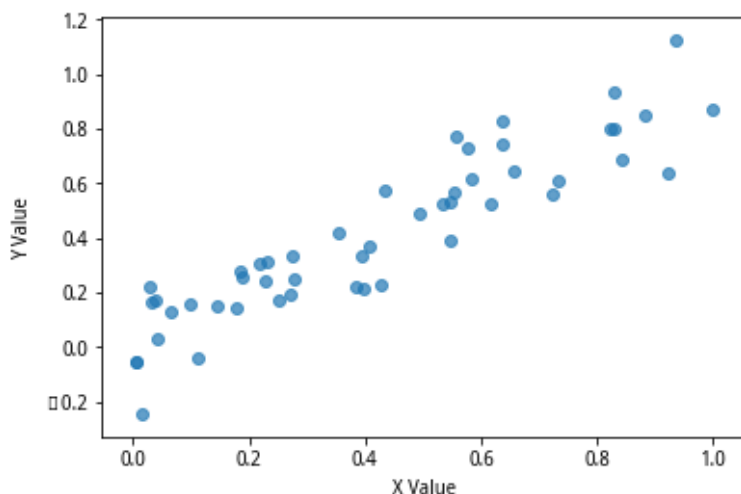
我们来看看相关系数高的数据集具体长什么样

```
X = np.random.rand(50)
Y = X + np.random.normal(0, 0.1, 50)

plt.scatter(X, Y, alpha=0.7)
plt.xlabel('X Value')
plt.ylabel('Y Value')

print('相关系数: ' + str(np.corrcoef(X, Y)[0, 1]))

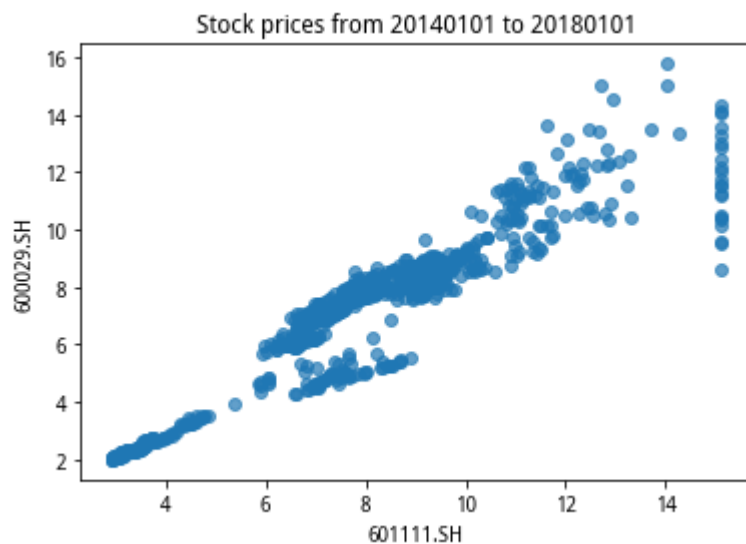
相关系数: 0.916457598007
```



从图像上看，数据基本都落在一条直线上那么它们之间的相关性就会很高。接下来我们来看看两只股票价格之间相关性高长什么样，我们以中国航空和南方航空为例。

```
start = '20140101'#此处时间一定要与回测的时间相对应，因为不同时间可能相关性不一致
end = '20180101'
stock1='601111.SH'
stock2='600029.SH'
a1 = get_price(stock1, start, end, '1d', ['close'], False)['close']
a2 = get_price(stock2, start, end, '1d', ['close'], False)['close']
#上图
plt.scatter(a1,a2,alpha=0.7)
plt.xlabel(stock1)
plt.ylabel(stock2)
plt.title('Stock prices from ' + start + ' to ' + end)
print(stock1+"与"+stock2+"之间的相关系数: ", np.corrcoef(a1,a2)[0,1])

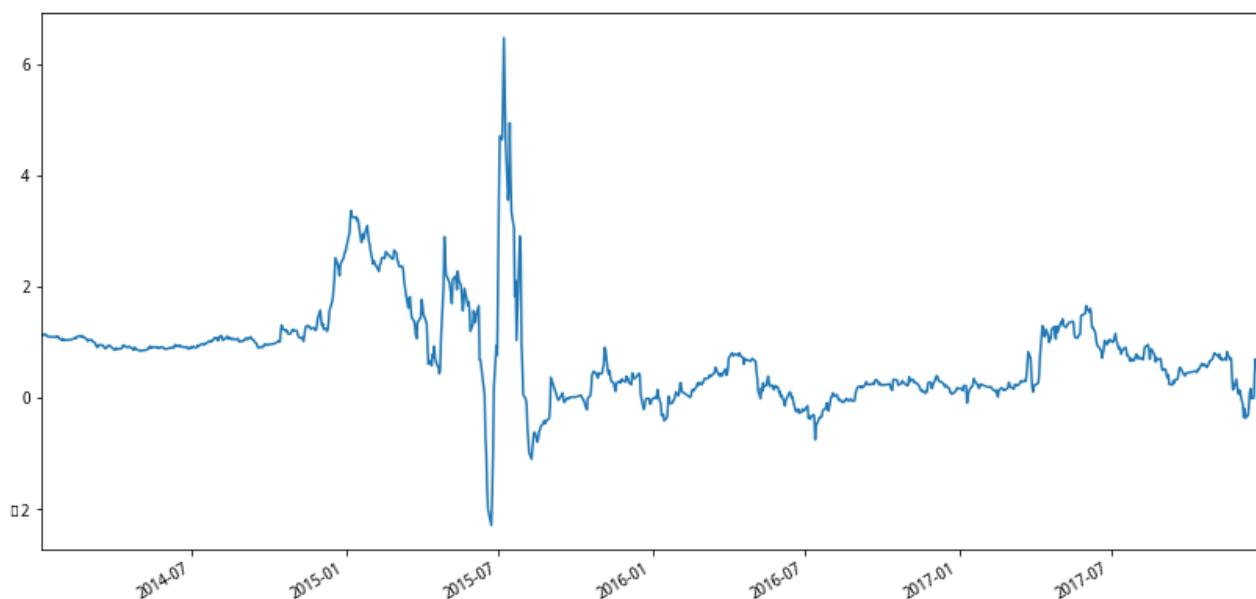
601111.SH 与 600029.SH 之间的相关系数: 0.952849080194
```



同样数据大多都集中在一条直线上

找到相关性高的股票对，我们要来研究它们之间的价差，因为这是我们策略套利的关键

```
a3=a1-a2
a3.plot(figsize=(14,7))
```



从图中看出，两只股票的相关系数高，两者之间的价差大致围绕一个常数波动，那么其价差是否平稳，我们通过 ADF 单位根检验，来判断一下序列是否平稳。

```
from statsmodels.tsa.stattools import adfuller
adftest = adfuller(a3)#使用 adf 单位根检验平稳性
result = pd.Series(adftest[0:4], index=["Test Statistic",'p-value','Lags Used','Number of Observations
Used'])
for key,value in adftest[4].items():
    result['Critical Value (%s)'%key] = value
print(result)
```

Test Statistic	-3.049276
p-value	0.030542
Lags Used	22.000000
Number of Observations Used	954.000000
Critical Value (5%)	-2.864574
Critical Value (10%)	-2.568386
Critical Value (1%)	-3.437223

dtype: float64

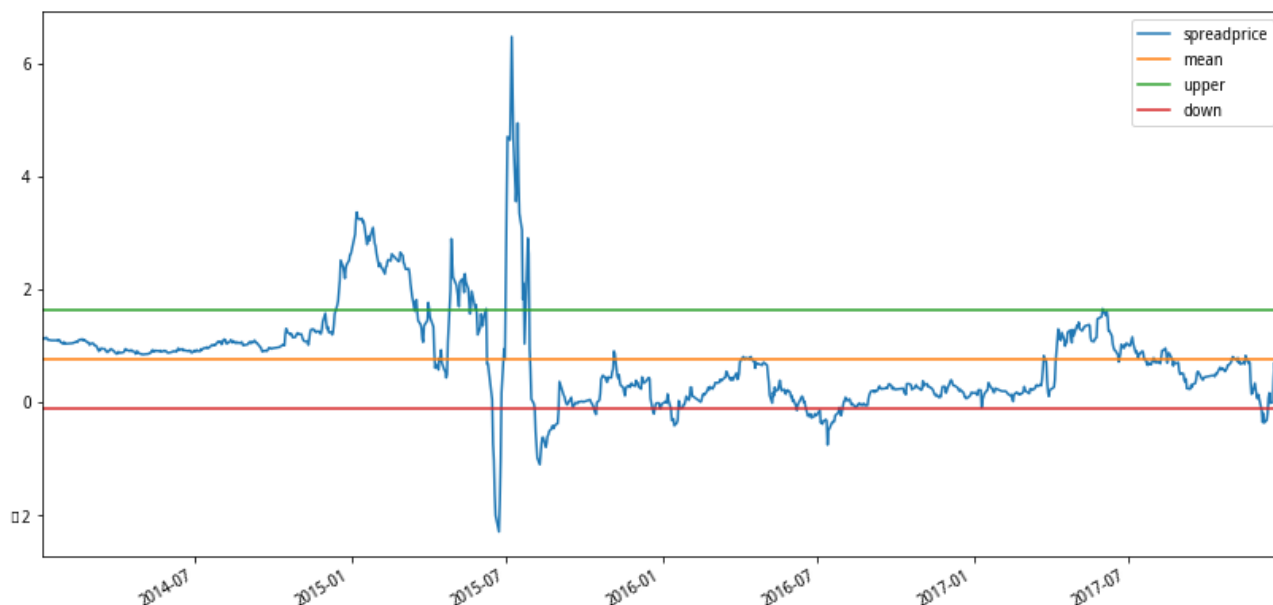
根据 ADF 单位根检验结果，此处检验出来价差平稳，价差基础对套利策略的实施有利。

ADF 单位根检验结果判断技巧：

A.看 p-value 值，一般小于 0.05 即为平稳

B.看 Test statistic 值，一般小于 Critical Value (5%)的值即为平稳

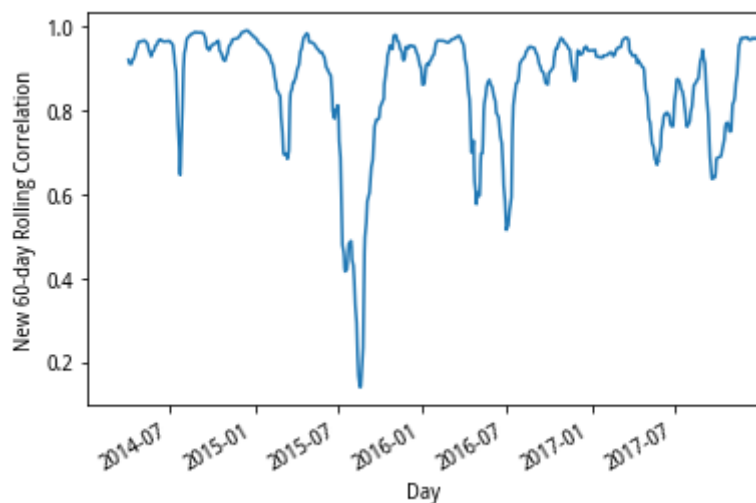
进一步的我们来看看价差均值基础上加减一个价差标准差作为套利上下轨，上下轨区间是否包含了大部分的差价区间？



可以看到上下轨区间包含了大部分价差区间，但是在牛市中，两者价差波动非常大，不利于套利策略实施，但是在牛市后，两者的价差稳定在上下轨之间波动，非常适合进行套利策略。

让我们计算两者 60 日滚动相关性，来看看两者变化趋势。

```
rolling_correlation_cn = pd.rolling_corr(a1, a2, 60)
rolling_correlation_cn.plot()
plt.xlabel('Day')
plt.ylabel('New 60-day Rolling Correlation')
```



从相关系数的 60 日滚动相关性来看其实这两只股票相关性是非常高的，除了牛市中相关性降低一段时间外，其余时间都保持在 0.7 以上。

通过简单的套利策略，针对中国国航和南方航空，我们展开套利回测，回测结果如下：



套利策略代码如下：

```
import numpy as np
```

```
def initialize(account):
```

```
    #选取研究后相关系数较高的股票对
```

```
    g.s1 = '601111.SH'
```

```
    g.s2 = '600029.SH'
```

```
def handle_data(account,data):
```

```
    #取得选取的两只股票 1 年的收盘价并算出差价
```

```
    price_stock1=history(g.s1,['close'],250,'1d',True)['close'].values
```

```
    price_stock2=history(g.s2,['close'],250,'1d',True)['close'].values
```

```
    diff=price_stock1-price_stock2
```

```
    #以均值加一倍标准差作为上开仓线
```

```
    up=np.mean(diff)+np.std(diff)
```

```
    #以均值减一倍标准差作为下开仓线
```

```
    down=np.mean(diff)-np.std(diff)
```

```
    #取得进行交易前一天股票的收盘价并计算差价
```

```
    yesterday_price1=history(g.s1,['close'],1,'1d',True)['close'][0]
```

```
    yesterday_price2=history(g.s2,['close'],1,'1d',True)['close'][0]
```

```
    yesterday_diff=yesterday_price1-yesterday_price2
```

```
    #前一天差价达到上开仓线即卖出股票 s1，做多 s2
```

```
    if yesterday_diff>up:
```

```
        order_target_percent(g.s1,0)
```

```
        order_target_percent(g.s2,1)
```

```
    #前一天的差价达到下开仓线即卖出股票 s2，做多 s1
```

```
    if yesterday_diff<down:
```

```
        order_target_percent(g.s2,0)
```

```
        order_target_percent(g.s1,1)
```

第五篇：数据处理专题：去极值、标准化、中性化

导语：一般的数据预处理中常提及到三类处理：去极值、标准化、中性化。我们将向大家讲述这常见的三种数据处理操作。

一、去极值

在分析上市公司当季净利润同比增长率数据时，我们往往会被其中一些公司的数据干扰，如图中江西长运，2017 三季度净利润同比增长率高达 32836.04%！而实际上大部分公司的当季净利润同比增长率的数值都远远达到这个值的百分之一。那么数据去极值操作就显得尤为关键，可以剔除掉数据干扰项，提高数据结论的准确性。

序号	股票代码	股票简称	现价(元)	涨跌幅(%)	净利润同比增长率(%) ?						
					2017.12.31	2017.09.30 ↓	2017.06.30	2017.03.31	2016.12.31	2015.12.31	2014.12.31
1	600561	江西长运	7.20	0.98	--	32,836.04	19,148.32	-144.38	-226.58	-48.17	9.72
2	600710	ST常林	5.47	-2.50	--	11,581.35	2,058.01	896.34	178.92	-192.44	16.66
3	603799	华友钴业	87.90	-0.88	--	9,723.35	2,257.15	820.97	128.15	-269.22	18.51
4	000011	深物业A	15.05	-1.31	--	8,099.55	7,041.82	52,167.63	126.28	-62.44	38.78
5	002068	黑猫股份	8.63	-2.60	--	7,728.76	477.04	181.25	502.61	-82.47	275.50
6	002529	海源机械	15.09	-0.26	--	7,238.52	232.98	229.58	-1,460.48	-13.26	-63.06
7	000877	天山股份	10.59	2.12	--	7,151.34	124.56	29.72	119.00	-307.95	-10.54
8	600231	凌钢股份	5.09	0.79	--	7,120.69	608.08	227.34	181.25	106.68	-1,032.64
9	603003	龙宇燃油	10.30	0.98	--	6,807.66	487.05	225.64	117.06	-2,269.29	114.24

一般去极值的处理方法就是确定该项指标的上下限，然后超过或者低于限值的数据统统即为限值。其中上下限数值判断标准有三种，分别为 MAD、 3σ 、百分位法。

以沪深 300 成分股的 pe 值为原始数据，向大家阐述 MAD、 3σ 、百分位法。

```
import numpy as np
import pandas as pd
import math
from statsmodels import regression
import statsmodels.api as sm
import matplotlib.pyplot as plt

date='20180125'
stock=get_index_stocks('000300.SH',date)
q = query(
    valuation.symbol,
    valuation.pe_ttm,
    valuation.current_market_cap
).filter(valuation.symbol.in_(stock))
data = get_fundamentals(q,date=date)
data.columns = [['symbol','pe_ratio','market_cap']]
data = data.set_index(data.symbol.values)
del data['symbol']
data['1/PE'] = 1/data['pe_ratio']
data.head()

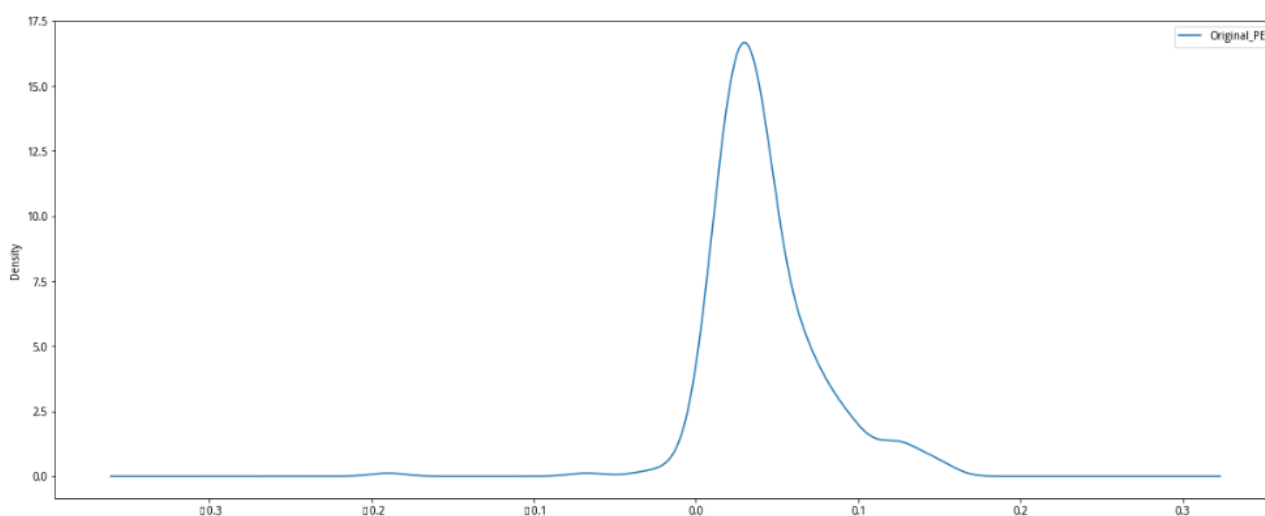
pe_ratio  market_cap  1/PE
```

000001.SZ	10.59	2.402355e+11	0.094429
000002.SZ	18.61	3.903084e+11	0.053735
000008.SZ	54.53	1.661121e+10	0.018339
000060.SZ	27.51	2.613508e+10	0.036350
000063.SZ	-115.27	1.237605e+11	-0.008675

将 1/PE 的数据分布，运用绘图函数展示出来：

```
fig = plt.figure(figsize = (20, 8))
ax = data['1/PE'].plot.kde(label = 'Original_PE')
ax.legend()
```

下图是 20180125 的沪深 300 指数成分股 1/PE 的数据分布：



1. MAD 法：

MAD 又称为绝对值差中位数法，是一种先需计算所有因子与平均值之间的距离总和来检测离群值的方法，处理的逻辑：

第一步，找出所有因子的中位数 X_{median}

第二步：得到每个因子与中位数的绝对偏差值 $X_i - X_{median}$

第三步：得到绝对偏差值的中位数 MAD

第四步：确定参数 n ，从而确定合理的范围为 $[X_{median} - nMAD, X_{median} + nMAD]$ ，并针对超出合理范围的因子值做如下的调整：

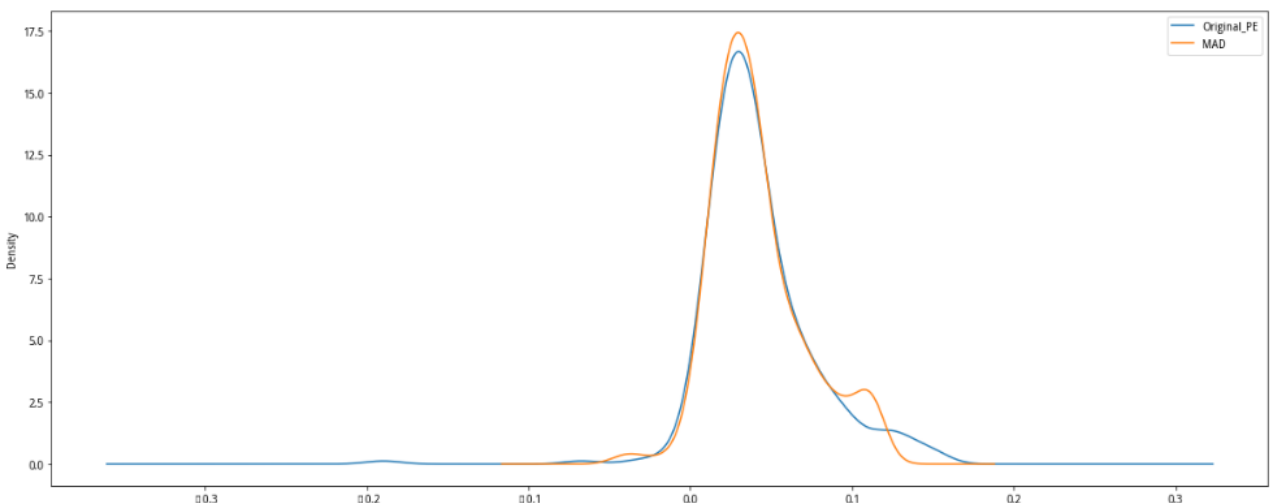
$$X'_i = \begin{cases} X_{median} + nMAD & \text{if } X_i > X_{median} + nMAD \\ X_{median} - nMAD & \text{if } X_i < X_{median} - nMAD \\ X_i & \text{if } X_{median} - nMAD < X_i < X_{median} + nMAD \end{cases}$$

MAD 法代码实现：

```
def filter_extreme_MAD(series,n): #MAD:中位数去极值
    median = series.quantile(0.5)
    new_median = ((series - median).abs()).quantile(0.50)
    max_range = median + n*new_median
    min_range = median - n*new_median
    return np.clip(series,min_range,max_range)
```

对原始数据进行 MAD 处理后的结果：

```
fig = plt.figure(figsize = (20, 8))
ax = data['1/PE'].plot.kde(label = 'Original_PE')
ax = filter_extreme_MAD(data['1/PE'],5).plot.kde(label = 'MAD')
ax.legend()
```



2. 3σ 法

3σ 法又称为标准差法。标准差本身可以体现因子的离散程度，是基于因子的平均值 X_{mean} 而定的。在离群值处理过程中，可通过用 $X_{mean} \pm n\sigma$ 来衡量因子与平均值的距离。

标准差法处理的逻辑与 MAD 法类似：

第一步：计算出因子的平均值与标准差

第二步：确认参数 n （这里选定 $n = 3$ ）

第三步：确认因子值的合理范围为 $[X_{mean} - n\sigma, X_{mean} + n\sigma]$ ，并对因子值作如下的调整：

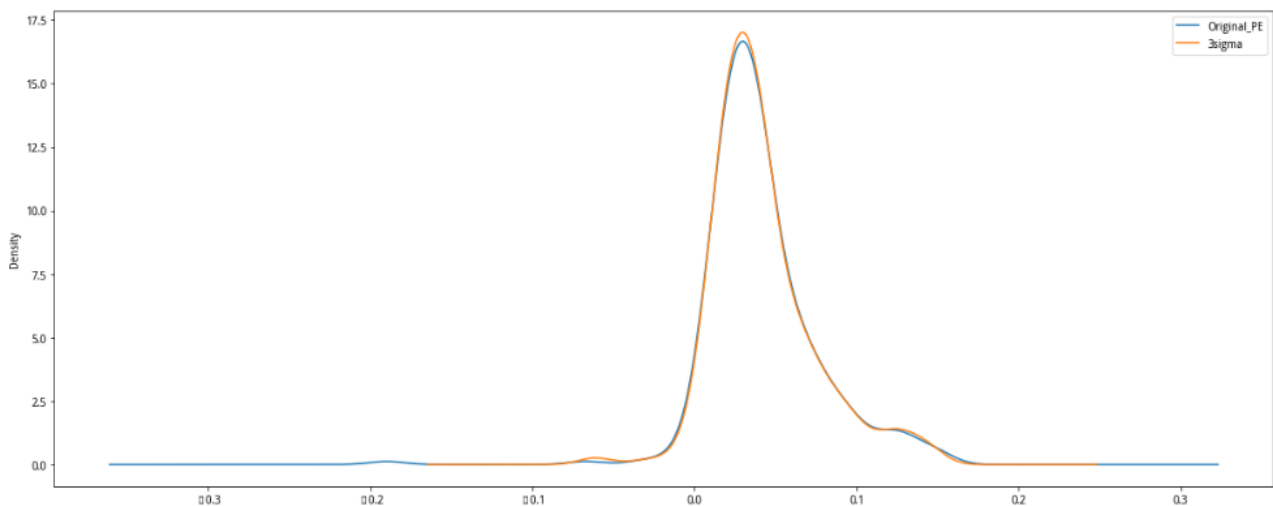
$$X'_i = \begin{cases} X_{mean} + n\sigma & \text{if } X_i > X_{mean} + n\sigma \\ X_{mean} - n\sigma & \text{if } X_i < X_{mean} - n\sigma \\ X_i & \text{if } X_{mean} - n\sigma < X_i < X_{mean} + n\sigma \end{cases}$$

3 σ 代码实现:

```
def filter_extreme_3sigma(series,n=3): #3 sigma
    mean = series.mean()
    std = series.std()
    max_range = mean + n*std
    min_range = mean - n*std
    return np.clip(series,min_range,max_range)
```

对原始数据进行 3 σ 处理后的结果:

```
fig = plt.figure(figsize = (20, 8))
ax = data['1/PE'].plot.kde(label = 'Original_PE')
ax = filter_extreme_3sigma(data['1/PE']).plot.kde(label = '3sigma')
ax.legend()
```



3. 百分位法:

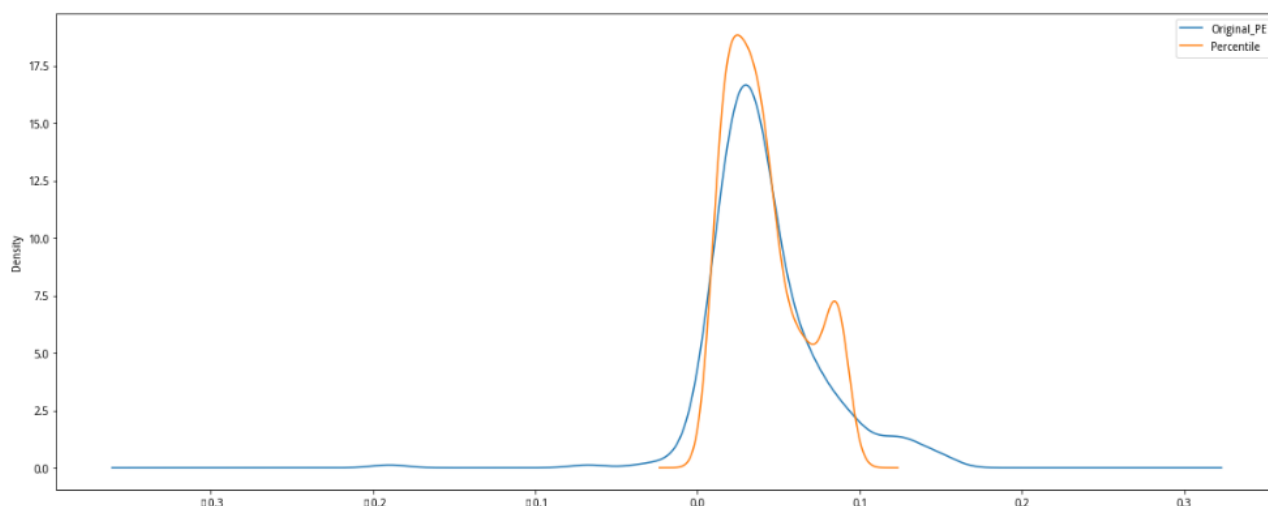
将因子值进行升序的排序, 对排位百分位高于 97.5%或排位百分位低于 2.5%的因子值, 进行类似于 MAD、3 σ 的方法进行调整。

百分位法代码实现:

```
def filter_extreme_percentile(series,min = 0.10,max = 0.90): #百分位法
    series = series.sort_values()
    q = series.quantile([min,max])
    return np.clip(series,q.iloc[0],q.iloc[1])
```

```
fig = plt.figure(figsize = (20, 8))
ax = data['1/PE'].plot.kde(label = 'Original_PE')
ax = filter_extreme_percentile(data['1/PE']).plot.kde(label = 'Percentile')
ax.legend()
```

对原始数据进行百分位法处理后的结果：



二、标准化

继续深入的做股票数据分析，假设我们需买入当季净利润同比增长率较高和股息率较高的股票，如果单纯的将两个指标相加，然后取最终值较大的那些股票，那么会面临一个非常严重的问题：两个指标的数值意义不同。上市公司达到股息率 10% 难于净利润同比增长率 10%，一般股息率都小于 5%，而净利润增长率则远远超过 5% 的数值，因此简单的相加，然后选股会淡化股息率指标。

那么这种情况下，数据标准化处理可以解决问题。

标准化在统计学中有一系列含义，一般使用 z-score 的方法。处理后的数据从有量纲转化为无量纲，使得不同的指标能够进行比较和回归。

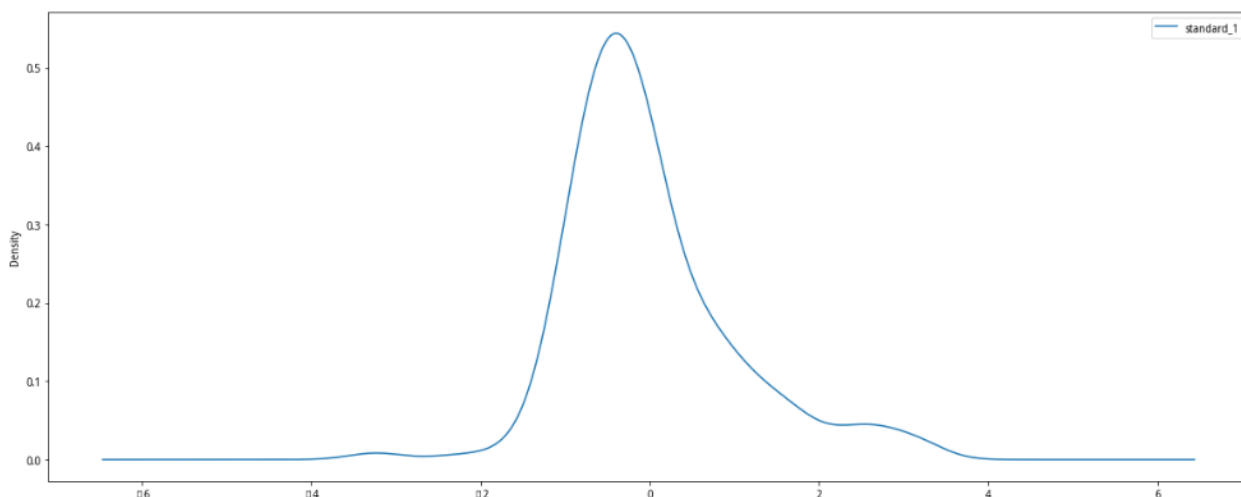
z-score 的方法介绍

标准化后的数值 = (原始值-单个指标内所有值的均值)/单个指标内所有值的标准差

z-score 的方法代码实现：

```
def standardize_series(series): #原始值法
    std = series.std()
    mean = series.mean()
    return (series-mean)/std
fig = plt.figure(figsize = (20, 8))
new = filter_extreme_3sigma(data['1/PE'])
ax = standardize_series(new).plot.kde(label = 'standard_1')
ax.legend()
```

对原始数据进行 Z-score 法处理后的结果：



局部数据展示：

```
standard_2 = standardize_series(new.rank())
standard_2.head()
```

```
000001.SZ    1.458269
000002.SZ    0.847295
000008.SZ   -1.135490
000060.SZ    0.028820
000063.SZ   -1.677298
Name: 1/PE, dtype: float64
```

三、中性化

继续深入的做股票数据分析，你可能会发现银行股的市盈率特别的低，而互联网行业的市盈率就特别的高，如果直接用市盈率指标，无论你做了标准化还是去极值，你只会挑选出银行股，而不可能挑选出互联网股，那么互联网中市盈率最低的股票和银行股中市盈率最高的股票，应该如何体现它们的真实意义呢？

这种情况下你就需要进行数据中性化处理。

中性化处理概念：为了在用某一因子时能剔除其他因素的影响，使得选出的股票更加分散。标准化用于多个不同量级指标之间需要互相比对或者数据需要变得集中，而中性化的目的在于消除因子中的偏差和不需要的因素。

具体方法：根据大部分的研报对于中性化的处理，主要的方法是利用回归得到一个与风险因子线性无关的因子，即通过建立线性回归，提取残差作为中性化后的新因子。这样处理后的中性化因子与风险因子之间的相关性严格为零。

$$Factor_i = \beta_M * \ln(MktVal_i) + \sum_{j=1}^n \beta_j * Industry_{j,i} + \varepsilon_i$$

用 python 实现中性化: 对原始数据进行 3σ 处理后的结果, 进行中性化处理。

数据获取

```
import numpy as np
import pandas as pd
import math
from statsmodels import regression
import statsmodels.api as sm
import matplotlib.pyplot as plt

date='20180125'
stock=get_index_stocks('000300.SH',date)
q = query(
    valuation.symbol,
    valuation.pe_ttm,
    valuation.current_market_cap
).filter(valuation.symbol.in_(stock))
data = get_fundamentals(q,date=date)
data.columns = [['symbol','pe_ratio','market_cap']]
data = data.set_index(data.symbol.values)
del data['symbol']
data['1/PE'] = 1/data['pe_ratio']
data.head()
```

	pe_ratio	market_cap	1/PE
000001.SZ	10.59	2.402355e+11	0.094429
000002.SZ	18.61	3.903084e+11	0.053735
000008.SZ	54.53	1.661121e+10	0.018339
000060.SZ	27.51	2.613508e+10	0.036350
000063.SZ	-115.27	1.237605e+11	-0.008675

3σ 处理处理函数

```
def filter_extreme_3sigma(series,n=3): #3 sigma
    mean = series.mean()
    std = series.std()
    max_range = mean + n*std
    min_range = mean - n*std
    return np.clip(series,min_range,max_range)
```

行业代码列表

```
SHENWAN_INDUSTRY_MAP = {
    'S11': '农林牧渔',
    'S21': '采掘',
    'S22': '化工',
    'S23': '钢铁',
    'S24': '有色金属',
    'S27': '电子',
    'S28': '汽车',
    'S33': '家用电器',
    'S34': '食品饮料',
    'S35': '纺织服装',
```

```

'S36': '轻工制造',
'S37': '医药生物',
'S41': '公用事业',
'S42': '交通运输',
'S43': '房地产',
'S45': '商业贸易',
'S46': '休闲服务',
'S48': '银行',
'S49': '非银金融',
'S51': '综合',
'S61': '建筑材料',
'S62': '建筑装饰',
'S63': '电气设备',
'S64': '机械设备',
'S65': '国防军工',
'S71': '计算机',
'S72': '传媒',
'S73': '通信'
}

```

行业代码标记

```

def get_industry_exposure(order_book_ids):
    df = pd.DataFrame(index=SHENWAN_INDUSTRY_MAP.keys(), columns=order_book_ids)
    for stk in order_book_ids:
        try:
            df[stk][get_symbol_industry(stk).s_industryid1] = 1
        except:
            continue
    return df.fillna(0)#将 NaN 赋为 0

```

中性化函数

```

# 需要传入单个因子值和总市值
def neutralization(factor,mkt_cap = False, industry = True):
    y = factor
    if type(mkt_cap) == pd.Series:
        LnMktCap = mkt_cap.apply(lambda x:math.log(x))
        if industry: #行业、市值
            dummy_industry = get_industry_exposure(factor.index)
            x = pd.concat([LnMktCap,dummy_industry.T],axis = 1)
        else: #仅市值
            x = LnMktCap
    elif industry: #仅行业
        dummy_industry = get_industry_exposure(factor.index)
        x = dummy_industry.T
    result = sm.OLS(y.astype(float),x.astype(float)).fit()
    return result.resid

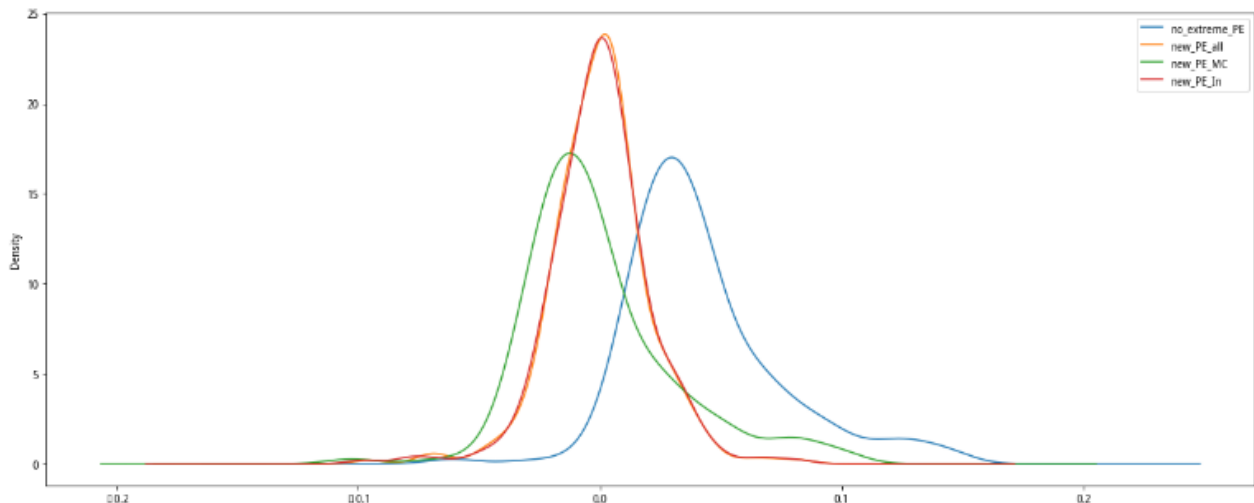
```

函数调用

```
#使用 3sigma 离群值处理法
no_extreme_PE = filter_extreme_3sigma(data['1/PE'])
#行业市值中性
new_PE_all = neutralization(no_extreme_PE, data['market_cap'])
#市值中性
new_PE_MC = neutralization(no_extreme_PE, data['market_cap'], industry = False)
#行业中性
new_PE_In = neutralization(no_extreme_PE)
```

```
fig = plt.figure(figsize = (20, 8))
ax = no_extreme_PE.plot.kde(label = 'no_extreme_PE')
ax = new_PE_all.plot.kde(label = 'new_PE_all')
ax = new_PE_MC.plot.kde(label = 'new_PE_MC')
ax = new_PE_In.plot.kde(label = 'new_PE_In')
ax.legend()
```

结果输出:能够看出市值和行业中性化后的 PE 分布变得更加均匀, 其中行业中性化比市值中性化的效果更好。



第六篇：数据挖掘专题：分类与预测

导语：数据挖掘，又译为数据采矿，是指从大量的数据中通过算法搜索隐藏于其中信息的过程。本篇内容主要向大家讲述如何使用 KNN 算法进行数据分类和数据预测。

基础概念

数据分类就是相同内容、相同性质的信息以及要求统一管理的信息集合在一起，而把相异的和需要分别管理的信息区分开来，然后确定各个集合之间的关系，形成一个有条理的分类系统。

举个最简单的例子：我们定义 K 线为三类：“上涨”：涨幅超过 1%，“下跌”：跌幅超过 1%，“震荡”涨跌幅不超过 1%，获取沪深 300 指数过去 250 个交易日的 K 线，将数据进行分类：

```
p=get_price('000300.SH', None, '20180125', '1d', ['quote_rate'], True, None, 250, is_panel=1)
n1=len(p[p['quote_rate']>1])
n2=len(p[p['quote_rate']<-1])
n3=len(p)-n1-n2
print('上涨 K 线： {}, 下跌 K 线： {}, 震荡 K 线： {}'.format(n1,n2,n3))
```

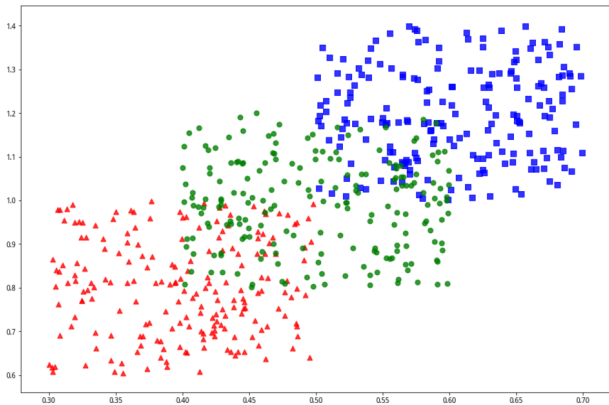
上涨 K 线： 18，下跌 K 线： 12，震荡 K 线： 220

数据预测即用数据分类得出的模型对未知变量的预言。预言其目的是对未来未知变量的预测。

假设我们用历史数据发现上涨股票平均比例，所有个股平均量比，这二个指标可以用来定义当天市场是上涨、下跌还是震荡。其特征如下：

市场	上涨股票平均比例均值	所有个股平均量化均值
上涨（沪深 300 涨幅超过 1%）	60%	1.2
下跌（沪深 300 跌幅超过 1%）	40%	0.8
震荡（沪深 300 涨跌幅不超过 1%）	50%	1

详细数据分布如下：



在我们完成分类模型后的下一个交易日，只给你们二个指标的具体数值：上涨股票比例为 55%，所有个股平均量比 1。请问当天的沪深 300 指数是上涨、下跌还是震荡。

KNN 算法

我们需要解决上述分类和预测的问题，就需要依赖于数据挖掘的算法，本篇文章主要介绍 KNN 算法，一个常用又简单的数据分类和预测算法。

KNN 算法核心函数：`neighbors.KNeighborsClassifier(n_neighbors=5, weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=1)`

函数内 `n_neighbors` 参数是做分类时选取参考的数据量，默认为 5 个。其余参数选取为默认，课外自行了解原理。

开始之前，我们先导入 KNN 算法模块

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import neighbors
```

回到之前的详细数据分布

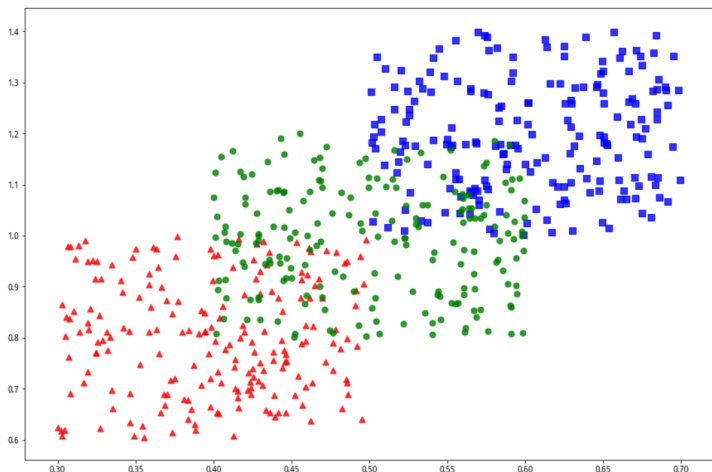
```
fig_size = plt.rcParams["figure.figsize"]
fig_size[0] = 15
fig_size[1] = 10
x1 = np.random.uniform(0.5, 0.7, 200)
y1 = np.random.uniform(1.4, 1, 200)

plt.scatter(x1, y1, c='b', marker='s', s=50, alpha=0.8)

x2 = np.random.uniform(0.3, 0.5, 200)
y2 = np.random.uniform(0.6, 1, 200)
plt.scatter(x2, y2, c='r', marker='^', s=50, alpha=0.8)

x3 = np.random.uniform(0.4, 0.6, 200)
```

```
y3 = np.random.uniform(0.8, 1.2, 200)
plt.scatter(x3,y3, c='g', s=50, alpha=0.8)
```



在创建了一个 `KNeighborsClassifier` 类之后，需要给它数据来进行学习并分类。需要使用 `fit()` 拟合功能：`neighbors.KNeighborsClassifier.fit(X,y)`

`X` 是一个 `list` 或 `array` 的数据，每一组数据可以是 `tuple` 也可以是 `list` 或者一维 `array`，但要注意所有数据的长度必须一样。当然，也可以把 `X` 理解为一个矩阵，其中每一横行是一个样本的特征数据。

`y` 是一个和 `X` 长度相同的 `list` 或 `array`，其中每个元素是 `X` 中相对应的数据的分类标签。

`KNeighborsClassifier` 类在对训练数据执行 `fit()` 后会根据函数内的 `algorithm` 参数，依据训练数据生成一个 `kd_tree` 或者 `ball_tree`。

其中第一类是上涨，第二类是下跌，第三类是震荡。

```
x_val = np.concatenate((x1,x2,x3))
y_val = np.concatenate((y1,y2,y3))
x_diff = max(x_val)-min(x_val)
y_diff = max(y_val)-min(y_val)
x_normalized = x_val/x_diff
y_normalized = y_val/y_diff
xy_normalized = list(zip(x_normalized,y_normalized))
labels = [1]*200+[2]*200+[3]*200
clf = neighbors.KNeighborsClassifier(30)
clf.fit(xy_normalized, labels)
```

```
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski', metric_params=None,
n_jobs=1, n_neighbors=30, p=2, weights='uniform')
```

注意：在分类时，我们对数据进行了标准化，主要是使得 X 轴和 Y 轴的数据间隔具有可比性，比如我们的上涨股票比例恒小于等于 1，而量比甚至可能大于 2，这样的话两个坐标轴没有可比性，间距不一致，因此我们需要进行标准化。

具体标准化公式：最终值=初始值/（最大值-最小值）

至此我们完成了数据分类。

让我们来尝试数据预测！

近邻数据获取

我们尝试来获取(0.5,1)这个数据点周围的 10 个数据点

```
nearests = clf.kneighbors([(0.5/x_diff, 1/y_diff)], 10, False)
nearests
```

```
array([[477, 416, 106, 553, 597, 468, 530, 76, 426, 56]])
```

结果显示为第 477,416,106,553,597,468,530,76,426,56 个数据点

数据预测：

继续回答上述问题：上涨股票比例为 55%，所有个股平均量比 1。请问当天的沪深 300 指数是上涨、下跌还是震荡。

```
prediction = clf.predict([(0.55/x_diff, 1/y_diff)])
prediction
```

```
array([1])
```

结果显示，该天沪深 300 指数属于第一类，上涨。

数据预测概率

```
prediction_proba = clf.predict_proba([(0.55/x_diff, 1/y_diff)])
prediction_proba
```

```
array([[ 0.5, 0. , 0.5]])
```

结果显示，(0.55,1)属于第一和第三类的概率为 50%。

我们再来试试(0.9,1.2)预测概率

```
prediction_proba = clf.predict_proba([(0.9/x_diff, 1.2/y_diff)])
prediction_proba
```

```
array([[ 1., 0., 0.]])
```

结果显示，(0.9,1.2)属于第一类的概率为 100%。

KNN 算法学习准确性打分

我们将上述数据分类 2 组，一组用于 KNN 学习，一组用于测试。

```
#构建测试组
x1_test = np.random.uniform(0.5, 0.7, 100)
y1_test = np.random.uniform(1.4, 1, 100)

x2_test = np.random.uniform(0.3, 0.5, 100)
y2_test = np.random.uniform(0.6, 1, 100)

x3_test = np.random.uniform(0.4, 0.6, 100)
y3_test = np.random.uniform(0.8, 1.2, 100)

xy_test_normalized =
list(zip(np.concatenate((x1_test, x2_test, x3_test))/x_diff, np.concatenate((y1_test, y2_test, y3_test))/y_diff))

labels_test = [1]*100+[2]*100+[3]*100

#测试组打分
score = clf.score(xy_test_normalized, labels_test)
score

0.84666666666666668

#修改 n_neighbors 参数，继续观察
clf1 = neighbors.KNeighborsClassifier(1)
clf1.fit(xy_test_normalized, labels)
clf1.score(xy_test_normalized, labels_test)
0.80333333333333334
```

结果显示，该模型的学习准确性达到 85% 左右，参考数据量的参数改变并未导致准确度出现大幅变化，因此较为稳定。

第七篇：算法交易入门—VWAP

导语：算法交易，又称订单执行算法，主要用于基金公司、券商等资金量庞大的主体。本篇内容主要向大家介绍标准的订单执行算法—VWAP 算法交易，又称交易量加权平均价格。

算法交易

众所周知，每笔交易都有成本，包括：佣金、税费、市场冲击、滑点等。即使两个策略都买卖同样的股票，也可能因为其控制成本的方法不同，导致其最终收益率不同。尤其对于资金体量较大的投资者，如果每次交易你都能省下成本，投资者组合的表现就会更好。一个明显的推论是，交易次数越多，每次交易时节省成本就显得越重要。而节省交易成本就不可避免的需要接触算法交易，因此对于一个理性的量化交易者，拥有一套能有效节省成本的算法交易是非常重要的。

再举个例子，如果我已经选好股，要大量买入 200 万股，但是单凭交易员操作海量单，这是有点困难的，但是我一下子挂单买 200 万股，则会暴露我的踪迹，最终导致成交成本提高。那么这时候怎样解决拆单，防止冲击成本的问题呢？只有依靠算法交易。

根据各个算法交易中算法的主动程度不同，可以把算法交易分为被动型算法交易、主动型算法交易、综合型算法交易三大类。而 TWAP（时间加权平均价格）、VWAP（成交量加权平均价格）就属于被动型算法交易，也是在日常算法交易中应用最为广泛的策略算法。

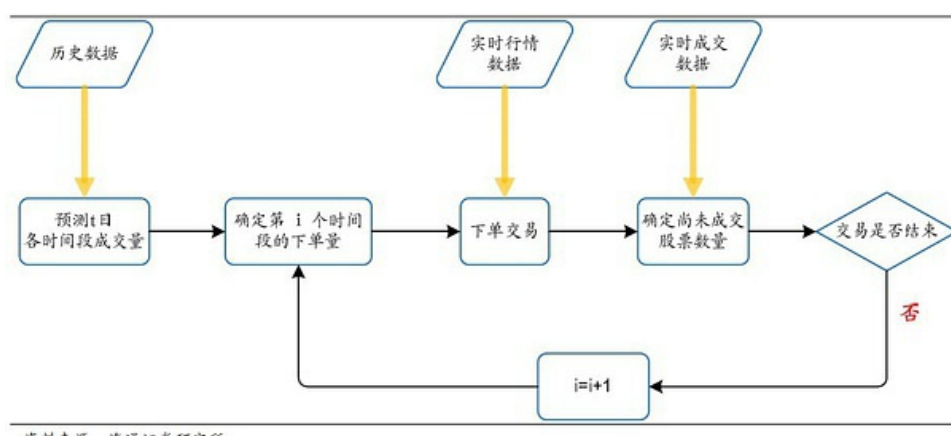
VWAP 算法交易

VWAP 简称成交量加权平均价，是一种拆分大额委托单，在约定时间段内分批执行，使得最终买入或卖出的成交均价尽量接近该段时间内整个市场成交均价的算法交易策略。具体公式如下：

$$VWAP^{(s)} = \frac{x_1 p_1 + x_2 p_2 + \dots + x_N p_N}{x_1 + x_2 + \dots + x_N} = w_1 p_1 + w_2 p_2 + \dots + w_N p_N$$

其中 $w_i = x_i / (x_1 + x_2 + \dots + x_N)$ ，表示第 i 次交易价格按成交量加权的权重。类似的我们也可以计算整个市场某只股票当天成交的 $VWAP^{(m)}$ 。

VWAP 算法交易有四个重要因素：历史成交量，未来的成交量预测、市场动态总成交量，拆单的时间段，整个流程如下图所示：



VWAP 算法交易关键在于有效解决未来成交量分布问题。假设我们已经知道下个交易日的成交量分布，那么只需要根据成交量分布，将大单拆成 N 个小单，并在对应 N 个交易日时段买入即可。通常以 5 分钟为单位，进行一次交易，一天进行 48 次。

那么如何解决未来成交量分布问题？

通常我们根据过去历史 20 个交易日的成交量分布，取各个时间段的均值来推测下个交易日的成交量分布，并结合当日市场动态总成交量与过去 20 日均值的偏差来确定单位时间内的下单量。

最终每 5 分钟的下单量= $\frac{Vp * Vt}{Vall * Vp} * N = \frac{Vt}{Vall} * N$ ，其中 Vt 代表当天交易日任意 5 分钟的交易量，Vp 代表过去 20 个交易日任意五分钟的成交量均值，Vall 代表过去 20 日平均交易量，N 代表当天任意 5 分钟需下单量。

我们构建 VWAP 算法交易，并针对平安银行进行了 5 天算法交易测试，结果如下：

VWAP 算法交易测试	测试结果
测试 1 次	交易日 20180124,执行 VWAP 算法交易,最终成本价为 14.841,市场均价为 14.814,偏差为 0.0018,一共下单量为 1288300,完成总量的 1.2883。
测试 2 次	交易日 20180125,执行 VWAP 算法交易,最终成本价为 14.189,市场均价为 14.181,偏差为 0.0006,一共下单量为 1139700,完成总量的 1.1397。
测试 3 次	交易日 20180126,执行 VWAP 算法交易,最终成本价为 14.230,市场均价为 14.191,偏差为 0.0013,一共下单量为 918600,完成总量的 0.9186。
测试 4 次	交易日 20180129,执行 VWAP 算法交易,最终成本价为 13.944,市场均价为 13.927,偏差为 0.0013,一共下单量为 969200,完成总量的 0.9692。
测试 5 次	交易日 20180130,执行 VWAP 算法交易,最终成本价为 13.703,市场均价为 13.681,偏差为 0.0016,一共下单量为 482300,完成总量的 0.4823。
总结	1.均价偏差度较小，突出 VWAP 算法的优势 2.根据当天市场的交易活跃度，适当在下单目标量上进行扩充或减弱

VWAP 算法交易的 python 代码

```
import pandas as pd
import numpy as np
#=====初始化账户=====
def initialize(account):
    #设置要交易的证券(000001.SZ 平安银行)
    g.security = '000001.SZ'
    g.N = 1000000 #设置当天需下单总量, 100 万股
    g.volume_mean = 0 #设置过去 20 日成交量均值
#=====盘前运行=====
def before_trading_start(account,data):
    volume = history(g.security, ['volume'], 20, '1d', False, 'pre', is_panel=1)
    g.volume_mean = volume.values.mean()
#设置买卖条件, 每个交易频率(日/分钟/tick)调用一次
def handle_data(account,data):
    date = get_datetime().strftime('%H%M%S')
    if date == '093000':
        pass
    else:
        if date[3] == '5' or date[3] == '0':#每 5 分钟交易一次
            volume = history(g.security, ['volume'], 5, '1m', False, 'pre', is_panel=1).sum().values
            VWAP=(volume/g.volume_mean)*g.N
            order(g.security,VWAP[0])
def after_trading_end(account,data):
    date=get_datetime().strftime('%Y%m%d')
    avg_price2 = account.positions[g.security]['cost_basis']
    avg_price1 = get_price(g.security, None, date, '1d', ['avg_price'], False, 'pre', 1,
is_panel=1).values[0][0]
    t=(avg_price2-avg_price1)/avg_price1
    log.error('交易日 {}, 执行 VWAP 算法交易, 最终成本价为 {}, 市场均价为 {}, 偏差为
{}'.format(date,avg_price2,avg_price1,t))
    pass
```

第八篇：Python 实现马克维兹投资组合理论

导语:针对著名的马克维兹投资组合理论，本篇内容深入浅出，完整的将整个理论过程通过python 语言实现。

马克维兹投资组合理论

1952 年,马克维兹发表了题为《投资组合的选择》的论文,首次用数学模型分析投资组合,从而使这项的革命性的科学方法对投资理论产生了重大的影响。资产选择分析的目标是要求出最有效的投资组合集,即投资的有效边界和资产市场线。

在有效证券组合可行域的上边缘部分称为有效边界，也称“马科维兹边界”。

有效边界一定是向外凸的,在它左方的投资组合是不可能的，而位于它右方的投资组合是没有效率的。

因为在有效边界上的投资组合较其右方与之风险相同的投资组合有较高的收益率，较其右方与之收益相同的投资组合有较低的风险。

资本市场线是指表明有效组合的期望收益率和标准差之间的一种简单的线性关系的一条射线。

它是沿着投资组合的有效边界，由风险资产和无风险资产构成的投资组合。

这里将介绍使用均值 - 方差矩阵计算夏普最大组合、方差最小组合以及有效边界和资本市场线

```
import pandas as pd
import numpy as np
import scipy.optimize as sco
import matplotlib.pyplot as plt
```

1. 以下面 5 只股票为例（回测周期：20150101 至 20151231）

```
stock_list = ['600050.SH','000528.SZ','000001.SZ','002007.SZ','300033.SZ']
num = len(stock_list)
df = pd.DataFrame()
```

```
for stock in stock_list:
    df_stock = get_price(stock, '20150101', '20151231', '1d', ['close'], skip_paused = False, fq =
'pre', is_panel = 0)
    df[stock] = df_stock['close']
```

```
df.head(1)
```

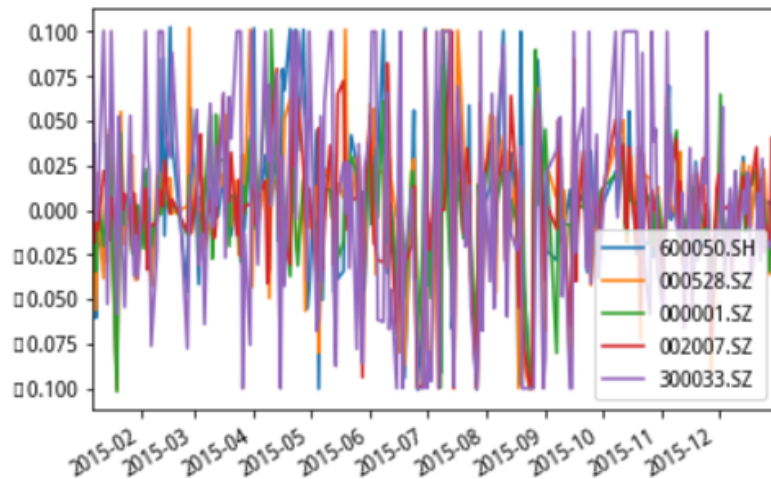
	600050.SH	000528.SZ	000001.SZ	002007.SZ	300033.SZ
2015-01-05	5.19	12.65	13.21	32.39	22.76

2. 计算股票每日收益率和协方差矩阵

```
df_returns = df.pct_change().dropna()  
df_returns.head(2)
```

```
        600050.SH  000528.SZ  000001.SZ  002007.SZ  300033.SZ  
2015-01-06 -0.017341 -0.048221 -0.015140  0.052794  0.100176  
2015-01-07  0.037255  0.006645 -0.019216 -0.018475  0.042732
```

```
df_returns.plot()
```



```
df_returns.mean() * 250
```

```
600050.SH    0.382423  
000528.SZ   -0.224440  
000001.SZ    0.002095  
002007.SZ    0.477311  
300033.SZ    1.640438  
dtype: float64
```

```
df_returns.cov() * 250 # 取一年 250 个交易日
```

```
        600050.SH  000528.SZ  000001.SZ  002007.SZ  300033.SZ  
600050.SH  0.408220  0.253687  0.168676  0.136914  0.249341  
000528.SZ  0.253687  0.416757  0.153297  0.208305  0.278565  
000001.SZ  0.168676  0.153297  0.203456  0.092488  0.129261  
002007.SZ  0.136914  0.208305  0.092488  0.322624  0.295975  
300033.SZ  0.249341  0.278565  0.129261  0.295975  0.941120
```

3. 计算组合的年化收益率、标准差和夏普比

先给每只股票分配随机权重

```
weights = np.random.random(num)
weights /= np.sum(weights)
weights

array([ 0.44376414, 0.26611341, 0.10039381, 0.16774609, 0.02198255])

value1 = np.dot(df_returns.mean(), weights) * 250
print('组合年化收益率: ' + str(value1.round(4)))

组合年化收益率: 0.2263

value2 = np.sqrt(np.dot(weights, np.dot(df_returns.cov(), weights))) * 250
print('组合年化标准差: ' + str(value2.round(4)))

组合年化标准差: 0.5075

value3 = (value1 - 0.04) / value2
print('组合夏普比: ' + str(value3.round(2)))

组合夏普比: 0.37
```

4. 求解夏普比最大的投资组合

```
def portfolio_stat(weights):
    risk_free_rate = 0.04
    weights = np.array(weights)
    port_return = np.dot(df_returns.mean(), weights) * 250
    port_std = np.sqrt(np.dot(weights, np.dot(df_returns.cov(), weights.T))) * 250
    return np.array([port_return, port_std, (port_return - risk_free_rate)/port_std])

# 因为 scipy.optimization 只有最小化函数，所以夏普比需要变成负值
def min_sharpe(weights):
    return -(portfolio_stat(weights)[2])

# 约束条件 1: 所有权重的总和为 1
cons = ({'type':'eq', 'fun':lambda x: np.sum(x)-1})

# 约束条件 2: 权重应限制在 0 和 1 之间。这些值以多个元组组成的一个元组形式提供给
bnds = tuple((0,1) for x in range(num))

# 优化函数调用中忽略的唯一输入是起始参数列表(对权重的初始猜测)。我们使用等权分布。
opts = sco.minimize(min_sharpe, num*[1./num,], method = 'SLSQP', bounds = bnds, constraints =
cons)
opts
```

```

    fun: -1.6497425960291425
    jac: array([ 0.04287983, 0.71966536, 0.22442886, 0.02681652, -0.04123227])
message: 'Optimization terminated successfully.'
    nfev: 14
    nit: 2
    njev: 2
    status: 0
success: True
    x: array([ 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
    5.55111512e-17, 1.00000000e+00])

```

得到的最优组合结果

```

print('权重: ')
print(opts['x'].round(4))
print()
print('组合的收益率、标准差和夏普比: ')
print(portfolio_stat(opts['x']))

```

```

权重:
[ 0.  0.  0.  0.  1.]

```

```

组合的收益率、标准差和夏普比:
[ 1.64043761  0.97011353  1.6497426 ]

```

5. 求解标准差最小的投资组合

#但是我们定义一个函数对 方差进行最小化

```

def min_std(weights):
    return portfolio_stat(weights)[1]

```

```

optv = sco.minimize(min_std, num*[1./num,],method = 'SLSQP', bounds = bnds, constraints =
cons)
optv

```

```

    fun: 0.40870394126925574
    jac: array([ 0.40821013, 0.4263164 , 0.40859295, 0.40898372, 0.45602276])
message: 'Optimization terminated successfully.'
    nfev: 49
    nit: 7
    njev: 7
    status: 0
success: True
    x: array([ 3.44582749e-02, 0.00000000e+00, 6.47754037e-01,
    3.17787688e-01, 2.76471554e-18])

```

得到的标准差最小的组合结果

```
print('权重: ')
print(optv['x'].round(4))
print()
print('组合的收益率、标准差和夏普比: ')
print(portfolio_stat(optv['x']))
```

```
权重:
[ 0.0345  0.    0.6478  0.3178  0.   ]
```

```
组合的收益率、标准差和夏普比:
[ 0.16621806  0.40870394  0.30882516]
```

6. 投资组合的有效边界和资本市场线

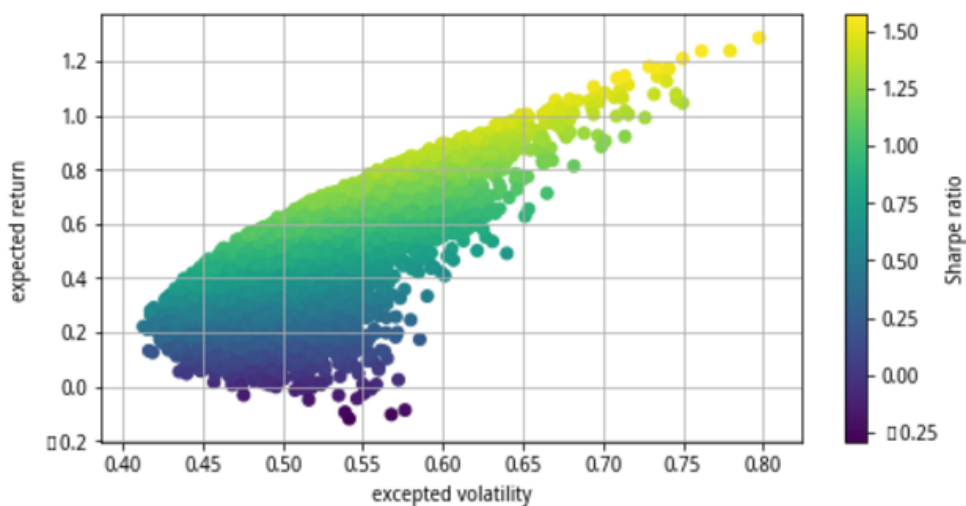
首先观察蒙特卡洛的模拟结果

有效边界为所有可能组合的上半边界，另外可以看出最左点应为（0.43, 0.32）附近

```
port_returns = []
port_std = []
for p in range(10000):
    weights = np.random.random(num)
    weights /= np.sum(weights)
    port_returns.append(np.dot(df_returns.mean(), weights) * 250)
    port_std.append(np.sqrt(np.dot(weights, np.dot(df_returns.cov(), weights.T)) * 250))
```

```
port_returns = np.array(port_returns)
port_std = np.array(port_std)
```

```
plt.figure(figsize = (8,4))
plt.scatter(port_std, port_returns, c=(port_returns - 0.04)/port_std, marker = 'o')
plt.grid(True)
plt.xlabel('excepted volatility')
plt.ylabel('expected return')
plt.colorbar(label = 'Sharpe ratio')
```



计算有效边界

```
# 在不同目标收益率水平（target_returns）循环时，最小化的一个约束条件会变化。
frontier_returns = np.linspace(0.32, 1.4, 50)
frontier_std = []
for tar in frontier_returns:
    cons = ({'type':'eq','fun':lambda x:portfolio_stat(x)[0]-tar}, {'type':'eq','fun':lambda
x:np.sum(x)-1})
    res = sco.minimize(min_std, num*[1./num,],method = 'SLSQP', bounds = bnds, constraints =
cons)
    frontier_std.append(res['fun'])

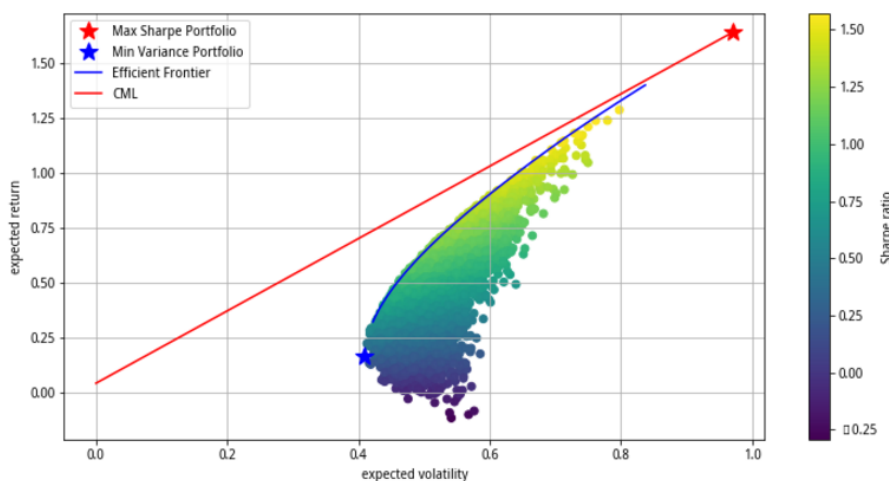
frontier_std = np.array(frontier_std)
```

资本市场线为无风险资产与夏普最优组合的连线

```
plt.figure(figsize = (12,6))
# 圆圈：蒙特卡洛随机产生的组合分布
plt.scatter(port_std, port_returns, c = (port_returns-0.04)/port_std,marker = 'o')
# 红星：标记最高 sharpe 组合
plt.plot(portfolio_stat(opts['x'])[1], portfolio_stat(opts['x'])[0], 'r*', markersize = 15.0)
# 蓝星：标记最小方差组合
plt.plot(portfolio_stat(optv['x'])[1], portfolio_stat(optv['x'])[0], 'b*', markersize = 15.0)
# 蓝线：有效边界
plt.plot(frontier_std, frontier_returns, 'b')
# 红线：资本市场线
plt.plot([0, portfolio_stat(opts['x'])[1]], [0.04, portfolio_stat(opts['x'])[0]], 'r')

plt.legend(['Max Sharpe Portfolio', 'Min Variance Portfolio', 'Efficient Frontier', 'CML'])

plt.grid(True)
plt.xlabel('expected volatility')
plt.ylabel('expected return')
plt.colorbar(label = 'Sharpe ratio')
```



根据有效边界理论，资本市场线上的组合是最优组合。如果可以以无风险利率贷款，那么资本市场线的延长线也可以供投资者选择。

第九篇：隐马尔科夫模型【机器学习+数据挖掘】

导语：作为现在最为热门的一项技术，机器学习以及数据挖掘变得逐渐令人熟知。隐马尔科夫模型(HMM)作为机器学习中的一个重要分支已经被人所广泛应用。著名数学家西蒙斯成立的大奖章基金纯粹采用技术量化方法进行投资，成立以来到 2008 年，大奖章基金的平均年度净回报是 35.6%。HMM 是大奖章基金一个重要的组成部分。本文首先就 HMM 进行介绍，并且对于 HMM 和股票市场的共性做出分析，最后写出 HMM 多因子择股的策略。

隐马尔科夫模型：

HMM 是统计模型，它用来描述一个含有隐含未知参数的马尔可夫过程。HMM 包含一个可观测序列以及一个隐藏状态序列，比如说我不知道这个地区是不是富裕，但是我根据我所看到的比如这个地区高楼的数量，汽车的数量等从这些变量中我可以推测出来这个地区是不是富裕，这样是否富裕就是一种隐藏状态，而高楼汽车的数量则是一个观察变量。

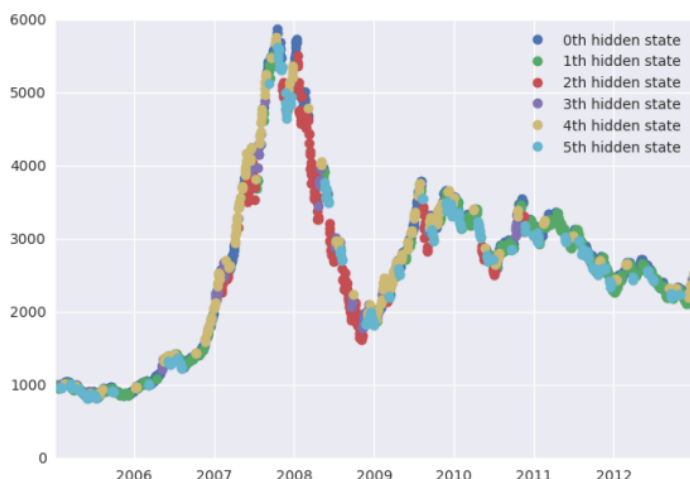
HMM 是一个典型的黑箱模型，但是金融市场很多也是同样的黑箱模型，比如说在金融市场特别是股票市场中，我们并不知道现在市场是不是在牛市中或者是熊市当中，我们可以将这个看做隐藏状态，然后我们可以得到一系列可观察的数据，比如说成交量，收益率，成交量差等当做观察序列对市场隐藏状态进行预测，并且根据现在市场的状态决定是否进行买卖操作，这样很明显我们构造出了一个隐马尔科夫模型。但是隐马尔科夫模型也有一个很大的缺点就是我们并不能直接知道哪个状态是买入状态，哪个状态是卖出状态，所以我们要对每个状态的收益进行比较从而得出买入和卖出的结论。

模型测试：

对中国的沪深 300 市场进行了 HMM 的训练集数据提取以及训练样本的测试，时间从 2005/01/01-2013/01/01。下图为各个状态与沪深 300 走势的拟合情况：

隐藏状态数：6

特征向量：一日对数收益率，五日对数收益率，和五日成交量对数差。



从图中我们可以看出，黄色的点明显都处于上升之中而红色的点基本都在下降之中出现，但是我们需要更加直观的观测出每个状态所代表的意义，所以我们对每个状态下的收益进行计算，结果如下图：



从图中能够清楚的看出 0,3 状态是明显的牛市状态，状态 2,5 是明显的熊市状态，状态 1 是一个震荡状态。所以我们可以很容易的得到一个策略，在状态 0,3 是进行开仓操作，状态 2,5 时如果有仓位则进行平仓，状态 1,4 我们认为是震荡状态，不进行操作。

但是由于 HMM 是一种概率模型，也就是说不是每次数据集回测的时候 0,3 状态永远表现最好，所以我们每次选择表现最好的两个状态作为买入状态而表现最差的两个状态作为卖出状态。

此外由于 HMM 对于特征值的输入需要时一组序列，所以我们将回测期进行循环合成组成序列。这样我们就得到了简单的策略。

隐马尔科夫模型策略思想：

第一步：选取沪深 300 中所有股票从 2012/01/01-2015/01/01 之间交易日的一日对数收益率，五日对数收益率，和五日成交量对数差作为观察序列属性。

第二步：假设这些属性值服从正太高斯分布（HMM 的强假设），我们直接利用这些历史数据完成 HMM 的构造，并且计算出其中表现最好的两个状态作为买入状态，其中表现最差的两个状态作为卖出状态

第三步：在回测过程中将当前回测日的特征值和之前训练集特征数据构成一个可观察序列，放入 HMM 中进行状态序列的预测。

第四步：每个回测日取出前一日状态，如果此状态在买入状态中则进行买入沪深 300，如果此状态在卖出状态中而且我们有持仓则卖出沪深 300。

最终结果:

策略回测区间: 2015.08.01-2017.05.31

回测频率: 日级回测

回测资金: 10000000

回测结果: 红色曲线为策略收益率曲线, 蓝色曲线为相应基准指数收益率曲线



策略源代码:

#HMM 择时

"""

策略思想：先选取沪深 300 中所有股票从 2012/01/01-2015/01/01 之间交易日的一日对数收益率，五日对数收益率，和五日成交量对数差作为观察序列属性，假设这些属性值服从正太高斯分布（HMM 的强假设），我们直接利用这些历史数据完成 HMM 的构造，并且计算出其中表现最好的两个状态作为买入状态，其中表现最差的两个状态作为卖出状态，然后在回测过程中将当前回测日的特征值和之前训练集特征数据构成一个可观察序列，放入 HMM 中进行状态序列的预测，每个回测日取出前一日的状态，如果此状态在买入状态中则进行买入沪深 300，如果此状态在卖出状态中而且我们有持仓则卖出沪深 300。

初始资金：1000000

回测频率：每天

回测日期：2015-08-01——2017-05-31

"""

#导入 HMM 模型所需的包 0

from hmmlearn.hmm import GaussianHMM

import datetime

import numpy as np

#导入画图所需要的包

from matplotlib import cm, pyplot as plt

import matplotlib.dates as dates

import pandas as pd

import seaborn as sns

def HMMModel(stklist, start, end, n):

"""获取训练好的 HMM 模型以及买入和卖出状态

参数

stklist : 列表

训练数据包含的股票代码.

start : '%Y%m%d'形式字符串

训练数据的起始日期.

end : '%Y%m%d'形式字符串

训练数据的结束日期.

n : 整数类型

HMM 模型中隐含的状态数.

返回值

model : HMM 模型
训练好的 HMM 模型.

buystates : 列表
买入的状态

sellstates : 列表
卖出的状态

feature : 数组
训练样本的特征数据集

"""

```
df = get_price(stklist, start, end, '1d', ['close', 'volume'], True)
```

```
#计算单日对数收益率
df['1return'] = np.log(df['close']).diff(1)
```

```
#计算五日对数收益率
df['5return'] = np.log(df['close']).diff(5)
```

```
#计算五日交易量对数差
df['volume'] = np.log(df['volume'])
df['5volume'] = df['volume'].diff(5)
```

```
df = df.dropna()
x = df['close']
del df['close']
```

#选取单日对数收益率，五日对数收益率，以及五日交易量对数差作为观测序列的特征因子

```
feature = df.iloc[:, -3:].values
```

#根据训练数据对 HMM 模型进行拟合

```
model = GaussianHMM(n_components= n, covariance_type='diag', n_iter=2000).fit(feature)
```

#通过训练数据计算不同隐含状态的累积收益率，选取前三作为买入状态后三个作为卖出状态

```
hidden_states = model.predict(feature)
```

```
x = pd.DataFrame(x)
```

```
for i in range(n):
```

```
    x['%s%i' % i] = x['close']
```

```
    x['%s%i' % i] = 1
```

```
for i in range(n):
```

```
    for j in range(0, len(x)-1):
```

```
        if hidden_states[j] == i:
```

```

        x.iloc[j+1,i+1] = (x.iloc[j+1,0]/x.iloc[j,0])*x.iloc[j,i+1]
    else:
        x.iloc[j+1,i+1] = x.iloc[j,i+1]
ans = x.iloc[-1,1:]
ans = ans.sort_values()
buystates = ans.index.values[-2:]
sellstates = ans.index.values[:2]
for i in range(len(buystates)):
    buystates[i] = int(buystates[i])
    sellstates[i] = int(sellstates[i])
return model, buystates, sellstates, feature

```

#初始化账户

```
def initialize(account):
```

```

    #设置要交易的证券(000300.SH 沪深 300)
    account.security = '000300.SH'

```

#初始化全局变量

```

    account.clf, account.buy, account.sell, account.feature = HMMModel(account.security,
'20120101', '20150101', 6)

```

#设置买卖条件，每个交易频率（日/分钟/tick）调用一次

```
def handle_data(account,data):
```

#计算测试集特征数据

```

testdf = data.attribute_history(account.security, ['close','volume'], 10, '1d', True)
testdf['1return'] = np.log(testdf['close']).diff(1)
testdf['5return'] = np.log(testdf['close']).diff(5)
testdf['volume'] = np.log(testdf['volume'])
testdf['5volume'] = testdf['volume'].diff(5)
testdf = testdf.dropna()

```

#得到回测日前一日的特征数据

```

feature_test = testdf.iloc[-1,-3:].values
feature_test = feature_test.reshape(1,3)

```

#将特征数据和之前的训练样本数据集结合起来当做观察序列

```
account.feature = np.concatenate((account.feature,feature_test))
```

```
if (account.clf.predict(account.feature)[-1] in account.buy):
```

```
    order_value(account.security,account.cash)
```

```
if account.positions_value > 0:
```

```
    if (account.clf.predict(account.feature)[-1] not in account.buy):
```

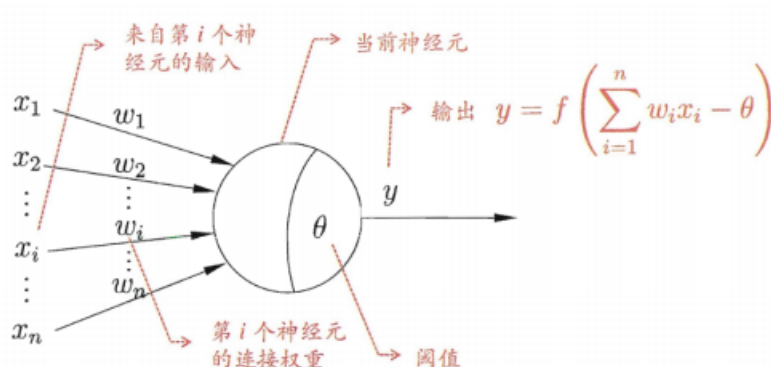
```
        order_target(account.security, 0)
```

第十篇：机器学习之神经网络入门

导语：神经网络是由具有适应性的简单单元组成的广泛并行互连的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的交互反应。神经网络用途极其广泛，本文采用神经网络算法用于金融领域量化选股策略。

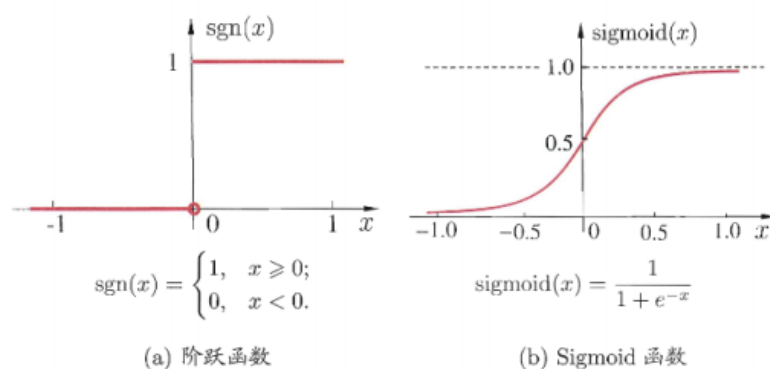
神经网络原理

神经网络中最基本的成分是神经元模型，在生物神经网络中，每个神经元与其他神经元相连，当它“兴奋”时，就会向相连的神经元发送化学物质，从而改变这些神经元的电位；如果某神经元的电位超过了一个“阈值”，那么它就会被激活，即“兴奋”起来，向其他神经元发送化学物质。下图为 M-P 神经元模型：



M-P 神经元模型（图片来源：周志华 - 机器学习）

在这个模型中，神经元接收到来自 n 个其他神经元传递过来的输入信号，这些输入信号通过带权重的连接进行传递，神经元接收到的总输入值将与神经元的阈值进行比较，然后通过“激活函数”处理以产生神经元的输出。



典型的神经元激活函数

理想中的激活函数是上图中的(a)阶跃函数，它将输入值映射为输出值“0”或“1”，显然“1”代表神经元兴奋，“0”代表神经元抑制，但由于其不连续，不光滑的性质，所以实际常用 (b)Sigmoid 函数作为激活函数。把许多个这样的神经元按一定的层次结构连接起来，就得到了神经网络。

基于神经网络的量化选股策略

策略思想：选取一些常用技术指标作为训练样本特征集，对于类别，如果未来 20 个工作日内上涨幅度超过 10% 则标记为 1，否则为 -1，采用神经网络算法进行训练，预测结果为 1 且未持仓则买入，预测结果为 -1 且已持仓则卖出。

特征因子选取：本文采用神经网络算法解决有监督学习的分类问题，特征因子选取了 2015-01-05 那天上证 50 指数成份股的总市值，OBV 能量潮，市盈率，布林线，KDJ 随机指标，RSI 相对强弱指标共 6 个指标。

数据标准化：数据标准化方法有很多，本文采用高斯预处理方法，即每个特征因子减去它对应的均值再除以它的标准差 $((x-x.\text{mean})/x.\text{std})$ 。

最终结果：

策略回测区间：2015.02.03-2017.12.31

回测频率：日级回测

回测资金：1000000

回测结果：红色曲线为策略收益率曲线，蓝色曲线为相应基准指数收益率曲线



策略源代码:

基于神经网络的量化选股策略

"""

策略思想: 选取一些常用技术指标作为训练样本特征集, 对于类别, 如果未来 20 个工作日内上涨幅度超过 5% 则标记为 1, 否则为 -1, 采用神经网络算法进行训练, 预测结果为 1 且未持仓则买入, 预测结果为 -1 且已持仓则卖出。

特征因子选取: 本文采用神经网络算法解决有监督学习的分类问题, 特征因子选取了 2015-01-05 那天沪深 300 成份股的总市值, OBV 能量潮, 市盈率, 布林线, KDJ 随机指标, RSI 相对强弱指标共 6 个指标。

数据标准化: 数据标准化方法有很多, 本文采用高斯预处理方法, 即每个特征因子减去它对应的均值再除以它的标准差 $((x-x.\text{mean})/x.\text{std})$ 。

初始资金: 100000000

回测频率: 每天

回测日期: 2015-03-02 — 2017-03-01

"""

```
import pandas as pd
import numpy as np
from sklearn import preprocessing
from sklearn.neural_network import BernoulliRBM
from sklearn.neural_network import MLPClassifier
def get_factor_data(stock_list, date):
    #获取相应的因子值
    q = query(
        factor.symbol,
        factor.zdzb,
        factor.market_cap,
        factor.obv,
        factor.pe_ttm,
        factor.boll,
        factor.kdj,
        factor.rsi
    ).filter(
        factor.symbol.in_(stock_list),
        factor.date == date
    )
    ans = get_factors(q)
    return ans
def get_date_delta(current_date, delta):
    #获取交易日日期
    date = list(get_trade_days('20050101', '20221231').strftime('%Y-%m-%d'))
    ans = date[date.index(current_date) + delta]
```

```

    return ans
def create_data(date, delta, stock_list):
    df = get_factor_data(stock_list, date)
    #在 df 数据集上生成 label（类别）列，涨幅超过 10%标为 1，反之为-1，注意 lambda 函数的应用
    df['label'] = df['factor_symbol'].map(lambda x: 1 if get_price([x], start_date = date, end_date =
get_date_delta(date, delta), fields = ['open'])[x].open[-1]/get_price([x], start_date = date, end_date =
get_date_delta(date, delta), fields = ['open'])[x].open[0] - 1 >= 0.05 else -1)
    df = df.dropna()
    df = df.set_index('factor_symbol')
    label = list(df['label'])
    del(df['label'])
    return df.values, label
def initialize(account):
    # 基准日期选取 2016-01-05，回测对象：沪深 300 成分股
    account.date = '2015-01-05'
    account.security = get_index_stocks('000016.SH', account.date)
    set_benchmark('000016.SH')
    # 获取训练集及类别
    df, label = create_data(account.date, 20, account.security)
    # 以下两行为数据标准化
    account.scaler = preprocessing.StandardScaler().fit(df)
    df_scaler = account.scaler.transform(df)
    # 神经网络算法模型构建
    account.clf = MLPClassifier(solver='lbfgs', alpha=1e-5,hidden_layer_sizes=(5, 2),
random_state=1)
    account.clf.fit(df_scaler, np.array(label))
def handle_data(account,data):
    # 获取上个交易日日期
    yesterday_date = get_last_datetime().strftime('%Y-%m-%d')
    # 以下三行获取上个交易日的特征因子数据
    factor_data = get_factor_data(account.security, yesterday_date)
    result = factor_data['factor_symbol'].values
    del(factor_data['factor_symbol'])
    # 对预测值为 1 的股票都放入 stocks_buy 列表中
    factor_data=factor_data.dropna()
    stocks_buy = result[account.clf.predict(account.scaler.transform(factor_data.values)) == 1]
    for stock in stocks_buy:
        if stock not in account.positions.keys():
            log.info('buying %s' % stock)
            order_value(stock, account.cash / (len(stocks_buy)+1))
    # 对预测值为-1 的股票都放入 stocks_buy 列表中
    stocks_sell = result[account.clf.predict(account.scaler.transform(factor_data.values)) == -1]
    for stock in stocks_sell:
        if stock in account.positions.keys():
            log.info('selling %s' % stock)
            order_target(stock, 0)

```